МИНОБРНАУКИ РОССИИ
Федеральное государственное бюджетное образовательное учреждение
высшего образования
«МИРЭА – Российский технологический университет»
РТУ МИРЭА

Практическое занятие № 11-12
по дисциплине «Программирование корпоративных систем»

Тема работы: КОМПЛЕКСНАЯ УМНАЯ АВТОМАТИЗАЦИЯ МАШИННОЕ ЗРЕНИЕ И ГЕНЕРАТИВНЫЙ ИИ В RPA

Вариант: 11

Выполнил: ЭФБО-09-23 *(группа)* *(подпись)* Калинин Н.В. *(Фамилия И.О.)*

Принял: доц. *(должность)* *(подпись)* Громов Е.В. *(Фамилия И.О.)*

Москва

ТЕОРЕТИЧЕСКАЯ ЧАСТЬ

В предыдущих практических работах мы освоили базовые операции автоматизации: клики, ввод текста, работу с таблицами и веб-интерфейсами. Это фундамент, на котором строятся любые RPA-решения. Однако реальные бизнес-задачи

редко ограничиваются простыми линейными сценариями, где все элементы доступны по ID или XPath.

В современных RPA-проектах робот сталкивается с более сложными вызовами:

Недоступность элементов интерфейса – в старых приложениях, эмуляторах терминалов и виртуальных рабочих столах (Citrix, RDP) нет программных идентификаторов, по которым можно найти кнопку или поле ввода. Робот не может получить доступ к DOM-модели или UI Automation.

Нестабильные селекторы – даже если элементы доступны, их ID и классы могут меняться при каждом обновлении системы. XPath, который работал сегодня, завтра может стать бесполезным.

Динамический контент – элементы загружаются асинхронно, появляются с задержкой, лоадеры скрывают кнопки, сообщения об ошибках возникают неожиданно.

Неструктурированные данные – роботу приходится обрабатывать сканы документов, фотографии, PDF-файлы без текстового слоя, электронные письма произвольного формата.

Нештатные ситуации – ошибки, сбои, таймауты, требующие интеллектуальной обработки и принятия решений в условиях неопределённости.

Эти вызовы и составляют содержание нашей одиннадцатой практической работы. Мы переходим от уровня «базового пользователя» к уровню «профессионального разработчика», способного создавать надежные, отказоустойчивые и интеллектуальные RPA-решения для любых интерфейсов – даже самых старых и недружелюбных.

Когда классического RPA недостаточно

Классический RPA эффективно работает со структурированными данными и интерфейсами, где у каждого элемента есть уникальный программный идентификатор. Однако существует целый класс задач, где традиционные методы автоматизации бессильны.

Проблема первая: отсутствие доступа к элементам интерфейса

Многие компании до сих пор используют старые приложения, разработанные десятилетия назад. Эти приложения не предоставляют программного интерфейса для доступа к их элементам. Они не поддерживают технологии UI Automation, не имеют DOM-модели, не генерируют события, на которые можно подписаться. Классический RPA просто не видит элементы в таких системах – для него это просто картинка.

Проблема вторая: виртуальные рабочие столы и эмуляторы терминалов

Среды удалённого доступа – Citrix, RDP, VNC, эмуляторы терминалов – передают только изображение рабочего стола. Робот, работающий внутри такой среды, не имеет прямого доступа к элементам интерфейса удалённого приложения. Он видит только картинку. Это как пытаться автоматизировать взаимодей-

ствие с компьютером, глядя на его монитор через веб-камеру – вы видите, что происходит, но не можете программно обратиться к кнопкам.

Проблема третья: неструктурированные данные

Входящие документы редко бывают идеально структурированными таблицами. Чаще всего это сканы договоров, фотографии накладных, PDF-файлы без текстового слоя, электронные письма в свободном формате. Робот не может просто считать ячейку Excel – ему нужно сначала распознать текст на изображении, понять его смысл, извлечь нужные данные.

Проблема четвёртая: изменяющиеся интерфейсы

Даже в современных веб-приложениях интерфейс может меняться. Разработчики обновляют систему, переименовывают кнопки, меняют расположение полей. Классический XPath, привязанный к конкретным ID или структуре DOM, перестаёт работать. Робот ломается, и его приходится вручную перенастраивать.

Проблема пятая: недетерминированность процессов

Классический RPA требует чёткого описания всех шагов: нажать сюда, ввести туда, прочитать оттуда. Но многие бизнес-процессы не являются детерминированными. Документ может быть счётом, накладной или договором – в зависимости от типа нужно выполнять разные действия. Робот должен сам определить, что перед ним, и выбрать правильную ветку алгоритма.

Именно для решения этих проблем и нужны технологии искусственного интеллекта и компьютерного зрения.

Что такое машинное зрение в RPA

Представьте, что вы учите робота смотреть на экран компьютера так же, как смотрит человек. Вы не объясняете ему, что кнопка «Отправить» имеет вот такой ID в коде страницы или вот такой XPath. Вы просто показываете ему эту кнопку один раз – говорите: «Видишь эту серую прямоугольную область с надписью "Отправить"? Вот она. Найди её на экране и нажми». И робот понимает. Это и есть машинное зрение в RPA. В отличие от классических методов автоматизации, которые взаимодействуют с элементами через их программные идентификаторы – ID, классы, XPath, атрибуты name – машинное зрение работает с графическим образом элемента. Робот не спрашивает у операционной системы «Где находится элемент с таким-то ID?». Он делает скриншот экрана и самостоятельно ищет на нём нужную картинку, так же как человек ищет знакомый значок на рабочем сто- ле.

Процесс начинается с подготовки эталона. Разработчик один раз делает скриншот того элемента, с которым робот будет работать. Это может быть кнопка, поле ввода, иконка, логотип, любой визуальный объект. Этот скриншот сохраняется в ресурсах проекта – он становится эталоном, тем, что робот будет искать. В момент выполнения робот захватывает текущее состояние экрана – делает скриншот всего рабочего стола или указанной области. Затем в дело вступает алгоритм сопоставления шаблонов. Он последовательно сканирует скриншот, накладывая эталонное изображение на различные области, и вычисляет степень

схожести. Когда алгоритм находит область с достаточной степенью совпадения, он возвращает координаты этой области. Дальше робот может сделать с этими координатами всё что угодно: кликнуть, навести мышь, ввести текст в определённую область, перетащить элемент из одной точки в другую.

Классический RPA – замечательная технология, но у неё есть слепые зоны. Есть целые классы приложений, где стандартные методы поиска элементов просто не работают. Старые программы, которые были написаны десять, двадцать, а то и тридцать лет назад, часто не предоставляют никакого программного интерфейса для доступа к их элементам. Они не поддерживают современные технологии автоматизации, у них нет DOM-модели, они не генерируют события, на которые можно подписаться. Для классического RPA такие приложения – чёрный ящик. Он видит только картинку, но не может «потрогать» отдельные кнопки и поля. Машинное зрение решает эту проблему – робот просто смотрит на экран и находит нужные элементы визуально, так же как это делает человек-оператор.

Виртуальные рабочие столы – Citrix, RDP, VNC – создают ещё одну серьёзную проблему. Когда вы подключаетесь к удалённому компьютеру через такие системы, ваш робот работает внутри этой сессии. Но он не имеет прямого доступа к элементам интерфейса удалённого приложения. Всё, что он получает – это поток изображений, видеокартинка того, что происходит на удалённой машине. Классический RPA в таких условиях бессилен – он просто не видит элементов, потому что их программные идентификаторы не передаются через удалённое соединение. Машинное зрение позволяет роботу взаимодействовать с этой картинкой так же, как это делает человек: посмотрел, увидел кнопку, нажал на неё.

Даже в современных веб-приложениях бывают проблемы, которые решаются машинным зрением. Некоторые системы генерируют случайные ID для элементов при каждой загрузке страницы. Сегодня кнопка имеет один ID, завтра – совершенно другой. Классический XPath, привязанный к этим ID, перестаёт работать после обновления страницы. Разработчики могут менять структуру DOM, переименовывать классы, добавлять новые обёртки. Но внешний вид кнопки – её цвет, форма, размер, текст на ней, расположение – обычно остаётся прежним. Машинное зрение смотрит именно на внешний вид, поэтому оно значительно устойчивее к таким изменениям.

Когда робот ищет картинку на экране, он не требует стопроцентного совпадения. Это было бы слишком жёстко – достаточно чуть-чуть изменить размер окна или разрешение экрана, и пиксели уже не совпадут. Вместо этого разработчик задаёт параметр точности – число от нуля до единицы, которое определяет, насколько похожими должны быть эталон и найденная область. Точность 0.99 означает, что робот требует почти идеального совпадения. Точность 0.8 – стандартное значение для большинства сценариев, робот найдёт кнопку, даже если её цвет немного изменился или шрифт слегка отличается. Точность 0.7 используется в сложных условиях, когда эталон и реальное изображение могут сильно различаться – например, при работе на разных разрешениях экрана или с разными темами оформления. Высокая точность снижает риск ложных срабатываний, но по-

вышает риск того, что робот вообще ничего не найдёт. Низкая точность увеличивает шанс найти элемент, но может привести к тому, что робот кликнет не туда, где похожая, но не та картинка.

У машинного зрения есть и ограничения, о которых важно знать. Поиск по картинке занимает время – от половины секунды до нескольких секунд в зависимости от размера экрана и эталона. Это медленнее, чем классический поиск по XPath, который работает практически мгновенно. Машинное зрение зависит от разрешения экрана, масштаба отображения, цветовой гаммы монитора. Эталон, созданный на мониторе с высоким разрешением, может не найтись на экране с низким разрешением. Поэтому эталонные изображения рекомендуется создавать на том же компьютере и в тех же условиях, где будет работать робот.

Но главное правило использования машинного зрения звучит так: это не замена классическому поиску, а его усиление для сложных сценариев. Оптимальная стратегия – комбинированный подход. Сначала робот пытается найти элемент классическими методами – по XPath, ID, CSS-селектору. Это быстро и надёжно, если селекторы стабильны. Если классический поиск не удался – например, элемент не найден или истекло время ожидания, – робот подключает машинное зрение. Он ищет элемент по картинке и, если находит, выполняет действие. Если и машинное зрение не помогло – робот передаёт задачу оператору на ручную обработку. Такая многоступенчатая стратегия делает робота по-настоящему устойчивым и готовым к любым неожиданностям.

Три базовые операции компьютерного зрения

Все сценарии машинного зрения в RPA строятся на трёх простых операциях, как дом строится из кирпичей. Эти три действия – поиск, клик и ожидание – в разных комбинациях покрывают практически любую задачу визуальной автоматизации. Освоив их, вы сможете заставить робота работать в любом приложении, даже самом старом и недружелюбном.

Первая операция – это поиск изображения на экране. Робот получает на вход эталонную картинку – например, скриншот кнопки «Сохранить», которую вы заранее подготовили. Он начинает сканировать экран, последовательно сравнивая эталон с разными областями. Это похоже на то, как если бы вы искали знакомое лицо на групповой фотографии: вы перемещаете взгляд по картинке, пока не находите то, что ищете. Когда робот находит область с достаточной степенью схожести – заданной параметром точности, – он возвращает координаты этой области: X и Y в пикселях. Если изображение не найдено, он возвращает признак неудачи. Эта операция нужна, когда роботу нужно не просто кликнуть, а сначала получить координаты элемента для сложных манипуляций. Например, вам нужно перетащить файл из одной папки в другую – для этого робот сначала находит иконку файла по картинке, запоминает её координаты, затем находит иконку папки назначения, запоминает её координаты, и только потом выполняет перетаскивание из первой точки во вторую. Или, например, нужно ввести текст в определённое поле, которое не имеет программного идентификатора – робот находит это

поле по картинке, получает его координаты и кликает туда, чтобы установить курсор, а потом уже вводит текст с клавиатуры.

Вторая операция – клик по изображению. Это самое востребованное дей- ствие в машинном зрении, потому что в большинстве случаев роботу нужно про- сто нажать на какую-то кнопку. Блок «Клик по картинке» объединяет поиск и клик в одном действии. Робот находит эталонное изображение на экране, вычис- ляет его центр, перемещает туда курсор мыши и выполняет клик. Можно настро- ить разные типы взаимодействия: левый клик – самый частый вариант, правый клик – для вызова контекстного меню, двойной клик – для открытия файлов или папок, наведение мыши без клика – для активации выпадающих меню. Также можно задать смещение – кликнуть не в центр найденного изображения, а, ска- жем, на десять пикселей правее и на пять пикселей выше. Это бывает нужно, ко- гда эталонная картинка слишком большая, а кликнуть нужно в конкретное место внутри неё. Клик по картинке применяется в огромном количестве сценариев: нажатие на кнопки в старых программах, открытие приложений по их иконкам на рабочем столе, выбор значений из выпадающих списков, закрытие модальных окон, активация вкладок в интерфейсе. Это та операция, которую вы будете ис- пользовать чаще всего.

Третья операция – ожидание изображения на экране. В реальных приложени- ях элементы появляются не мгновенно. После нажатия кнопки «Рассчитать» мо- жет пройти несколько секунд или даже минут, прежде чем появится результат. При загрузке программы окно открывается не сразу. Всплывающие уведомления возникают с задержкой. Классический подход – поставить фиксированную паузу на десять секунд – работает плохо: если операция завершилась за две секунды, робот будет пять секунд просто ждать впустую; если операция заняла пятнадцать секунд, робот продолжит работу раньше времени и наткнётся на ошибку. Ожида- ние изображения решает эту проблему. Робот начинает циклически проверять наличие эталонной картинки на экране. Между проверками он делает небольшую паузу – обычно 200-500 миллисекунд, – чтобы не перегружать процессор. Поиск продолжается либо до успешного нахождения изображения, либо до истечения заданного таймаута – максимального времени ожидания. Если изображение по- явилось, робот сразу же замечает это и переходит к следующему шагу, не теряя ни секунды. Если изображение так и не появилось за отведённое время, робот фиксирует ошибку и действует по сценарию обработки исключений. Ожидание изображения используется повсеместно: проверка загрузки главного окна прило- жения после его запуска, ожидание завершения длительной операции (когда про- падает лоадер или появляется надпись «Готово»), контроль появления модальных окон подтверждения перед тем, как нажать кнопку «ОК» или «Отмена», ожидание появления результатов поиска в интерфейсе.

Эти три операции редко используются по отдельности. Чаще всего они ком- бинируются в последовательности. Типичный сценарий выглядит так: сначала робот ожидает появления главного окна приложения – использует операцию ожи- дания с эталоном этого окна. Когда окно загрузилось, он находит поле ввода ло-

гина по картинке и кликает туда – комбинация поиска и клика. Затем вводит логин с клавиатуры. Потом находит поле ввода пароля, кликает, вводит пароль. Затем находит кнопку «Вход» и кликает по ней. После этого ожидает появления главного меню – снова операция ожидания. И только потом приступает к основной работе. Такая комбинация трёх базовых операций позволяет автоматизировать вход в любое приложение, даже если у него нет ни одного программного идентификатора, а есть только картинка на экране.

Важно понимать, что каждая из этих операций может завершиться неудачей. Изображение может не найтись – например, если окно приложения не загрузилось, если эталон устарел или если на экране появилось неожиданное окно с ошибкой. Поэтому при построении сценариев с машинным зрением всегда нужно предусматривать обработку этих ситуаций. Что делать, если изображение не найдено? Попробовать ещё раз с меньшей точностью? Сделать скриншот экрана для последующего анализа? Передать управление оператору? Завершить робота с понятной ошибкой? Ответ зависит от конкретного бизнес-процесса и цены ошибки. Но главное правило остаётся неизменным: устойчивый робот – это не тот, который никогда не ошибается, а тот, который видит ошибку и знает, что с ней делать.

Параметр точности поиска

Когда вы ищете знакомое лицо в толпе, вы не требуете стопроцентного совпадения с фотографией в паспорте. Вы узнаёте человека по общим чертам: форме лица, причёске, походке, манере одеваться. Вы допускаете, что сегодня он в другой куртке или не выспался и выглядит иначе. Примерно так же работает машинное зрение в RPA. Робот не требует, чтобы эталонная картинка совпадала с областью на экране пиксель в пиксель. Вместо этого разработчик задаёт параметр точности – число, которое определяет, насколько похожими должны быть изображения, чтобы считаться совпадением. И от того, какое число вы поставите, зависит, найдёт ли робот нужный элемент или пройдёт мимо.

Параметр точности задаётся в диапазоне от нуля до единицы, где единица – это сто процентов совпадения. Но на практике никто не ставит единицу. Почему? Потому что в реальном мире всё постоянно меняется. Сегодня вы сделали скриншот кнопки на своём мониторе с разрешением 1920 на 1080. Завтра робот запускается на компьютере коллеги, у которого монитор 1366 на 768 – кнопка на экране стала чуть меньше. Послезавтра система обновилась, и у кнопки изменил- ся цвет с серого на светло-серый. На следующей неделе разработчики поменяли шрифт с Arial на Calibri. При точности 0.99 робот не найдёт кнопку ни в одном из этих случаев, потому что пиксели перестали совпадать. А при точности 0.8 – вполне возможно, найдёт. Потому что 0.8 означает, что роботу достаточно восьмидесятипроцентного сходства.

Как выбрать правильную точность? Это зависит от конкретной ситуации, но есть общие рекомендации. Если ваш робот будет работать всегда на одном и том же компьютере, с одним и тем же разрешением экрана, в одной и той же операционной системе, и интерфейс приложения никогда не меняется – можно поставить

высокую точность, 0.95 или даже 0.98. Робот будет работать быстро и точно, риск ложных срабатываний минимален. Но в реальной жизни такое бывает редко. Чаще всего робот запускается на разных компьютерах, пользователи могут менять масштаб интерфейса, администраторы обновляют системы. Для таких условий стандартным выбором является точность 0.8. Это золотая середина: робот найдёт кнопку, даже если её цвет слегка изменился или шрифт стал чуть жирнее, но при этом он вряд ли перепутает нужную кнопку с похожей, но другой.

Бывают ситуации, когда приходится снижать точность до 0.7 или даже 0.65. Например, если робот работает на разных разрешениях экрана и эталонная картинка была сделана на одном разрешении, а поиск происходит на другом. Или если вы ищете элемент на скриншоте с низким качеством – скажем, на фотографии экрана, сделанной на телефон. Или если интерфейс приложения сильно изменился, но вы ещё не успели обновить эталоны. В этих случаях снижение точности может спасти ситуацию – робот найдёт элемент там, где с высокой точностью прошёл бы мимо. Но будьте осторожны: чем ниже точность, тем выше риск ложных срабатываний. Робот может принять совсем другую кнопку за ту, что вы ищете, если она отдалённо похожа на эталон. Например, если вы ищете красную круглую кнопку «Стоп», а на экране есть красный круглый логотип – при точности 0.6 робот может кликнуть по логотипу, думая, что это нужная кнопка.

Как понять, что точность выбрана неправильно? Есть два симптома. Первый: робот часто не находит элемент, который точно есть на экране. Значит, точность слишком высокая, нужно её снизить. Второй: робот кликает не туда, где нужно, ошибается, путает элементы. Значит, точность слишком низкая, и робот принимает чужие картинки за свои. Идеальная точность – это баланс между двумя этими крайностями.

Опытные разработчики RPA часто используют приём адаптивной точности. Сначала робот пытается найти элемент с высокой точностью – 0.9. Если не нашёл, снижает точность до 0.85 и пробует снова. Потом до 0.8, до 0.75, до 0.7. Это повышает устойчивость робота: он начинает с максимально точного поиска, а если не получается, постепенно смягчает требования, пока не найдёт элемент или не достигнет минимально допустимого порога. При этом в лог записывается, при какой точности элемент был найден. Если вы заметили, что робот регулярно находит элемент только при точности 0.7, это сигнал: пора обновить эталонные изображения, потому что интерфейс изменился.

И последнее важное замечание. Точность поиска – это не единственный способ настройки машинного зрения. Если робот никак не может найти элемент, не торопитесь просто снижать точность. Сначала попробуйте сделать новый, более качественный эталон. Обрежьте картинку, оставив только самую уникальную часть элемента – так роботу будет легче её найти. Уберите фон, который может меняться. Добавьте на эталон небольшую рамку вокруг элемента, чтобы алгоритм лучше понимал границы. Иногда лучше потратить пять минут на создание хоро- шего эталона, чем потом неделю отлаживать робота, который кликает не туда из-

за слишком низкой точности. Параметр точности – это мощный инструмент, но как любой инструмент, он требует осознанного подхода.

Что такое языковые модели (LLM) в RPA

Представьте, что ваш робот умеет не только кликать по кнопкам и копиро- вать данные из таблиц, но ещё и читать, и понимать тексты, и даже писать их. Он может получить письмо от клиента, прочитать его, понять, о чём идёт речь, и написать вежливый ответ. Он может прочитать длинный договор и кратко пере- сказать его суть. Он может посмотреть на груду входящих документов и опреде- лить, какие из них счета, какие накладные, а какие акты выполненных работ. Всё это становится возможным благодаря большим языковым моделям, которые встраиваются в RPA-роботов.

Большие языковые модели – это нейросети, которые обучены на огромном количестве текстов. Они не просто заучивают слова и правила грамматики, они улавливают смысл, контекст, связи между понятиями. Самые известные примеры – это ChatGPT от компании OpenAI, YandexGPT от Яндекса, GigaChat от Сбера, а также встроенный PuzzleGPT в платформе Puzzle RPA. Эти модели умеют отвечать на вопросы, писать письма, переводить с языка на язык, выделять из текста ключевые сущности – даты, суммы, имена, номера документов, – определять тональность сообщения, то есть понимать, написал клиент жалобу, вопрос или благодарность, и даже кратко пересказывать длинные документы.

Как это работает в RPA? Робот получает какую-то текстовую информацию – это может быть входящее письмо, скан документа, обращение в чат-бот, внутренняя служебная записка. Он передаёт этот текст в языковую модель вместе с инструкцией: «Определи тип этого документа», или «Выдели из этого текста дату и сумму», или «Напиши вежливый ответ клиенту». Модель анализирует текст, выполняет задание и возвращает результат. Всё взаимодействие происходит через API – робот отправляет запрос и получает ответ. Ему не нужно знать, как устрое- на нейросеть внутри, какие у неё слои и веса. Достаточно просто позвать её и передать задание.

Раньше, до появления доступных языковых моделей, разработчикам RPA приходилось решать эти задачи очень сложными и несовершенными способами. Чтобы определить тип документа, нужно было прописывать сотни правил: если в тексте встречается слово «счёт» или «оплата» или «ИНН» – то это, вероятно, счёт. Но люди пишут по-разному. Кто-то напишет «прошу выставить счёт», кто-то – «необходимо оплатить», кто-то – «отправьте, пожалуйста, платёжные реквизиты». Никакой набор правил не охватит всё разнообразие живого языка. Языковая модель решает эту проблему кардинально: она понимает смысл, а не ищет ключевые слова. Вы просто говорите ей: «Определи тип документа», и она делает это, даже если документ написан нестандартно, с ошибками или в необычном стиле.

Особого внимания заслуживает технология RAG – Retrieval-Augmented Generation. Это сложное название на практике означает простую и очень полез- ную вещь: вы можете научить языковую модель отвечать на вопросы, используя ваши собственные документы. Вы загружаете в PuzzleGPT файл – это может быть

Word-документ с внутренней инструкцией, PDF-файл с регламентом, текст корпоративной политики. Модель обрабатывает этот файл и запоминает его содержание. После этого, когда вы задаёте вопрос, модель ищет ответ именно в вашем документе, а не в своей общей базе знаний. Это особенно ценно для компаний, у которых есть свои внутренние правила, процедуры, базы знаний. Робот может стать консультантом для сотрудников: задаёшь ему вопрос по правилам компании – он открывает нужный документ, находит ответ и формулирует его понятным языком. И при этом он никогда не ошибётся, не перепутает старую версию ин- струкции с новой, потому что вы сами загружаете ему актуальные документы.

Какие задачи языковые модели решают в RPA чаще всего? Генерация текста – робот получает шаблон письма и несколько параметров, модель на основе этого пишет персонализированное обращение к клиенту. Анализ обращений – в службу поддержки приходит десятки писем в час, робот отправляет каждое в модель, модель определяет тему, тональность, срочность, и на основе этого письмо направляется в нужный отдел или получает автоматический ответ. Извлечение данных – из длинного юридического договора нужно вытащить дату подписания, сумму контракта, стороны сделки. Модель делает это за секунды. Суммаризация – огромный отчёт по итогам квартала нужно превратить в короткую выжимку для руководителя. Модель читает отчёт и пишет краткое резюме. Ответы на вопросы – сотрудник спрашивает робота: «Какой порядок согласования командировок?», а робот, обученный на внутреннем регламенте компании, даёт точный ответ со ссылкой на пункт документа.

Важно понимать, что языковые модели – это не магия. Они не всегда дают точные ответы, особенно если вопрос сформулирован нечётко или если информации недостаточно. Они могут ошибаться, и иногда довольно уверенно. Поэтому при построении RPA-сценариев с использованием LLM всегда нужно предусматривать контроль качества. Проверять, что модель вернула ответ в ожидаемом формате. Сравнивать извлечённые данные с допустимыми диапазонами. При низкой уверенности или подозрении на ошибку – передавать задачу оператору. Но при разумном использовании языковые модели превращают RPA-робота из про- стого исполнителя команд в интеллектуального ассистента, который понимает язык, читает документы, пишет письма и принимает решения в ситуациях, где раньше требовалось участие человека.

Устойчивость умного робота: четыре механизма

Устойчивый робот – это не тот, который никогда не ошибается. Ошибки будут всегда, потому что мир вокруг меняется. Интерфейсы обновляются, данные приходят в неожиданном формате, сервера зависают, сеть работает с перебоями. Устойчивый робот – это тот, который замечает ошибку, не падает в ноль, а пытается исправиться сам или, если не получается, зовёт человека на помощь. В этой части мы разберём четыре механизма, которые делают умного робота по-настоящему устойчивым и готовым к любым неожиданностям.

Первый механизм – многоканальный поиск элементов. Классический робот ищет элемент одним способом. Чаще всего это поиск по XPath или ID. Если этот

единственный способ не сработал – например, разработчики системы изменили ID кнопки или переименовали класс, – робот падает с ошибкой. Всё. Дальше ну- жен человек, который откроет сценарий, найдёт сломанный блок, исправит селек- тор и только потом перезапустит робота. Умный робот устроен иначе. Он пробует несколько способов поиска последовательно, как будто у него в арсенале есть разные инструменты, и он перебирает их, пока не найдёт подходящий. Сначала робот пытается найти элемент по XPath – это самый быстрый способ. Если не получилось, пробует найти по ID или CSS-селектору. Если и это не помогло, подключает компьютерное зрение – ищет элемент по картинке. Если и машинное зрение не дало результата, он может попробовать найти элемент по тексту, который на нём написан, или по его расположению относительно других элементов. Только после того, как все возможные способы перепробованы и ни один не сработал, робот признаёт поражение и передаёт задачу оператору. Такой многоканальный подход резко повышает устойчивость. Изменился ID – робот всё равно найдёт элемент по картинке. Изменился внешний вид кнопки – возможно, старый XPath всё ещё работает. У каждого способа поиска есть свои слабые места, но вместе они дополняют друг друга.

Второй механизм – умные ожидания и повторные попытки. Классический робот часто использует фиксированные паузы. Нажал кнопку – подожди пять секунд и продолжай. Но мир нелинейный. Иногда операция занимает две секунды, и тогда три секунды робот ждёт впустую. Иногда операция занимает десять секунд, и тогда через пять секунд робот продолжает работу, а нужный элемент ещё не появился, и робот падает с ошибкой. Умные ожидания работают иначе. Робот не ждёт вслепую, он активно проверяет условие. Каждые полсекунды он смотрит, появился ли нужный элемент. Как только элемент появился – сразу продолжает работу, не теряя ни секунды. Если элемент не появился за разумное время, робот не падает сразу, а делает повторную попытку. Причём интервалы между попыт- ками увеличиваются – это называется экспоненциальной задержкой. Первая по- пытка – подождали две секунды, вторая – четыре, третья – восемь. Такая страте- гия позволяет роботу пережить временные сбои: сервер перегружен, сеть «задумалась», база данных выполняет тяжёлый запрос. Робот даёт системе время прийти в себя, а не бьёт тревогу при первой же задержке. И только после нескольких неудачных попыток, когда стало ясно, что проблема не временная, робот фиксирует ошибку и переходит к следующему механизму защиты.

Третий механизм – восстановление контекста. Представьте, что робот обрабатывает список из ста заявок. На пятьдесят пятой заявке произошёл сбой – например, потерялась связь с базой данных. Классический робот после перезапуска начнёт всё сначала: снова обработает первые пятьдесят четыре заявки, которые уже были успешно обработаны. Это теряет время, ресурсы и создаёт риск дублирования операций. Умный робот устроен умнее. Он постоянно сохраняет свой прогресс – запоминает, на каком шаге он находится, какие данные уже обработаны, какие результаты получены. Когда происходит сбой и робот перезапускается, он не начинает с нуля. Он считывает сохранённое состояние и продолжает ровно

с того места, где остановился. Более того, он может проанализировать тип ошибки и сам выбрать правильную точку входа. Если ошибка произошла потому, что элемент не загрузился, робот может просто повторить последнее действие. Если ошибка критическая, он может перезапустить не весь процесс, а только текущий шаг. Восстановление контекста требует дисциплины от разработчика – нужно проектировать сценарий так, чтобы каждый важный шаг логировался, а состояние сохранялось. Но затраты на это многократно окупаются надёжностью и эффективностью робота.

Четвёртый механизм – человек в контуре. Это самый важный принцип для серьёзных бизнес-процессов. Когда уверенность робота в правильности действия ниже определённого порога, он не должен рисковать. Он должен остановиться, позвать оператора, показать ему проблему и попросить помощи. Это не слабость робота, это разумная осторожность. Представьте, что робот обрабатывает финансовые документы. Модель классификации определила документ как счёт, но с вероятностью только шестьдесят процентов. В сорока процентах случаев это может быть не счёт, а что-то другое. Если робот ошибётся и отправит не тот документ в систему оплаты, последствия могут быть серьёзными. В такой ситуации правильное решение – передать документ оператору на ручную проверку. Оператор за пять секунд посмотрит и скажет: «Да, это счёт» или «Нет, это накладная». Робот запомнит этот пример, и в следующий раз, когда встретит похожий документ, его уверенность будет выше. Человек в контуре – это не просто обработка ошибок. Это механизм обучения. Каждый случай, когда оператор подтвердил или испра- вил решение робота, становится новым обучающим примером. Робот становится точнее, а со временем оператор нужен всё реже. Но полностью убирать человека нельзя – всегда должны оставаться ситуации, где окончательное решение принимает человек.

Эти четыре механизма работают вместе. Многоканальный поиск помогает роботу найти элемент разными способами. Умные ожидания дают системе время на загрузку. Восстановление контекста позволяет продолжить работу после сбоя. Человек в контуре подстраховывает там, где автоматизация пока не может быть достаточно точной. Вместе они создают робота, который не боится перемен, не ломается при первой ошибке и умеет просить о помощи, когда она нужна. Это и есть настоящая устойчивость.

Когда НЕ стоит использовать машинное зрение

Машинное зрение – это мощный инструмент, но, как любой инструмент, он хорош только тогда, когда применяется по назначению. Молоток отлично забивает гвозди, но пытаться закрутить им шуруп – плохая идея. Точно так же машин- ное зрение может стать обузой, если использовать его там, где есть более простые и надёжные решения. Опытный разработчик RPA всегда задаёт себе вопрос: «А нужно ли здесь вообще машинное зрение? Или можно обойтись классическими методами?» И часто ответ – можно.

Первая и самая главная ситуация, когда машинное зрение не нужно, – это когда элемент интерфейса имеет стабильный, надёжный программный идентифика-

тор. Если кнопка на веб-странице имеет уникальный ID, который не меняется годами, если поле ввода доступно по понятному XPath, если таблица имеет стабильную структуру DOM – используйте классические методы поиска. Они работают быстрее, потребляют меньше ресурсов, их поведение абсолютно предсказуемо, а ошибки легко диагностировать. Встраивать сюда машинное зрение – всё равно что пользоваться микроскопом, чтобы забить гвоздь. Вы только усложните себе жизнь, добавите зависимость от качества скриншотов и разрешения экрана, замедлите работу робота – и не получите ровно никакого преимущества.

Вторая ситуация – когда интерфейс часто и сильно меняется визуально. Машинное зрение опирается на внешний вид элементов. Если приложение каждый месяц меняет дизайн – перекрашивает кнопки, перерисовывает иконки, меняет расположение блоков, – ваши эталонные изображения будут постоянно устаревать. Вы будете тратить время на то, чтобы заново делать скриншоты, обновлять эталоны, перетестировать робота. Это бесконечная гонка. В таких условиях лучше использовать гибкие XPath-запросы, которые ищут элемент по его тексту или по отношению к другим элементам, – они будут работать даже при смене цветовой гаммы. Или, если уж совсем плохо, комбинированный подход: искать по XPath, а машинное зрение оставить как запасной вариант на случай, если интерфейс изменился настолько, что XPath сломался.

Третья ситуация – когда скорость выполнения критична. Поиск по картинке – операция дорогая. Робот должен сделать скриншот, просканировать его, срав- нить эталон с разными областями. В зависимости от размера экрана и эталона это может занимать от полусекунды до трёх-пяти секунд. Если ваш робот выполняет тысячу операций, и каждая операция требует поиска по картинке, набегут часы вместо минут. Для сравнения, поиск по ID или XPath работает практически мгно- венно – миллисекунды. Если вам нужно обрабатывать большие объёмы данных или работать в реальном времени, машинное зрение может стать узким горлыш- ком. Используйте его только там, где без него действительно не обойтись, а для остального применяйте классические методы.

Четвёртая ситуация – когда эталонное изображение невозможно создать или оно будет ненадёжным. Некоторые элементы меняют свой внешний вид динамически. Часы показывают текущее время – каждую секунду картинка меняется. Счётчик показывает число посетителей – он постоянно обновляется. Анимированная иконка моргает или вращается. По таким элементам невозможно создать стабильный эталон – вы сделаете скриншот в один момент, а через секунду элемент уже выглядит иначе. Машинное зрение будет то находить его, то нет, и вы никогда не поймёте, в чём дело. Для таких элементов нужно искать другие способы взаимодействия – может быть, читать текст из элемента классическим способом или обходить его стороной.

Пятая ситуация – когда требуется абсолютная точность, а цена ошибки очень высока. Машинное зрение даёт вероятностный результат. Даже при высокой точности в 0.99 остаётся один процент случаев, когда робот может ошибиться: принять не ту кнопку за нужную, кликнуть в неправильное место. В большинстве

бизнес-процессов один процент ошибки допустим. Но есть области, где ошибка недопустима. Медицина, атомная энергетика, военная сфера, управление опасными производствами – там цена ошибки может быть жизнью человека или огромным ущербом. В таких областях лучше использовать классические методы с детерминированным поведением или, если автоматизация всё же нужна, предусматривать многоступенчатый контроль, включающий человека в контуре.

Шестая ситуация – когда у вас есть возможность сделать интерфейс доступным для классической автоматизации. Иногда проще один раз попросить разработчиков системы добавить к кнопкам понятные ID или атрибуты, чем потом мучиться с эталонами и точностью поиска. Конечно, не всегда разработчики идут навстречу, особенно если система старая или её уже не поддерживают. Но если такая возможность есть – используйте её. Классический поиск всегда будет надёжнее и быстрее машинного зрения.

Итоговая рекомендация звучит так: машинное зрение – это инструмент для сложных случаев, когда простые методы не работают. Начинайте всегда с классических методов. Если они работают – отлично, задача решена просто и быстро. Если не работают – пробуйте XPath с функциями contains, starts-with, text. Если и это не помогает – подключайте машинное зрение как запасной вариант. И только когда вы точно знаете, что классические методы принципиально неприменимы – например, вы работаете через Citrix или с эмулятором терминала, – используйте машинное зрение как основной метод. Такой подход – от простого к сложному – сэкономит вам время, нервы и сделает робота более надёжным.

Основные блоки машинного зрения и AI в Puzzle RPA

В предыдущих практических работах мы освоили базовые операции автоматизации: клики, ввод текста, работу с таблицами, поиск элементов по XPath и ID. Это необходимый фундамент, который позволяет роботу выполнять простые линейные сценарии в хорошо структурированных интерфейсах. Однако реальные бизнес-задачи редко ограничиваются такими идеальными условиями. На практике робот сталкивается с приложениями, где нет программных идентификаторов, с интерфейсами, работающими через Citrix и RDP, со сканами документов, из которых нужно извлечь текст, с неструктурированными обращениями клиентов, которые нужно понять и классифицировать. Для решения этих задач требуются принципиально иные инструменты, основанные на искусственном интеллекте, машинном обучении и компьютерном зрении.

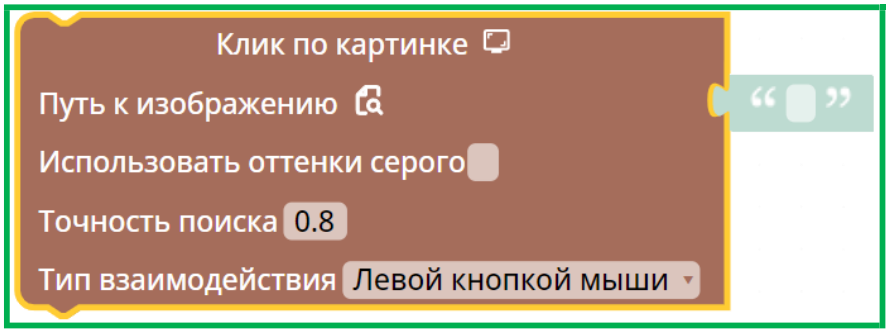
Сейчас мы переходим к изучению блоков машинного зрения и искусственного интеллекта в Puzzle RPA. Цель этой части практической работы – освоить инструменты, которые позволяют роботу видеть экран так же, как человек, пони- мать текст и принимать решения в нестандартных ситуациях. В результате вы сможете создавать не просто роботов, работающих в идеальных условиях, а ум- ных роботов, способных автоматизировать любые процессы, независимо от воз- раста приложений, наличия программных интерфейсов и структурированности данных. Приступаем к изучению основных блоков машинного зрения и AI в Puzzle RPA.

Платформа Puzzle RPA предоставляет готовые блоки для умной автоматизации, собранные в отдельные группы в панели инструментов. Все эти блоки работают без написания кода – достаточно выбрать нужный блок, настроить его параметры, и робот готов выполнять интеллектуальные задачи.

Группа блоков «Машинное зрение»

Первая группа блоков – это машинное зрение, или работа с экраном.

Блок «Клик по картинке»



Этот блок позволяет роботу найти на экране нужную картинку и кликнуть по ней. Проще говоря, робот смотрит на экран, ищет знакомое изображение – например, кнопку «Отправить» или иконку сохранения – и нажимает на неё.

Параметр «Тип взаимодействия» – это выпадающий список, в котором вы выбираете, что именно робот должен сделать с найденной картинкой. Это может быть обычный клик левой кнопкой мыши, клик правой кнопкой, двойной клик, наведение курсора без клика или другие варианты. Вы просто выбираете нужное действие из списка.

Параметр «Путь к изображению» – это адрес файла-эталона на вашем компьютере. Вы заранее делаете скриншот кнопки или иконки, сохраняете его в папку с проектом и указываете путь к этому файлу. Робот будет искать на экране именно это изображение.

Параметр «Точность поиска» – это число от 0 до 1, по умолчанию оно равно 0.8, то есть 80 процентов. Этот параметр определяет, насколько точно изображение на экране должно совпадать с эталоном. Если поставить 1.0, робот будет требовать идеального совпадения пиксель в пиксель – при малейшем изменении кнопки поиск не сработает. Если поставить 0.7, робот станет более терпимым к небольшим отличиям – например, если кнопка чуть изменила цвет или размер. Значение 0.8 – это золотая середина, которая подходит для большинства случаев.

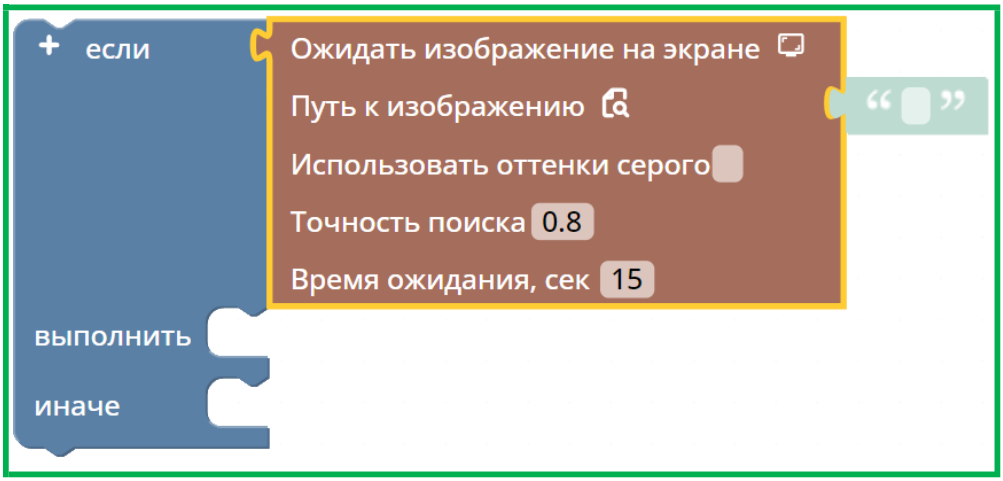
Чекбокс «Оттенки серого» – если его включить, робот перед поиском преобразует и эталонное изображение, и текущий экран в чёрно-белый формат. Это очень полезно, когда кнопка меняет цвет в зависимости от состояния. Например, кнопка может быть красной в активном состоянии и серой – в неактивном. Если вам нужно найти её в любом состоянии, вы включаете оттенки серого, и робот будет искать по форме и яркости, игнорируя цвет.

Значок редактора скриншотов – это маленькая иконка рядом с полем пути. При нажатии на неё открывается простой редактор, где вы можете прямо сейчас выделить мышкой нужную область экрана, обрезать её и сохранить как эталонное

изображение. После сохранения путь к файлу автоматически подставится в поле «Путь к изображению», и вам не придётся вводить его вручную.

Блок «Клик по картинке» не возвращает никакого значения в переменную – он просто выполняет действие. Если изображение не найдено, блок выбросит ошибку, и робот остановится. Поэтому на практике часто сначала используют блок «Найти картинку» или «Ожидать изображение», чтобы убедиться, что эле- мент есть на экране, а уже потом выполняют клик.

Блок «Ожидать изображение на экране»



Этот блок нужен, когда роботу нужно дождаться появления какого-то элемента на экране. Например, вы нажали кнопку «Рассчитать», и расчёт может длиться от двух секунд до минуты. После завершения расчёта на экране появляется кнопка «Сохранить результат». Вместо того чтобы ставить фиксированную задержку в 30 секунд (которая может оказаться слишком короткой или слишком длинной), вы используете блок «Ожидать изображение». Робот будет каждые полсекунды проверять, не появилась ли нужная картинка, и как только она появится – сразу пойдёт дальше.

Параметр «Путь к изображению» – это эталонная картинка, появления которой мы ждём. Например, скриншот кнопки «Сохранить результат» или скриншот загрузившегося окна.

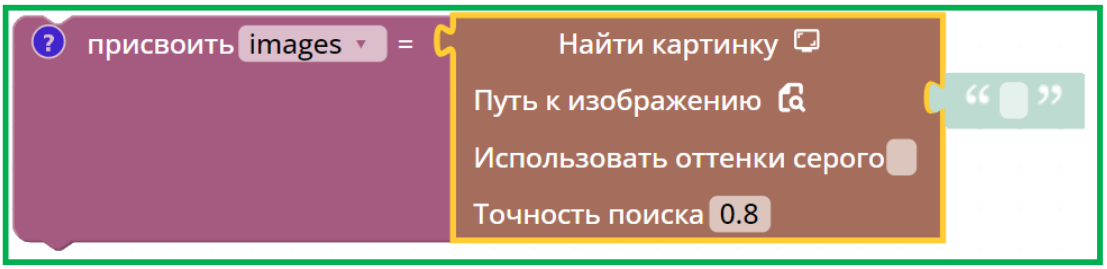
Параметр «Точность поиска» работает так же, как в блоке клика – число от 0 до 1, по умолчанию 0.8. Определяет, насколько точно изображение должно совпадать с эталоном.

Чекбокс «Оттенки серого» тоже работает аналогично – преобразует всё в чёрно-белый формат, чтобы не зависеть от цвета.

Параметр «Время ожидания» – это таймаут в секундах. По умолчанию он равен 15 секундам. Вы можете увеличить или уменьшить это значение в зависимости от того, как долго обычно появляется нужный элемент. Если за это время картинка так и не появилась, блок завершится с результатом «ложь».

Блок возвращает логическое значение: «истина», если изображение появилось в течение заданного времени, и «ложь», если не появилось. Важно понимать: этот блок не кликает по картинке и не возвращает её координаты. Он только ждёт и сообщает о результате. Если вам нужно и дождаться, и кликнуть, вы сначала используете «Ожидать изображение», а потом – «Клик по картинке».

Блок «Найти картинку»



Этот блок ищет картинку на экране прямо сейчас, не ожидая. Он делает одну проверку и сразу возвращает результат. Если картинка есть – он сообщает, где именно она находится. Если картинки нет – возвращает ложь.

Параметр «Путь к изображению» – эталонная картинка, которую мы ищем.

Параметр «Точность поиска» – от 0 до 1, по умолчанию 0.8.

Чекбокс «Оттенки серого» – включает поиск по форме без учёта цвета.

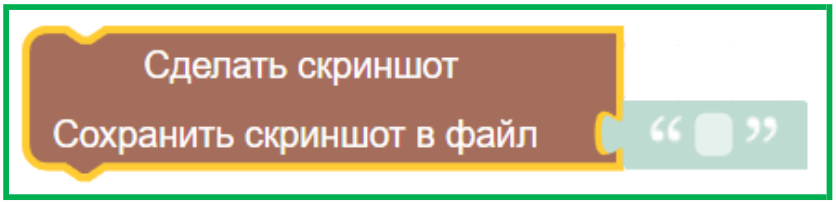
Что возвращает блок? Если изображение найдено, он возвращает массив координат найденных совпадений. Формат выглядит как объект с четырьмя числами: left – расстояние от левого края экрана до левого верхнего угла найденной картинки в пикселях; top – расстояние от верхнего края экрана до левого верхнего угла; width – ширина найденной картинки; height – её высота.

Если на экране есть несколько одинаковых картинок, массив будет содержать несколько таких объектов – по одному на каждое совпадение.

Если изображение не найдено, блок возвращает просто ложь (False).

Блок «Найти картинку» часто используют перед «Кликом по картинке», когда нужно сначала убедиться, что картинка есть, а потом кликнуть по её координатам. Это позволяет избежать ошибки, если картинка вдруг отсутствует. Также этот блок используют, когда нужно не просто кликнуть, а, например, переместить мышь в определённое место или выполнить действие относительно найденной картинки.

Блок «Сделать скриншот»



Этот блок делает скриншот всего экрана и сохраняет его в файл. Всё очень просто и прямолинейно.

Параметр «Путь к файлу» – это полный адрес, куда нужно сохранить скриншот. Вы указываете путь к папке, придумываете имя файла и обязательно указываете расширение .png. Например: C:\скриншоты\мой_скриншот.png. Файл будет создан автоматически по этому пути. Другие форматы, кроме PNG, не поддерживаются.

Блок не возвращает никаких значений. Он просто делает своё дело – создаёт файл с изображением экрана.

Зачем нужны скриншоты? Во-первых, для логирования ошибок. Если робот упал, вы можете сделать скриншот экрана прямо в блоке обработки ошибки, и по-

том будет видно, что происходило в момент сбоя. Во-вторых, для верификации – проверки, что операция действительно завершилась успешно. Например, после сохранения документа можно сделать скриншот и с помощью компьютерного зрения проверить, появилось ли сообщение «Сохранено успешно». В-третьих, для создания эталонных изображений для других блоков машинного зрения – вы можете сделать скриншот и потом вырезать из него нужную кнопку в любом графическом редакторе.

Важная особенность: скриншот делается со всего экрана целиком, а не с отдельного окна. Если вам нужен скриншот только части экрана, придётся сначала найти нужную область (например, через «Найти картинку»), а затем обрезать изображение. Сам блок «Сделать скриншот» такой возможности не даёт.

Четыре блока машинного зрения решают разные задачи, и важно понимать, какой из них когда использовать.

Блок «Клик по картинке» – самый востребованный. Он ищет картинку и сразу кликает по ней. Это готовое решение для ситуации «увидел кнопку – нажал». Если картинка не найдена, блок выбросит ошибку.

Блок «Ожидать изображение» – это блок ожидания. Он не кликает, не возвращает координаты. Он только ждёт и отвечает на вопрос «появилась ли картинка за отведённое время?». Вернёт истину – значит, появилась. Вернёт ложь – значит, нет.

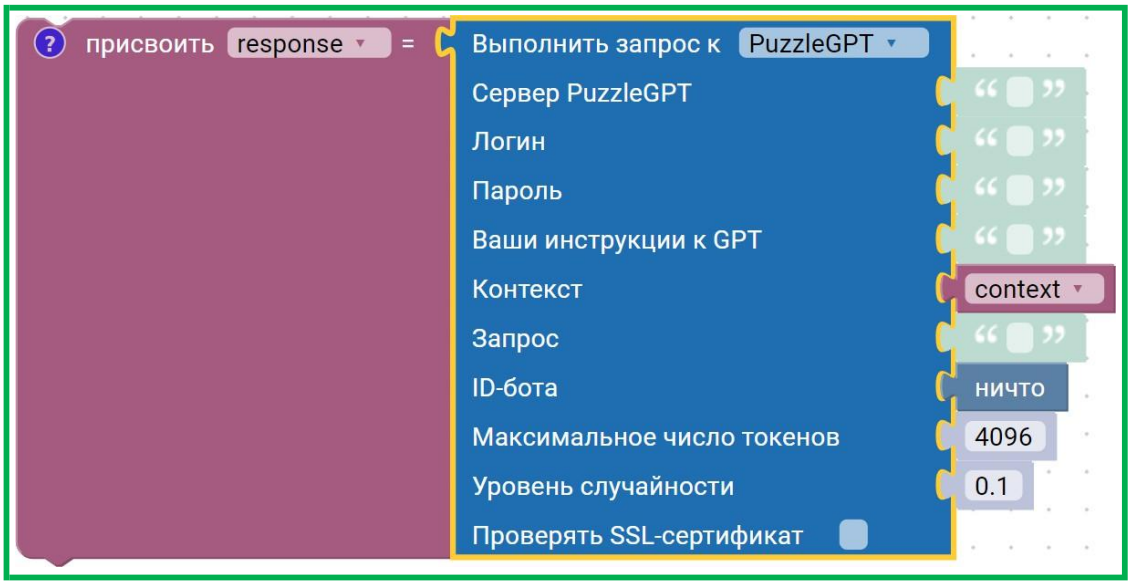
Блок «Найти картинку» – это блок поиска. Он не ждёт, не кликает. Он один раз проверяет, есть ли картинка на экране прямо сейчас. Если есть – возвращает координаты. Если нет – возвращает ложь.

Блок «Сделать скриншот» стоит немного отдельно. Он не ищет и не ждёт. Он просто сохраняет изображение экрана в файл. Это вспомогательный блок для логирования и верификации.

Все блоки машинного зрения чувствительны к разрешению экрана и масштабу. Эталон, созданный на мониторе с разрешением 1920 на 1080 при 100-процентном масштабе, может не найтись на мониторе с разрешением 1366 на 768 или при масштабе 125 процентов. Поэтому всегда создавайте эталонные изображения на том же разрешении и с теми же настройками масштаба, которые будут использоваться при работе робота в продуктивной среде.

Следущая группа – работа с текстом и большими языковыми моделями. Здесь робот умеет задавать вопросы к GPT и даже обучать нейросеть на ваших собственных документах.

Блок «Выполнить запрос в GPT»



Это блок, который позволяет вашему роботу поговорить с нейросетью. Вы задаёте вопрос текстом, робот отправляет его в выбранную языковую модель (например, ChatGPT, YandexGPT, DeepSeek), а модель возвращает ответ. Всё просто: вопрос → ответ.

Ключевые параметры:

Какую нейросеть вы хотите использовать? PuzzleGPT (своя), ChatGPT, YandexGPT, DeepSeek, Qwen, Claude, Grok. Выбирайте ту, которая есть у вас в доступе.

Сервер PuzzleGPT / Логин / Пароль Адрес и учётные данные для доступа к PuzzleGPT. Это как логин и пароль от вашей корпоративной нейросети.

Ваши инструкции к GPT. Это «настройка личности» модели. Например: «Ты – помощник по работе с Puzzle RPA. Отвечай кратко и по делу». Модель будет следовать этой инструкции при каждом ответе.

Контекст История предыдущих вопросов и ответов. Передаётся в виде списка словарей. Зачем? Чтобы модель помнила, о чём вы говорили раньше. Например, вы спросили «Что такое RPA?», модель ответила, а потом вы спраши- ваете «А какие плюсы?» – без контекста модель не поймёт, плюсы чего именно.

Запрос. Ваш вопрос к нейросети. Самое главное поле. Чем чётче вопрос, тем лучше ответ.

ID бота. Уникальный идентификатор обученной модели. Если вы дообучили нейросеть на своих документах (через блок «Дообучить нейросеть»), вы указываете её ID, и модель будет отвечать, опираясь на ваши файлы.

Максимальное число токенов. Ограничение на длину ответа. Токены – это кусочки текста (буквы, слова). Чем больше значение, тем длиннее может быть ответ, но тем дольше он генерируется.

Почему мы ставим НЕ БОЛЕЕ 4000 – это важно.

В Puzzle RPA рекомендуется ставить не более 4000 токенов. Почему?

Причина	Объяснение
Скорость	Чем больше токенов, тем дольше модель думает. 8000 токенов могут генерироваться 30-60 секунд

Стоимость	В коммерческих версиях токены стоят денег. Больше токенов = дороже
Надёжность	Длинные ответы чаще содержат ошибки и «галлюцинации» (модель начинает выдумывать)

Уровень случайности От 0.0 до 1.0. Если поставить 0.0 – модель будет отвечать жёстко, одинаково на одинаковые вопросы (как робот). Если поставить 1.0 – модель начнёт фантазировать, ответы будут разнообразными и креативными.

Проверять SSL-сертификат Галочка, которая включает проверку безопасности соединения.

Что возвращает блок?

Строку с ответом нейросети. Просто текст.

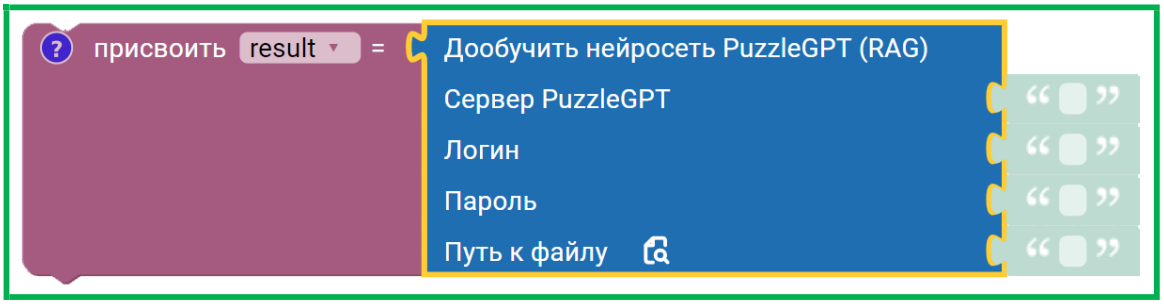
Важное исключение (YandexGPT):

Если вы выбрали YandexGPT, блок возвращает не одну строку, а массив из 4 элементов:

- Сам ответ на запрос.
- Полный ответ от сервера Yandex.
- Заголовки HTTP-ответа.
- Код ответа.

Это техническая особенность. Обычно вам нужен первый элемент – сам ответ.

Блок «Дообучить нейросеть PuzzleGPT (RAG)»



Этот блок позволяет научить нейросеть читать ваш документ и отвечать на вопросы только на основе того, что написано в этом документе. Технология называется RAG (Retrieval-Augmented Generation) – поиск ответов по вашей базе знаний.

Представьте, что вы загружаете в нейросеть корпоративный регламент или инструкцию. Теперь робот сможет отвечать на вопросы сотрудников, ссылаясь именно на этот документ, а не на общие знания из интернета.

Вы даёте модели ваши знания. Она не просто «умный чат-бот», а эксперт именно в вашей теме. Отвечает только по вашим документам, не фантазирует на свободные темы.

Ключевые параметры:

Сервер PuzzleGPT / Логин / Пароль. Всё то же самое – куда и под какой учёткой загружать.

Путь к файлу. Word-файл (DOC/DOCX) или PDF с текстовым слоем (то есть не просто сканированная картинка, а настоящий текст внутри).

Блок возвращает словарь (объект) с двумя ключами:

bot_id – уникальный идентификатор вашей обученной модели (бота).

file_id – идентификатор загруженного файла.

Эти ID нужно сохранить в переменные, потому что без них вы не сможете задавать вопросы обученной модели.

Два шага использования:

Шаг 1. Обучаем. Загружаем документ через блок «Дообучить нейросеть». Получаем bot_id и file_id. Сохраняем их в переменные.

Шаг 2. Спрашиваем. В блоке «Выполнить запрос в GPT» в поле ID бота указываем тот самый bot_id. Теперь модель будет искать ответ в вашем документе.

Важные моменты:

- Файл должен быть с текстовым слоем. Если это просто скан PDF без распознанного текста – OCR не поможет. Блок RAG работает с текстом, а не с картинками.
- Обучение не навсегда. Модель не переучивается глобально, она просто получает доступ к вашему документу при каждом запросе. Загрузили файл – модель может по нему отвечать.
- Два ID обязательны. Без bot_id вы не сможете обратиться к обученной модели. Без file_id – не сможете обновить или удалить файл.

Главное отличие двух блоков:

Блок	Что делает	Когда использовать
Выполнить запрос в GPT	Задаёт вопрос общей нейросети. Модель отвечает на основе всего, что знает из интернета/обучения.	Когда нужны общие знания, креатив, помощь с текстом, перевод, резюмирование.
Дообучить нейросеть (RAG)	Загружает ваш документ, чтобы модель отвечала строго по нему.	Когда нужны точные ответы на основе корпоративных документов, регламентов, инструкций, базы знаний.

В совокупности представленные блоки формируют полноценный инструментарий для автоматизации самых разнообразных веб-интерфейсов – от простого извлечения данных до сложного взаимодействия с динамическими элементами, таблицами, drag-and-drop интерфейсами и диалоговыми окнами. Освоение этих блоков позволяет разработчику создавать устойчивые, гибкие и масштабируемые сценарии RPA для работы с веб-приложениями любой сложности.

Подведение итогов практической работы

Мы завершили изучение темы машинного зрения и устойчивости умного робота в Puzzle RPA. За время освоения этого раздела мы углубили свои знания и освоили сложные инструменты, которые позволяют создавать надёжные, отказоустойчивые и интеллектуальные сценарии для работы с любыми интерфейсами – от современных веб-приложений до старых десктопных программ и виртуальных рабочих столов.

Мы поняли, когда классического RPA недостаточно. Осознали, что существуют целые классы приложений – legacy-системы, эмуляторы терминалов, виртуальные рабочие столы Citrix и RDP, – где стандартные методы поиска элементов по ID, XPath и CSS-селекторам просто не работают. В этих средах робот ви-

дит только картинку, и единственный способ взаимодействия – это компьютерное зрение.

Мы освоили базовые операции компьютерного зрения: поиск изображения на экране, клик по картинке и ожидание появления элемента. Научились комбинировать эти операции для построения сложных сценариев автоматизации. Поняли, как блок «Найти картинку» возвращает координаты, блок «Клик по картинке» объединяет поиск и действие, а блок «Ожидать изображение на экране» позволяет роботу терпеливо ждать загрузки интерфейса вместо того, чтобы падать при первой задержке.

Особую ценность представляет понимание параметра точности поиска. Мы научились выбирать значение точности в зависимости от конкретной задачи: высокая точность для стабильных интерфейсов, низкая для работы на разных разрешениях или при изменённом дизайне. Освоили адаптивный подход, когда робот сначала пробует найти элемент с высокой точностью, а если не получается, постепенно снижает требования. Поняли, что точность – это баланс между риском не найти элемент и риском найти неправильный.

Мы познакомились с языковыми моделями в RPA и узнали, как большие языковые модели, такие как PuzzleGPT, могут генерировать текст, анализировать обращения клиентов, извлекать сущности из документов, отвечать на вопросы и писать письма. Освоили технологию RAG, которая позволяет дообучать нейросеть на собственных документах компании – инструкциях, регламентах, ба- зе знаний. Теперь робот может быть не просто исполнителем, но и интеллекту- альным консультантом.

Критически важной темой стала устойчивость умного робота. Мы изучили четыре механизма, которые делают робота по-настоящему надёжным. Первый – многоканальный поиск, когда робот пробует разные способы найти элемент: сначала по XPath, потом по картинке, потом по тексту, и только после этого зовёт оператора. Второй – умные ожидания и повторные попытки, когда робот не падает при первой задержке, а ждёт с экспоненциально увеличивающимися интервалами. Третий – восстановление контекста, когда робот запоминает своё состояние и после сбоя продолжает с того же места, а не начинает заново. Четвёртый – человек в контуре, когда робот при низкой уверенности передаёт задачу оператору, а не рискует ошибиться в дорогом процессе.

Мы разобрали, когда НЕ стоит использовать машинное зрение, и научились выбирать правильный инструмент для каждой задачи. Поняли, что машинное зрение – это не замена классическим методам, а их усиление для сложных случаев. Если элемент имеет стабильный XPath или ID, классический поиск будет быстрее и надёжнее. Машинное зрение нужно подключать только там, где без него не обойтись – в Citrix, legacy-системах, при работе с динамическими идентификаторами.

Мы детально разобрали основные блоки машинного зрения и искусственного интеллекта в Puzzle RPA. Изучили блоки «Найти картинку», «Клик по картинке», «Ожидать изображение на экране», «Сделать скриншот», «Выполнить запрос к

языковой модели», «Дообучить нейросеть». Научились настраивать их параметры, комбинировать друг с другом и с обычными блоками автоматизации для построения комплексных интеллектуальных решений.

В ходе практической работы мы сформировали целостную систему знаний и навыков, необходимых для профессиональной умной автоматизации. От простых сценариев кликов и ввода текста мы перешли к сложным, интеллектуальным решениям, способным работать в любых интерфейсах, обрабатывать неструктурированные данные, понимать текст и принимать решения в условиях неопределённости.

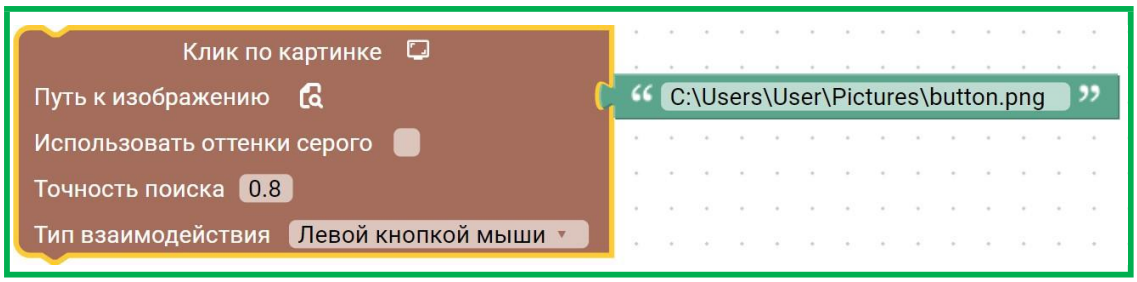
Главный результат – мы научились не просто использовать готовые блоки машинного зрения и AI, но и понимать, когда и как их применять. Мы освоили принципы построения устойчивых роботов, которые не боятся изменений, не ломаются при первой ошибке и умеют просить о помощи, когда она нужна. Полученные компетенции являются фундаментом для реализации проектов любой сложности в области умной автоматизации – от обработки входящих документов до интеллектуального ассистента на базе корпоративной базы знаний. Теперь мы готовы создавать роботов, которые не просто выполняют инструкции, а видят экран, понимают текст и принимают решения.

ПРИМЕРЫ ЗАДАЧ

Пример 1 – Клик по картинке

В данном примере блок находит значок на экране и выполняет по нему клик.

- Добавить блок «Клик по картинке» в проект и указать путь в поле «Путь к изображению.»
- При необходимости включить параметр «Использовать оттенки серого», задать значение «Точность поиска» и выбрать вариант в списке «Тип взаимодействия».
- Запустить робота, чтобы найти изображение на экране и выполнить выбранное действие.



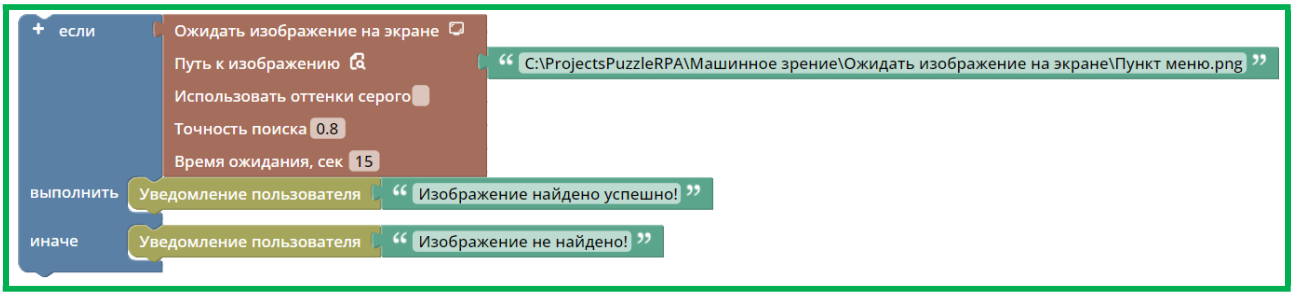
После выполнения блока взаимодействие с найденным изображением будет выполнено на экране:



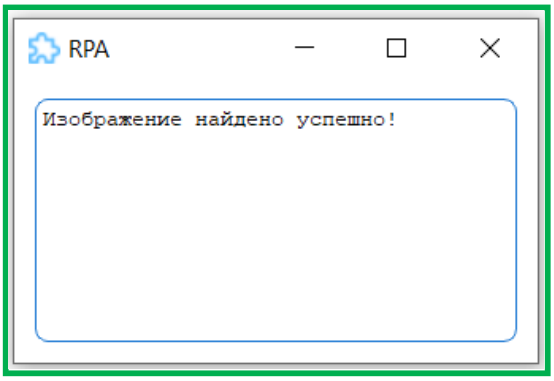
Пример 2 – Ожидать изображение на экране

В данном примере блок ожидает появления изображения на экране и сообщает об успешном результате.

- Добавить блок «Ожидать изображение на экране» в проект и указать значение параметра «Путь к изображению».
- При необходимости включить параметр «Использовать оттенки серого», задать значения «Точность поиска» и «Время ожидания, сек».
- Запустить робота, чтобы дождаться появления изображения на экране.



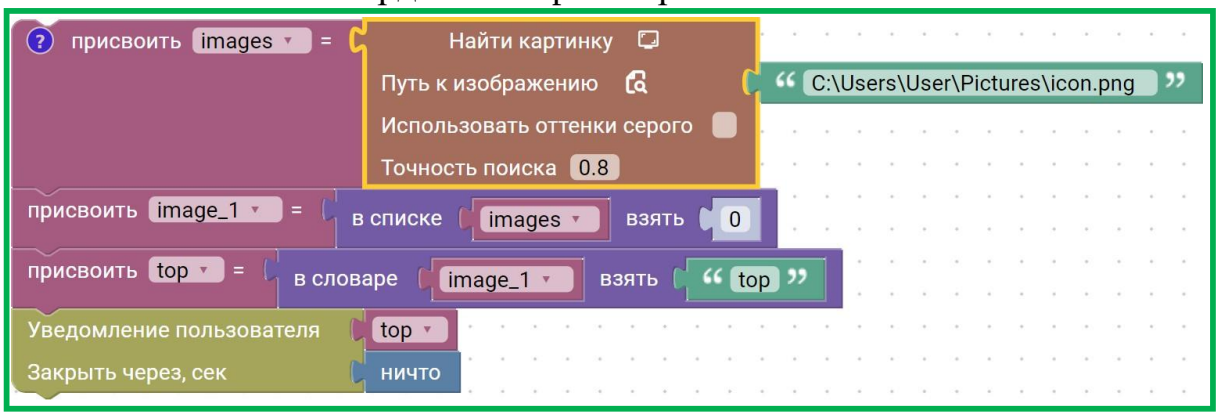
Вывод на экран сообщения об успешном появлении изображения на экране:



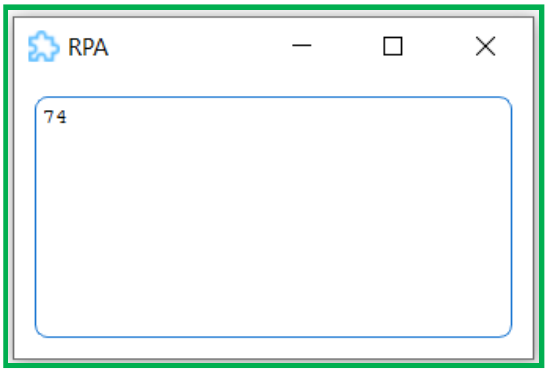
Пример 3 – Найти картинку

В данном примере блок находит изображение на экране, после чего используется координата top первого найденного совпадения.

- Добавить блок «Найти картинку» в проект и указать значение параметра «Путь к изображению».
- При необходимости включить параметр «Использовать оттенки серого» и задать значение «Точность поиска».
- Обратиться к первому элементу возвращаемого списка.
- Получить значение координаты top объекта первого найденного совпадения.
- Вывести значение координаты top на экран.



Вывод на экран значения координаты top:



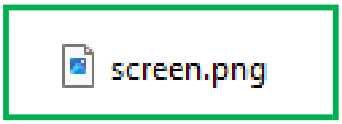
Пример 4 – Сделать скриншот

В данном примере блок сохраняет скриншот рабочего стола в файл и выводит уведомление об успешном сохранении.

- Добавить блок «Сделать скриншот» в проект и указать путь в параметре «Сохранить скриншот в файл».
- Запустить робота, чтобы сохранить скриншот всего экрана в указанный файл.



Созданный файл скриншота:



Пример 5 – Выполнить запрос в GPT

В приведённом примере к модели PuzzleGPT последовательно отправляются два запроса:

Объявляется переменная context, предназначенная для хранения истории диалога;

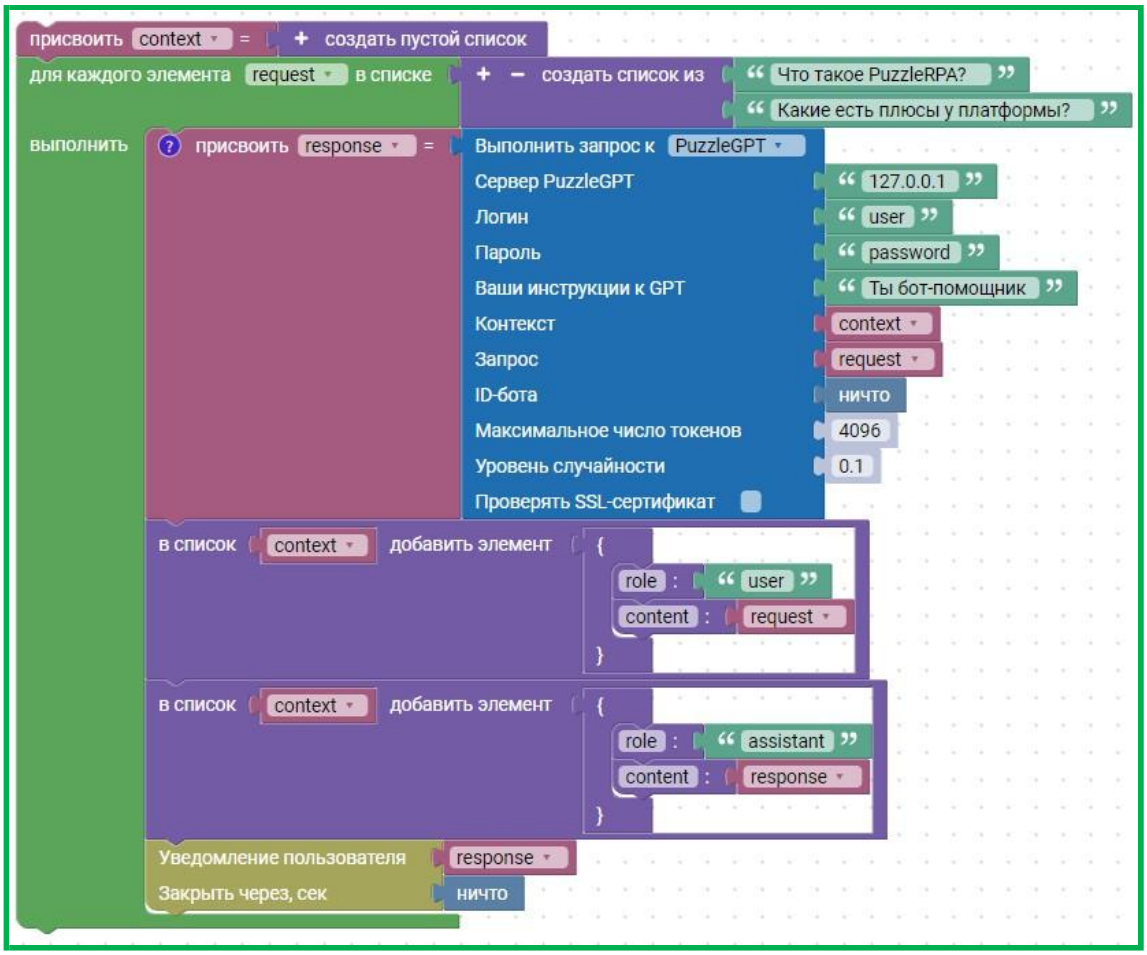
Выполняется цикл из двух запросов:

- Что такое PuzzleRPA?
- Какие есть плюсы у платформы?
- Выполняется первый запрос к модели PuzzleGPT;

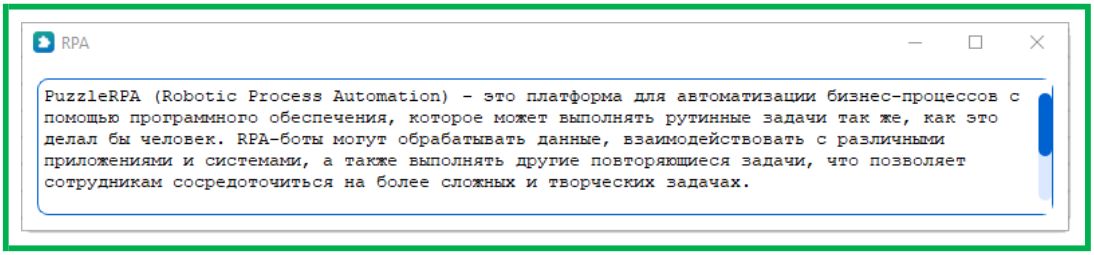
Запрос и ответ PuzzleGPT добавляется в список переменной context для учета общего смысла в следующем запросе;

Выполняется второй запрос к PuzzleGPT с учетом контекста.

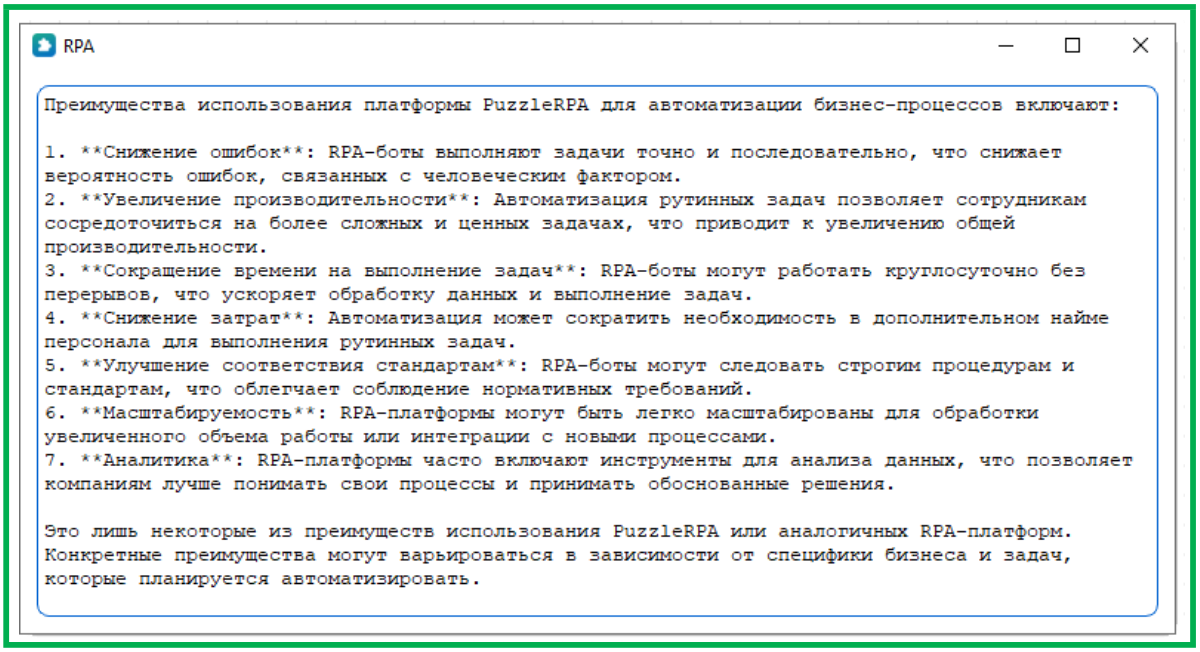
Важно: Во втором запросе явно не указано название платформы. Модель PuzzleGPT строит ответ исходя из передаваемого контекста.



Вывод на экран первого ответа нейросети:

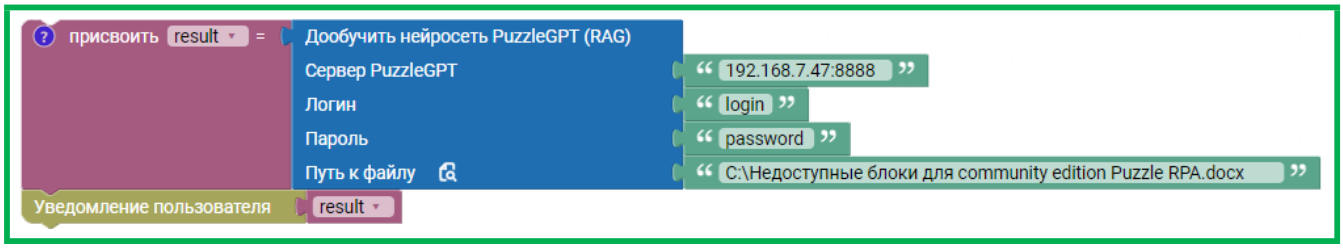


Вывод на экран второго ответа нейросети:

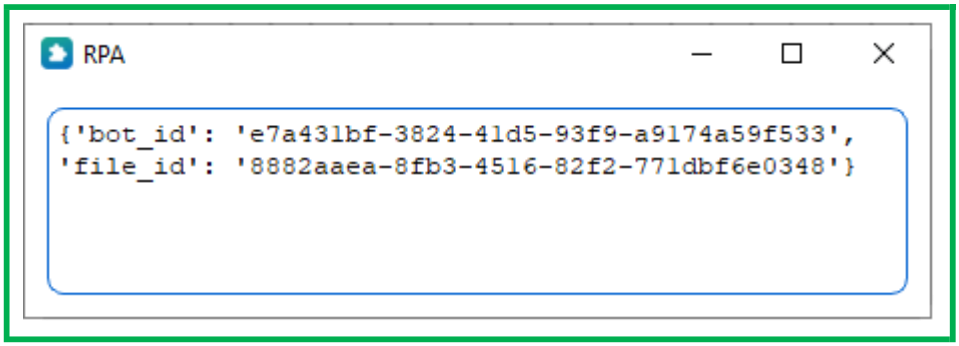


Пример 6 – Дообучить нейросеть PuzzleGPT (RAG)

В данном примере выполняется дообучение PuzzleGPT. В качестве файла передается Word-файл с информацией о Недоступных блоках для версии Puzzle RPA Community edition.

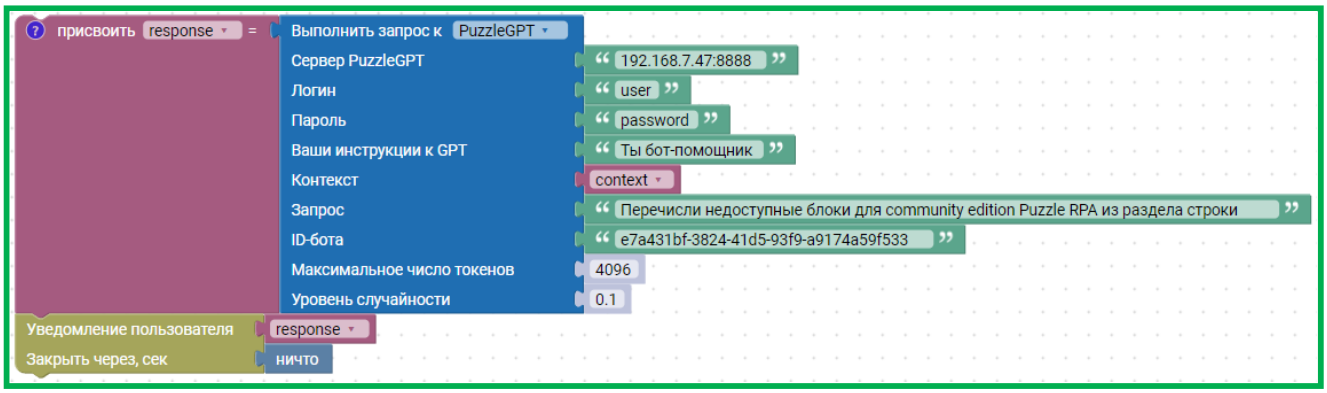


В переменной result содержится словарь с 2 элементами: “bot_id” и “file_id”.

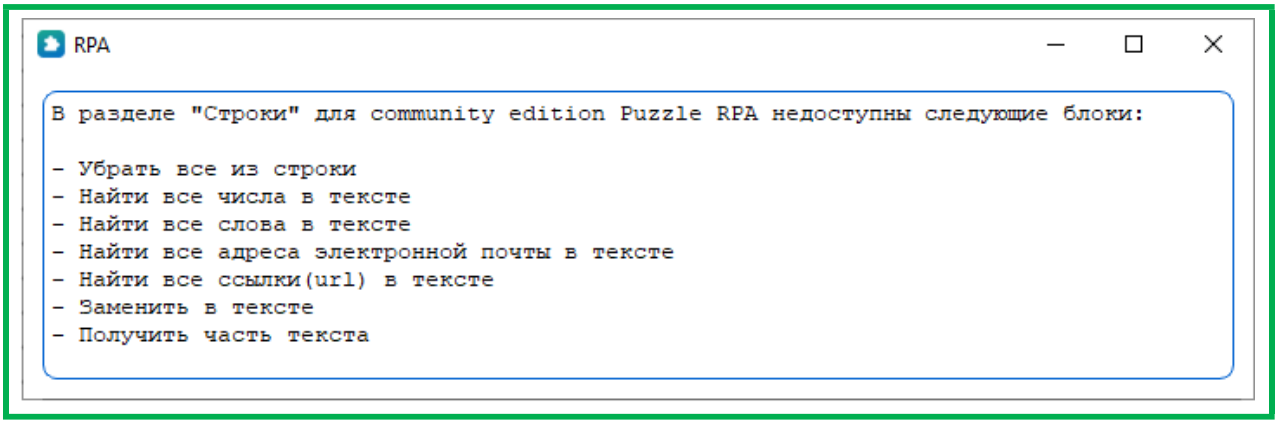


2 шаг – Использование созданного бота PuzzleGPT

В данном примере выполняется запрос в PuzzleGPT. В поле ID-бота указан ID ранее созданного бота.



Вывод на экран ответа нейросети:



Кейс 1. Автоматизация десктоп-приложения с использованием Machine Vision и AI

Постановка задачи

В компании TechFlow сотрудники регулярно создают внутренние сервисные заявки: запросы на техническую поддержку, доступ к корпоративным системам, настройку оборудования и другие рабочие задачи.

Проблема в том, что компания работает в сложной IT-инфраструктуре. Приложение может запускаться через удалённые рабочие столы, внутри виртуальных сред или через терминальные серверы. В таких условиях традиционная автоматизация на основе UI-селекторов часто недоступна или работает нестабильно.

Поэтому для автоматизации используется подход Machine Vision – робот анализирует изображение интерфейса, находит элементы по их эталонным картинкам и выполняет действия по координатам найденных элементов.

Дополнительно в этой работе используется интеграция с искусственным интеллектом. Робот будет генерировать названия заявок с помощью GPT, а затем автоматически добавлять их в систему.

Перед выполнением практической работы необходимо подготовить изображения элементов интерфейса. Для этого нужно запустить приложение, сделать скриншоты всех ключевых элементов и сохранить их в папку с роботом.

Приложение состоит из нескольких частей.

В верхней части находится форма создания заявки. Здесь есть поле Request Title для ввода темы заявки, поле Employee для указания имени сотрудника и выпадающий список Priority для выбора приоритета (Low, Medium или High).

В нижней части окна находится таблица зарегистрированных заявок. Таблица содержит колонки: ID, Title, Employee, Priority и Status. У новой заявки статус автоматически устанавливается как New.

Также в приложении есть кнопки управления: Mark as Done – меняет статус выбранной заявки на Done, и Delete Selected – удаляет выбранную запись из таблицы. Перед удалением появляется окно подтверждения.

Для работы робота используется Excel-файл с именем requests.xlsx. В этом файле содержатся две колонки: Employee с именами сотрудников и Priority с приоритетами заявок. В файле 10 строк с данными.

Детализация решения задачи

Задание состоит из двух уровней.

Уровень первый – базовая автоматизация интерфейса

- Робот должен запустить приложение и дождаться его полной загрузки.
- Затем робот обращается к искусственному интеллекту. Он просит GPT сгенерировать 10 названий заявок.
- Далее робот читает Excel-файл, получая таблицу с именами сотрудников и приоритетами.
- После этого робот поэтапно вводит и обрабатывает заявки.

Все эти действия робот выполняет, используя только Machine Vision – каждый клик происходит по заранее подготовленному изображению элемента.

Уровень второй – обработка заявок

После того как все 10 заявок добавлены, робот переходит к их обработке.

- Сначала он находит в таблице заявку с приоритетом High. Затем кликает по этому элементу, выделяя всю строку таблицы.
- После этого робот нажимает кнопку Mark as Done, меняя статус заявки на Done. Затем нажимает кнопку Delete Selected, вызывая окно подтверждения удаления.
- Робот ожидает появления окна подтверждения и нажимает кнопку подтверждения.
- В конце робот сохраняет скриншот итоговой таблицы.

Робот должен быть построен модульно: каждая функция отвечает за свою задачу – запуск приложения, генерация названий, чтение Excel, заполнение формы, обработка заявок и логирование. Это позволяет легко изменять и дополнять робота в будущем.

Все действия робота фиксируются в лог-файле для отслеживания выполнения процесса и диагностики возможных ошибок.

Теперь перейдём к реализации RPA-сценария в среде Puzzle RPA.

В рамках практической работы был разработан продвинутый RPA-робот в среде Puzzle RPA для автоматизации процесса регистрации и обработки сервисных заявок в десктопном приложении с использованием технологий машинного зрения и искусственного интеллекта.

Робот построен по модульному принципу. Каждая функция отвечает за свою конкретную задачу, что обеспечивает читаемость кода, удобство отладки и возможность повторного использования компонентов.

Функция TextTooLog

Функция TextTooLog предназначена для записи информационных сообщений в текстовый файл лога с указанием точного времени события.

Что делает функция: Первое – получает текущую дату и время. Второе – добавляет к ним переданное в функцию сообщение. Третье – записывает всё в файл с именем log.txt. Каждое сообщение записывается с новой строки.

Функция принимает один параметр – info. Это строковое значение, содержащее сообщение, которое нужно записать в лог.



Шаг 1. Получение текущей даты и времени

Какой блок используется: Получить текущую дату

Что делаем в этом блоке: Указываем формат даты и времени.

Что возвращает блок: Возвращает строку с текущей датой и временем в указанном формате.

Шаг 2. Формирование строки для записи

Какой блок используется: Создать текст из

Что делаем в этом шаге: Нам нужно собрать одну строку из трёх частей. Первая часть – это дата и время, полученные на предыдущем шаге. Вторая часть – это символы двоеточие и пробел, чтобы отделить время от сообщения. Третья часть – это само сообщение, которое пришло в параметре info. В конце добавляем символ перевода строки, чтобы каждое новое сообщение записывалось с новой строки.

Шаг 3. Получение пути к файлу лога

Какой блок используется: Относительный путь

Что делаем в этом блоке: Указываем имя файла, в который будем писать лог. В данном случае это log.txt. Блок автоматически определит полный путь к папке, где находится ваш робот, и добавит к нему имя файла.

Шаг 4. Запись строки в файл

Какой блок используется: Дописать в файл.

Что делаем в этом блоке: Берём сформированную на шаге 2 строку и записываем её в файл, путь к которому мы получили на шаге 3. Важно: мы используем режим дописать, а не перезаписать. Это значит, что новое сообщение добавится в конец файла, а старые сообщения останутся на месте. Указываем кодировку utf-8, чтобы корректно записывались русские буквы.

Функция start_exe

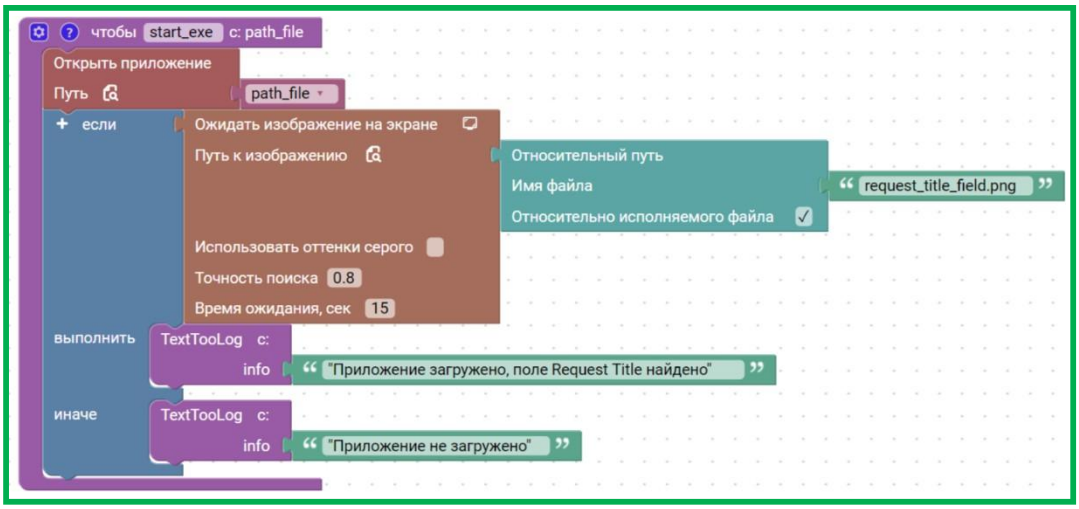
Функция start_exe предназначена для запуска десктопного приложения и проверки его успешной загрузки.

Что делает функция:

Первое – запускает исполняемый файл приложения по указанному пути. Второе – ожидает появления на экране элемента интерфейса (поля Request Title). Третье – записывает в лог сообщение о том, загрузилось приложение или нет.

В условиях работы через удалённые рабочие столы или виртуальные среды приложение может загружаться медленно. Функция дожидается полной загрузки интерфейса, прежде чем робот начнёт выполнять следующие действия.

Функция принимает один параметр – path_file. Это строковое значение, содержащее полный путь к исполняемому файлу приложения, которое нужно запустить.



Шаг 1. Запуск приложения

Какой блок используется: Открыть приложение

Что делаем в этом блоке: Передаём в блок путь к исполняемому файлу, который пришёл в параметре path_file. Блок запускает это приложение так же, как если бы вы дважды кликнули по файлу мышью.

Шаг 2. Ожидание появления элемента интерфейса

Какой блок используется: Ожидать изображение на экране

Что делаем в этом блоке: Указываем изображение-эталон, которое должно появиться на экране после загрузки приложения. В данном случае это поле request_title_field.png. Указываем точность совпадения 0.8 (80 процентов). Указываем таймаут 15 секунд – это максимальное время ожидания. Если элемент появится раньше, блок завершится досрочно.

Что возвращает блок: Возвращает значение True, если изображение найдено в течение таймаута. Возвращает False, если изображение не появилось за 15 се- кунд.

Шаг 3. Проверка результата ожидания (условие)

Какой блок используется: Конструкция Если – выполнить-иначе

Что делаем в этом блоке: Проверяем, что вернул блок ожидания. Если вернул True, значит приложение загрузилось и поле найдено. Если вернул False, значит приложение не загрузилось или поле не появилось.

Шаг 4. Действие при успешной загрузке (ветка Если)

Какой блок используется: TextTooLog (наша собственная функция)

Что делаем в этом блоке: Вызываем функцию TextTooLog и передаём ей сообщение "Приложение загружено, поле Request Title найдено". Это сообщение запишется в файл лога с указанием текущего времени.

Шаг 5. Действие при ошибке загрузки (ветка Иначе)

Какой блок используется: TextTooLog (наша собственная функция)

Что делаем в этом блоке: Вызываем функцию TextTooLog и передаём ей сообщение "Приложение не загружено". Это сообщение запишется в файл лога с указанием текущего времени.

Шаг 6. Завершение функции

Функция не возвращает никакого значения. Она либо успешно запускает приложение и фиксирует это в логе, либо фиксирует ошибку. В любом случае функция завершает свою работу.

Важные замечания:

- Первое. Изображение request_title_field.png должно быть подготовлено заранее. Нужно запустить приложение вручную, сделать скриншот поля Request Title и сохранить его в папку с роботом.
- Второе. Точность 0.8 означает 80-процентное совпадение. Это значение подходит для большинства случаев. Если робот не находит изображение, точность можно уменьшить. Если находит не то, что нужно – увеличить.
- Третье. Таймаут 15 секунд выбран с учётом того, что приложение может запускаться через удалённый рабочий стол или виртуальную среду. Если приложение загружается дольше, таймаут можно увеличить.

Функция generate_fromGPT

Функция generate_fromGPT предназначена для генерации списка названий заявок с помощью искусственного интеллекта (модели PuzzleGPT).

Что делает функция:

Первое – отправляет запрос к серверу PuzzleGPT с указанными инструкциями и промптом. Второе – получает от GPT строку с названиями заявок, разделёнными вертикальной чертой. Третье – преобразует полученную строку в массив (список) для дальнейшего использования в роботе. Четвёртое – показывает полученный массив пользователю в виде уведомления. Пятое – записывает в лог сообщение об успешной генерации. Шестое – возвращает массив названий в основную программу.

Робот должен самостоятельно придумывать названия заявок, используя искусственный интеллект. Это демонстрирует интеграцию больших языковых моделей (LLM) в RPA-процесс.

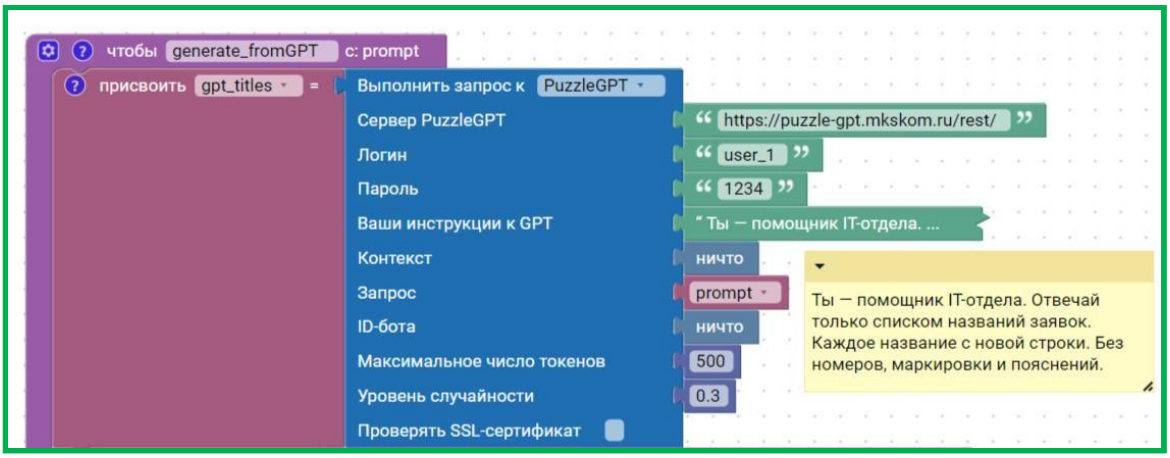
Функция принимает один параметр – prompt. Это текстовая строка, содержащая запрос к GPT.

Шаг 1. Выполнение запроса к GPT

Какой блок используется: Выполнить запрос к GPT

Что делаем в этом блоке: Настраиваем подключение к серверу PuzzleGPT. Указываем адрес сервера, логин и пароль. Задаём инструкции для GPT, которые определяют его роль и стиль ответа. Пишем промпт – конкретную задачу, которую GPT должен выполнить. Настраиваем технические параметры: максимальное количество токенов и температуру.

Что возвращает блок: Возвращает строку с ответом GPT.



Шаг 2. Преобразование строки в массив

Какой блок используется: Сделать список из текста с разделителем

Что делаем в этом блоке: Берём строку, полученную от GPT, и разделяем её на отдельные элементы. В качестве разделителя указываем символ вертикальной черты |. Блок находит все места в строке, где стоит этот символ, и разбивает строку на части.

Что возвращает блок: Возвращает массив (список) строк. Каждый элемент массива – одно название заявки.

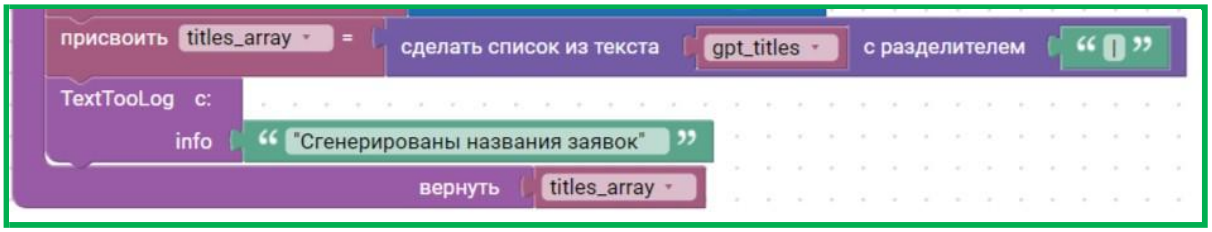
Шаг 3. Запись в лог

Какой блок используется: TextTooLog (наша собственная функция)

Что делаем в этом блоке: Вызываем функцию TextTooLog и передаём ей сообщение "Сгенерированы названия заявок". Это сообщение записывается в файл log.txt с указанием текущего времени.

Шаг 5. Возврат результата

Что делаем: Возвращаем массив titles_array в основную программу. Теперь вызывающий код может использовать этот массив для заполнения заявок.



Важные замечания:

Первое. Температура 0.3 выбрана для получения стабильных и предсказуемых ответов. Если вы хотите более креативные и разнообразные названия, температуру можно повысить до 0.7 или 0.9.

Второе. Разделитель | (вертикальная черта) выбран потому, что этот символ почти никогда не встречается в обычных названиях заявок. Это гарантирует правильное разделение.

Третье. Максимальное количество токенов 500 достаточно для 10 названий. Если названия будут длиннее, значение можно увеличить.

Функция select_priority

Функция select_priority предназначена для выбора приоритета заявки в десктопном приложении. Работа основана на технологии Machine Vision – робот ищет элементы интерфейса по их изображениям.

Что делает функция:

Первое – кликает по выпадающему списку Priority, чтобы раскрыть его. Второе – делает задержку на 2 секунды, чтобы список успел раскрыться. Третье – проверяет, какой приоритет передан в функцию (Low, Medium или High). Четвёртое – кликает по соответствующему изображению (priority_low.png, priority_medium.png или priority_high.png). Пятое – записывает в лог сообщение о том, какой приоритет был выбран.

Почему это важно: В условиях работы через удалённые рабочие столы или виртуальные среды традиционная автоматизация на основе UI-селекторов может быть недоступна. Machine Vision позволяет роботу взаимодействовать с интерфейсом, анализируя изображение экрана.

Функция принимает один параметр – current_priority. Это строковое значение, которое может быть равно 'Low', 'Medium' или 'High'. Функция использует это значение, чтобы определить, на какую картинку нужно кликнуть.

Шаг 1. Клик по выпадающему списку Priority

Какой блок используется: Клик по картинке

Что делаем в этом блоке: Указываем путь к изображению priority_dropdown.png – это эталонное изображение выпадающего списка. Указываем точность совпадения 0.8 (80 процентов). Блок ищет это изображение на экране и кликает по нему левой кнопкой мыши.

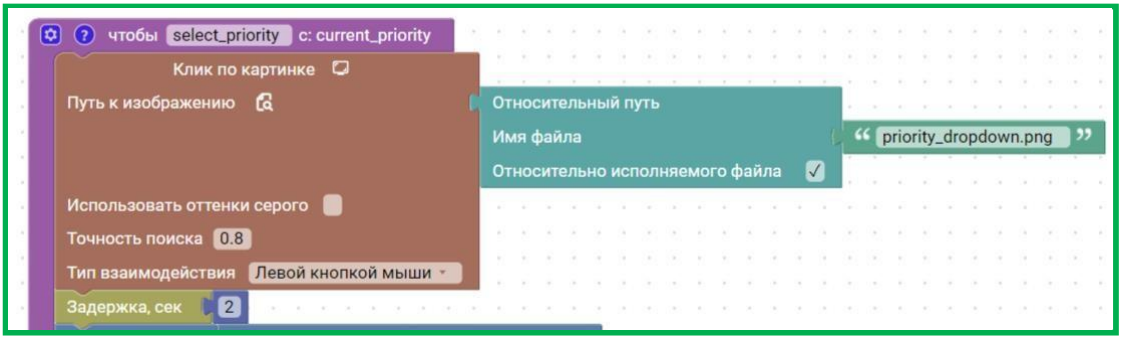
Что происходит на экране: Открывается выпадающий список с тремя вариантами: Low, Medium, High.

Шаг 2. Задержка

Какой блок используется: Задержка

Что делаем в этом блоке: Указываем время задержки – 2 секунды. Робот просто ждёт указанное количество времени, ничего не делая.

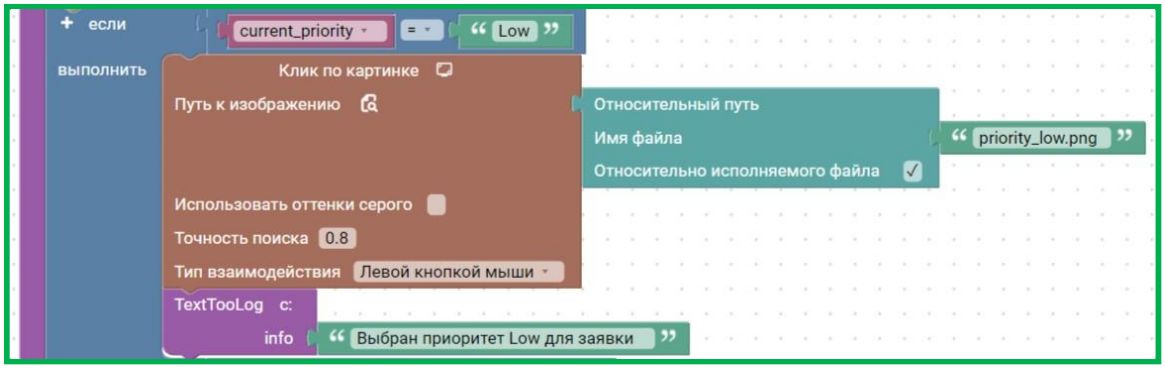
Зачем нужна задержка: Выпадающему списку нужно время, чтобы полностью раскрыться и отрисовать все элементы. Если кликнуть по элементу сразу, робот может не найти нужную картинку, потому что список ещё не отобразился на экране.



Шаг 3. Проверка значения приоритета (Условие)

Какой блок используется: Конструкция if – elif – else

Что делаем в этом блоке: Проверяем, какое значение пришло в параметре current_priority. Сравниваем его со строками 'Low', 'Medium' и 'High'. В зависимости от результата выполняем разные действия.



Шаг 4. Выбор приоритета Low (ветка если)

Какой блок используется: Клик по картинке

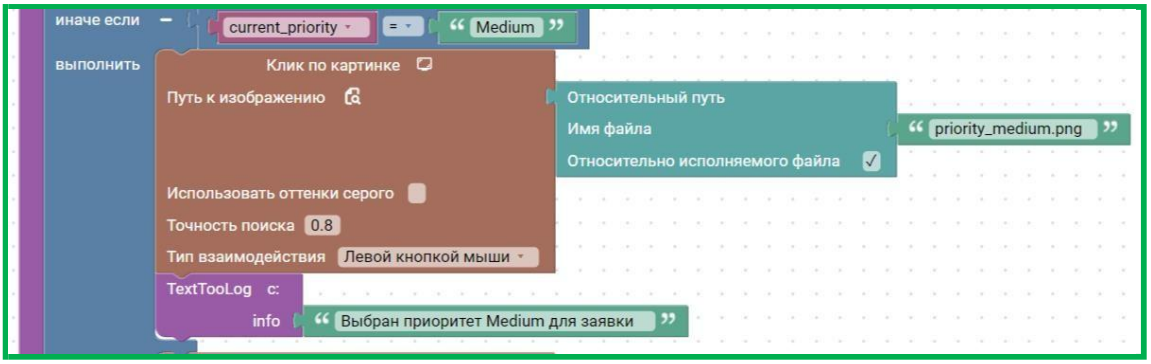
Что делаем в этом блоке: Указываем путь к изображению priority_low.png – это эталонное изображение элемента Low в выпадающем списке. Блок ищет это изображение на экране и кликает по нему.

Что происходит на экране: Выбирается значение Low в выпадающем списке. Список закрывается, в поле Priority отображается Low.

Шаг 5. Лог для приоритета Low

Какой блок используется: TextTooLog (наша собственная функция)

Что делаем в этом блоке: Вызываем функцию TextTooLog и передаём ей сообщение "Выбран приоритет Low для заявки". Это сообщение записывается в файл log.txt с указанием текущего времени.



Шаг 6. Выбор приоритета Medium (ветка elif)

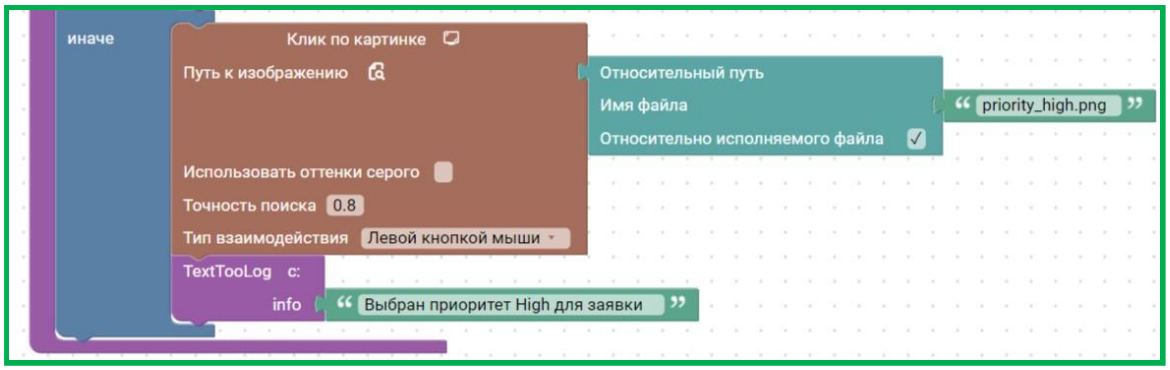
Какой блок используется: Клик по картинке

Что делаем в этом блоке: Указываем путь к изображению priority_medium.png – эталонное изображение элемента Medium. Блок находит его на экране и кликает.

Шаг 7. Лог для приоритета Medium

Какой блок используется: TextTooLog

Что делаем: Записываем в лог сообщение "Выбран приоритет Medium для заявки".



Шаг 8. Выбор приоритета High (ветка else)

Какой блок используется: Клик по картинке

Что делаем в этом блоке: Указываем путь к изображению priority_high.png – эталонное изображение элемента High. Блок находит его на экране и кликает.

Важно: Ветка else срабатывает не только для значения 'High', но и для любого другого значения, которое не равно 'Low' или 'Medium'. Это сделано для надёжности – если вдруг придёт неизвестное значение, робот всё равно выберет High как приоритет по умолчанию.

Шаг 9. Лог для приоритета High

Какой блок используется: TextTooLog

Что делаем: Записываем в лог сообщение "Выбран приоритет High для заявки".

Шаг 10. Завершение функции

Функция не возвращает никакого значения. Она просто выполняет выбор приоритета и записывает информацию в лог.

Важные замечания:

Первое. Изображения priority_dropdown.png, priority_low.png, priority_medium.png и priority_high.png должны быть подготовлены заранее. Нужно запустить приложение вручную, сделать скриншоты каждого элемента и сохранить их в папку с роботом.

Второе. Точность 0.8 означает 80-процентное совпадение. Это значение подходит для большинства случаев. Если робот не находит изображение, точность можно уменьшить. Если находит не то, что нужно – увеличить.

Третье. Задержка в 2 секунды подобрана экспериментально. Если приложение работает медленно, задержку можно увеличить до 3-5 секунд. Если быстро – уменьшить до 1 секунды.

Четвёртое. Ветка else выбирает High для любого значения, не равного Low или Medium. Это сделано для надёжности – даже если в Excel попадёт пустое зна- чение или опечатка, робот всё равно выберет какой-то приоритет.

Функция read_excel

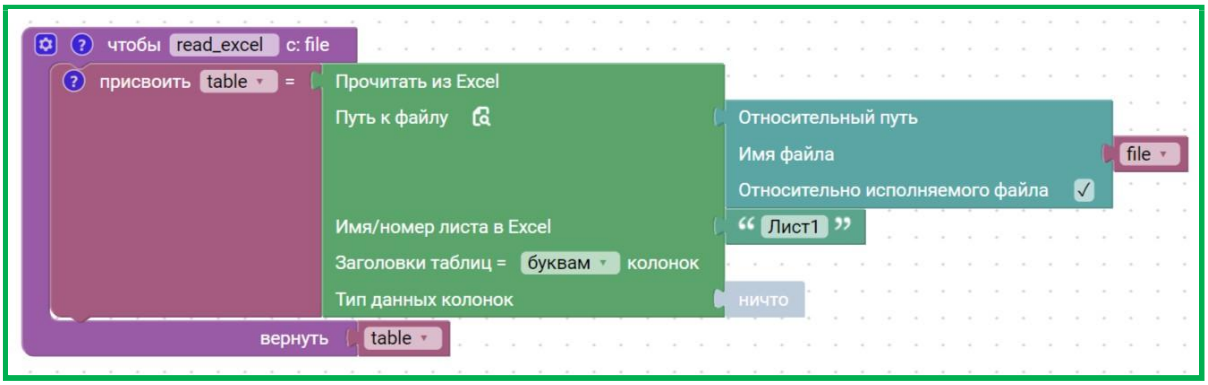
Функция read_excel предназначена для чтения данных из Excel-файла, который содержит информацию о сотрудниках и приоритетах заявок.

Что делает функция:

Первое – получает полный путь к Excel-файлу по его имени. Второе – читает данные с первого листа файла. Третье – возвращает таблицу с данными в основную программу для дальнейшей обработки.

Почему это важно: Входные данные для робота хранятся в Excel-файле. Функция читает этот файл и преобразует его в формат, с которым удобно рабо- тать внутри Puzzle RPA. Робот получает список сотрудников и соответствующих им приоритетов, чтобы затем заполнять заявки.

Функция принимает один параметр – file. Это строковое значение, содержащее имя Excel-файла, который нужно прочитать.



Шаг 1. Получение полного пути к файлу

Какой блок используется: Относительный путь

Что делаем в этом блоке: Передаём в блок имя файла, которое пришло в параметре file. Блок автоматически определяет папку, в которой находится исполняемый файл робота, и добавляет к ней имя файла.

Что возвращает блок: Возвращает полный путь к файлу. Почему это важно: Робот не знает, где лежит файл, если ему просто передать имя. Блок Относительный путь решает эту проблему – он всегда находит файл в папке с роботом, даже если робота перенести на другой компьютер.

Шаг 2. Чтение данных из Excel

Какой блок используется: Прочитать из Excel

Что делаем в этом блоке: Передаём в блок полный путь к файлу, полученный на предыдущем шаге. Указываем имя листа, с которого нужно читать данные – 'Лист1'. Указываем параметр True, который означает, что первая строка файла содержит заголовки столбцов.

Что возвращает блок: Возвращает таблицу данных. К этой таблице можно обращаться по номерам строк и столбцов или по именам колонок.

Шаг 3. Возврат результата

Что делаем: Возвращаем таблицу table в основную программу. Теперь вызывающий код может использовать эту таблицу для получения данных о сотрудниках и приоритетах.

Функция fill_form

Функция fill_form предназначена для автоматического заполнения и отправки 10 заявок в десктопном приложении. Она использует технологию Machine Vision для взаимодействия с интерфейсом и получает данные из двух источников: Excel-файла (сотрудники и приоритеты) и GPT (названия заявок).

Что делает функция:

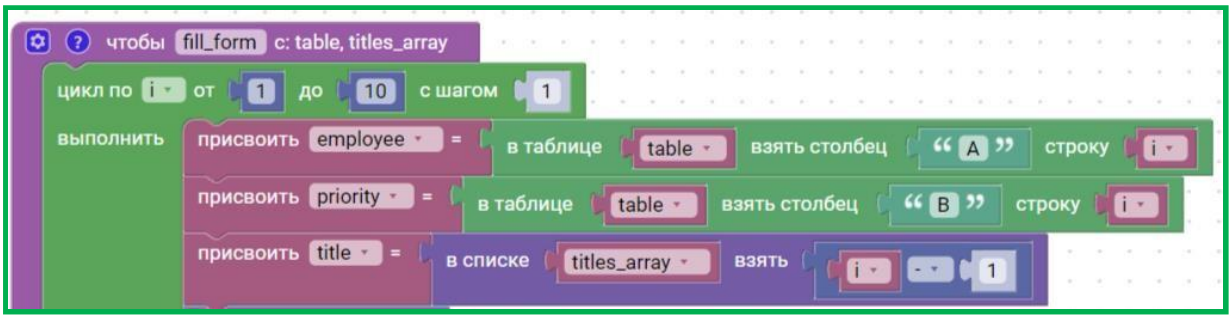
Первое – запускает цикл на 10 итераций (по количеству заявок). Второе – на каждой итерации получает из Excel имя сотрудника и приоритет, а из массива GPT – название заявки. Третье – заполняет поле Request Title сгенерированным названием. Четвёртое – заполняет поле Employee именем сотрудника. Пятое – выбирает приоритет из выпадающего списка. Шестое – нажимает кнопку Add Request для добавления заявки в таблицу. Седьмое – записывает в лог информа- цию об успешном вводе каждой заявки.

Это основная бизнес-логика робота. Функция демонстрирует комплексную автоматизацию, где сочетаются Machine Vision, работа с Excel и интеграция с искусственным интеллектом.

Функция принимает два параметра:

Параметр table – это таблица с данными из Excel-файла. Содержит две колонки: колонка A с именами сотрудников (Employee) и колонка B с приоритетами (Priority). Всего 10 строк данных.

Параметр titles_array – это массив строк, сгенерированных GPT. Содержит 10 названий заявок. Каждый элемент массива – одно название.



Шаг 1. Начало цикла

Какой блок используется: Цикл по (от до с шагом)

Что делаем в этом блоке: Создаём цикл, который будет выполняться 10 раз. Переменная i принимает значения от 1 до 10 включительно. На каждой итерации обрабатывается одна заявка.

Шаг 2. Получение имени сотрудника из Excel

Какой блок используется: Получить значение ячейки табличных данных

Что делаем в этом блоке: Обращаемся к таблице table, указываем колонку 'A' (где хранятся имена сотрудников) и индекс i (текущий номер итерации). Блок извлекает значение из указанной ячейки.

Что возвращает блок: Возвращает имя сотрудника.

Шаг 3. Получение приоритета из Excel

Какой блок используется: Получить значение ячейки табличных данных

Что делаем в этом блоке: Обращаемся к таблице table, указываем колонку 'B' (где хранятся приоритеты) и индекс i. Блок извлекает значение приоритета.

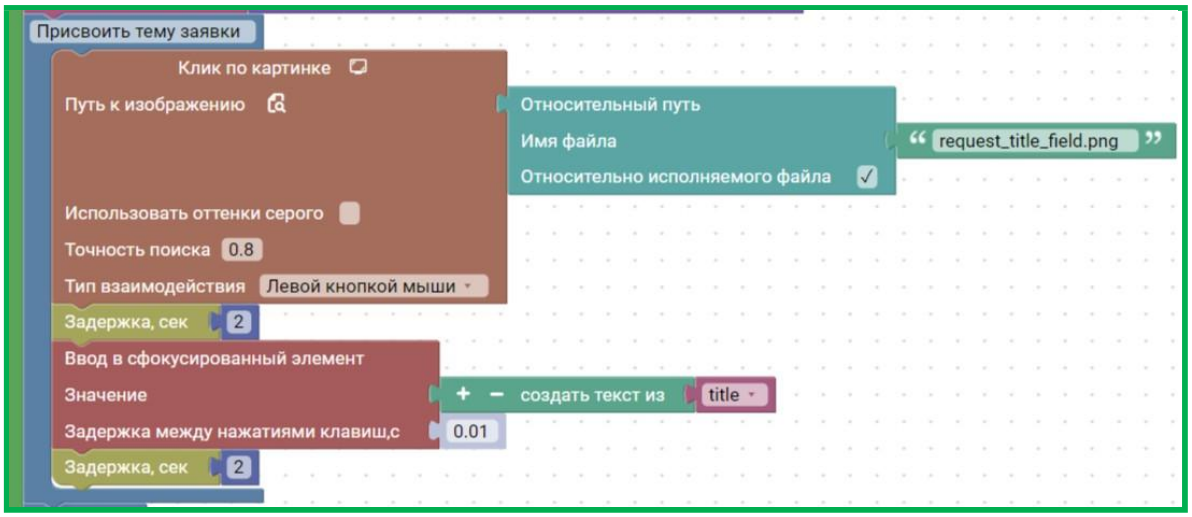
Что возвращает блок: Возвращает приоритет заявки.

Шаг 4. Получение названия заявки из массива GPT

Какой блок используется: Обращение к элементу массива по индексу

Что делаем в этом блоке: Берём из массива titles_array элемент с индексом (i – 1). Почему i минус 1? Потому что в программировании счёт элементов массива начинается с нуля. Когда i=1, нам нужен элемент с индексом 0 – первое название. Когда i=2, нужен индекс 1 – второе название, и так далее.

Что возвращает блок: Возвращает название заявки. Например, на первой итерации – первое название из списка GPT, на второй – второе название и так далее.



Шаг 5. Заполнение поля Request Title

Подшаг 5.1 – Клик по полю Title

Какой блок используется: Клик по картинке

Что делаем: Указываем изображение request_title_field.png. Робот находит это поле на экране и кликает по нему, чтобы активировать ввод текста.

Подшаг 5.2 – Задержка

Какой блок используется: Задержка

Что делаем: Ждём 2 секунды, чтобы поле точно успело активироваться.

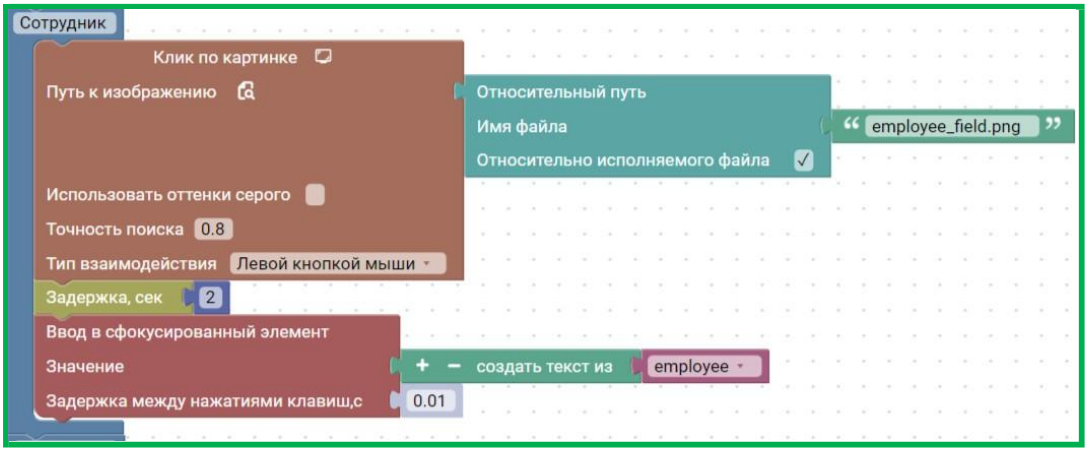
Подшаг 5.3 – Ввод названия

Какой блок используется: Ввод в сфокусированный элемент

Что делаем: Вводим текст title (название заявки) в то поле, которое сейчас в фокусе. А в фокусе у нас поле Request Title, потому что мы кликнули по нему.

Подшаг 5.4 – Задержка

Что делаем: Ждём ещё 2 секунды перед переходом к следующему полю.



Шаг 6. Заполнение поля Employee

Подшаг 6.1 – Клик по полю Employee

Какой блок используется: Клик по картинке

Название блока: Клик по картинке

Что делаем: Указываем изображение employee_field.png. Робот находит поле Employee и кликает по нему.

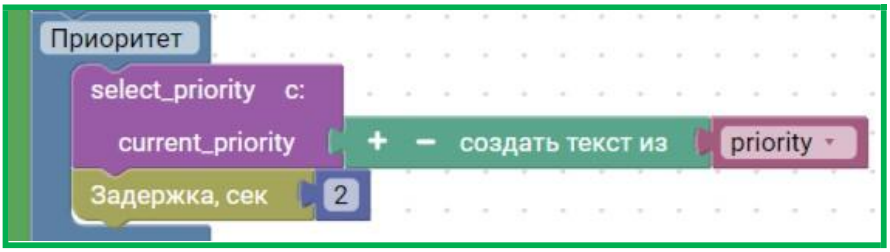
Подшаг 6.2 – Задержка

Что делаем: Ждём 2 секунды.

Подшаг 6.3 – Ввод имени сотрудника

Какой блок используется: Ввод в сфокусированный элемент

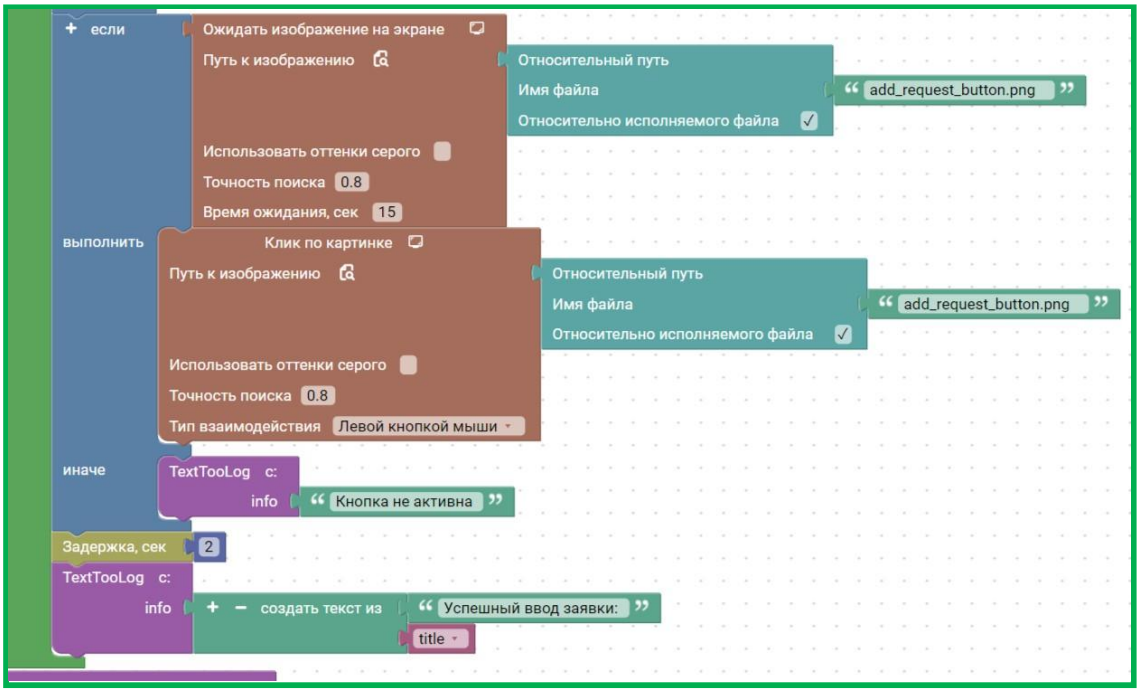
Что делаем: Вводим текст employee (имя сотрудника) в активное поле.



Шаг 7. Выбор приоритета Какой блок используется: Вызов функции select_priority

Что делаем: Вызываем ранее созданную функцию select_priority и передаём ей значение приоритета. Функция сама раскроет выпадающий список, выберет нужный пункт (Low, Medium или High) и запишет в лог.

Задержка после выбора приоритета: Делаем задержку 2 секунды, чтобы список успел закрыться.



Шаг 8. Нажатие кнопки Add Request

Подшаг 8.1 – Ожидание кнопки

Какой блок используется: Ожидать изображение на экране

Название блока: Ожидать изображение на экране

Что делаем: Ждём появления кнопки add_request_button.png на экране. Указываем таймаут 15 секунд. Если кнопка появилась, блок возвращает True. Если не появилась за 15 секунд – возвращает False.

Подшаг 8.2 – Условие проверки

Какой блок используется: if – else

Что делаем: Проверяем, вернул ли блок ожидания True. Если да – кнопка активна и по ней можно кликнуть. Если нет – кнопка не появилась, записываем ошибку в лог.

Подшаг 8.3 – Клик по кнопке (ветка если)

Какой блок используется: Кликаем по кнопке Add Request.

Что делаем: Кликаем по кнопке Add Request.

Подшаг 8.4 – Лог ошибки (ветка иначе)

Какой блок используется: TextTooLog

Что делаем: Записываем в лог сообщение 'Кнопка не активна'.

Задержка после добавления: Делаем задержку 2 секунды, чтобы заявка успела добавиться в таблицу.

Шаг 9. Лог успешного ввода

Какой блок используется: TextTooLog

Что делаем: Записываем в лог сообщение 'Успешный ввод заявки:' и название заявки. Это позволяет отследить, какая именно заявка была добавлена.

Шаг 10. Завершение цикла

Цикл повторяется 10 раз. После того как все 10 заявок обработаны, функция завершает работу.

Важные замечания:

Первое. Все изображения (request_title_field.png, employee_field.png, add_request_button.png и другие) должны быть подготовлены заранее и лежать в папке с роботом.

Второе. Задержки в 2 секунды подобраны экспериментально. Если приложение работает медленно, задержки можно увеличить.

Третье. Таймаут ожидания кнопки Add Request – 15 секунд. Этого достаточно для нормальной работы приложения.

Четвертое. Функция использует keyboard_input.keyboard_input_focused для ввода текста. Этот блок вводит текст в то поле, которое находится в фокусе. Поэтому перед вводом мы обязательно кликаем по нужному полю.

Функция process_high_priority

Функция process_high_priority предназначена для обработки заявки с приоритетом High. Она находит такую заявку в таблице, меняет её статус на Done, удаляет её, подтверждает удаление и сохраняет скриншот результата.

Что делает функция:

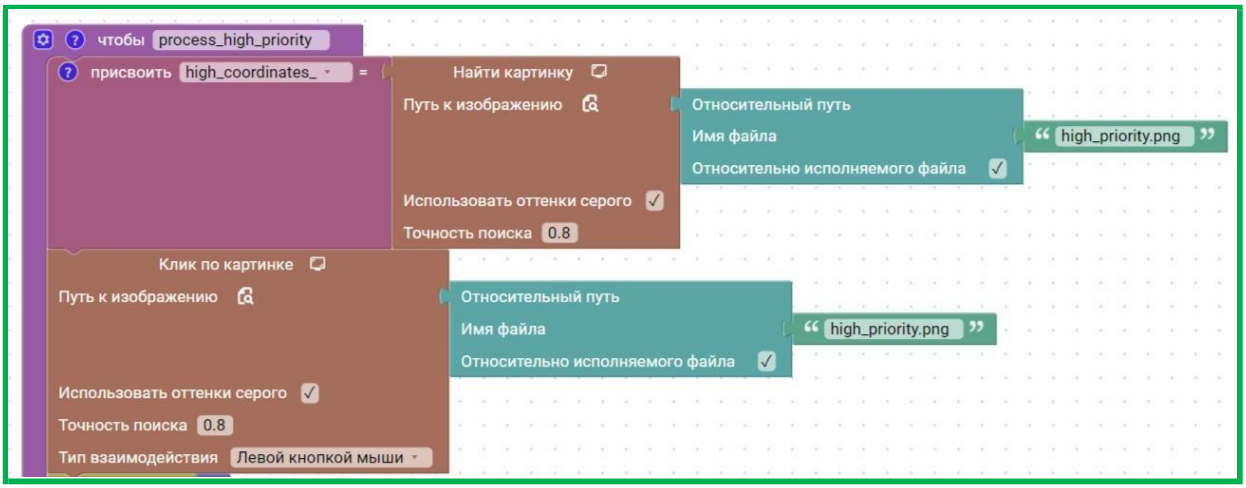
Первое – находит на экране изображение high_priority.png (ячейка с приоритетом High в таблице). Второе – кликает по ней, выделяя всю строку заявки. Третье – нажимает кнопку Mark as Done, меняя статус на Done. Четвёртое – нажимает кнопку Delete Selected, вызывая окно подтверждения удаления. Пятое – ожидает появление окна подтверждения и нажимает кнопку Confirm. Шестое – при необходимости закрывает дополнительное информационное окно с кнопкой OK. Седьмое – записывает в лог результат обработки. Восьмое – сохраняет скриншот

итоговой таблицы в файл result_table.png. Девятое – возвращает результат выполнения.

Это демонстрация полного цикла обработки заявки – от поиска до удаления. Также здесь показана работа с модальными окнами и подтверждениями, что часто встречается в реальных бизнес-процессах.

Функция не принимает входных параметров. Она работает автономно, используя предварительно подготовленные изображения-эталоны.

Функция возвращает строковое значение result, которое содержит сообщение о результате обработки.



Шаг 1. Поиск заявки с приоритетом High

Какой блок используется: Найти картинку

Что делаем в этом блоке: Указываем изображение high_priority.png – это эталон ячейки с текстом High в таблице заявок. Указываем точность совпадения 0.8 (80 процентов). Блок ищет это изображение на экране и возвращает его координаты.

Что возвращает блок: Возвращает координаты найденного изображения (x, y, ширина, высота). Эти координаты сохраняются в переменную high_coordinates_, хотя в дальнейшем они не используются – поиск нужен только для проверки наличия элемента.

Шаг 2. Выбор строки таблицы

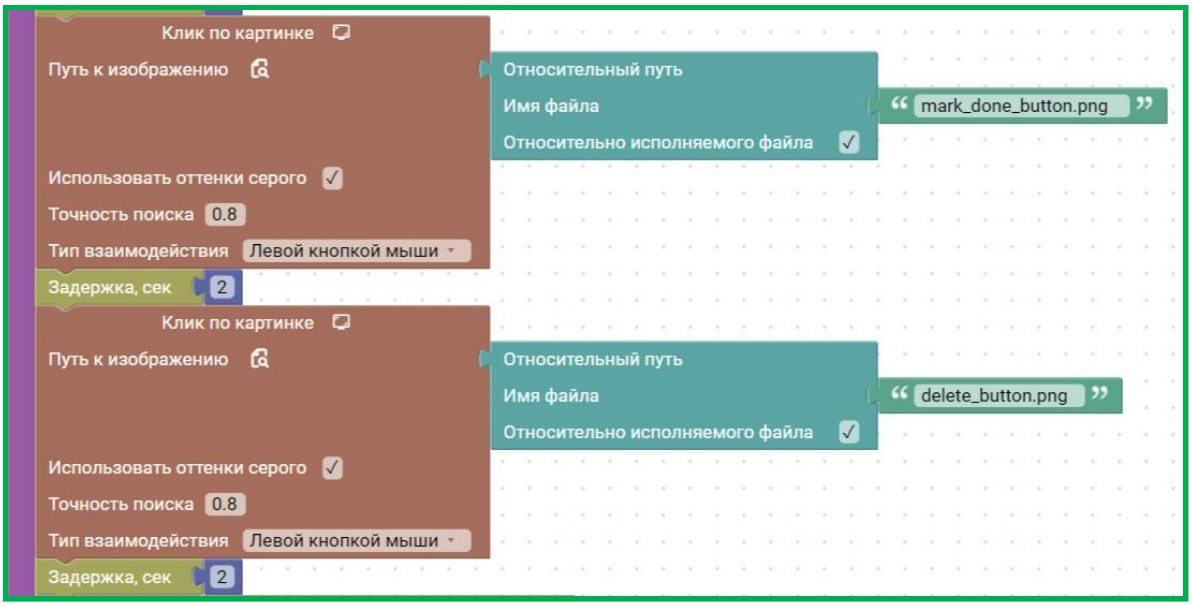
Какой блок используется: Клик по картинке

Что делаем в этом блоке: Кликаем по тому же изображению high_priority.png. Это выделяет всю строку таблицы, в которой находится эта заявка.

Что происходит на экране: Строка с заявкой подсвечивается, показывая, что она выбрана.

Шаг 3. Задержка после выбора строки

Что делаем: Ждём 2 секунды, чтобы выделение строки точно зафиксировалось.



Шаг 4. Изменение статуса на Done

Какой блок используется: Клик по картинке

Что делаем: Находим и кликаем по кнопке mark_done_button.png. Эта кнопка меняет статус выбранной заявки на Done.

Что происходит на экране: Статус заявки в таблице меняется с New на Done.

Шаг 5. Задержка после изменения статуса

Что делаем: Ждём 2 секунды.

Шаг 6. Удаление записи

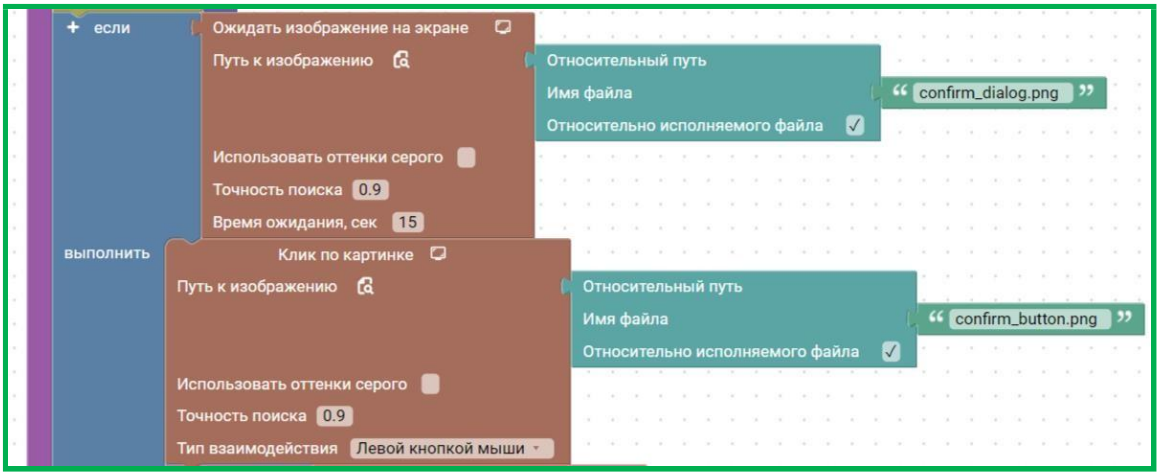
Какой блок используется: Клик по картинке

Что делаем: Находим и кликаем по кнопке delete_button.png. Эта кнопка вызывает окно подтверждения удаления.

Что происходит на экране: Появляется модальное окно с вопросом: действительно ли вы хотите удалить запись?

Шаг 7. Задержка после вызова удаления

Что делаем: Ждём 2 секунды, чтобы окно подтверждения успело появиться.



Шаг 8. Ожидание окна подтверждения

Какой блок используется: Ожидать изображение на экране

Что делаем: Ожидаем появления окна подтверждения confirm_dialog.png. Указываем точность 0.9 (90 процентов) и таймаут 15 секунд.

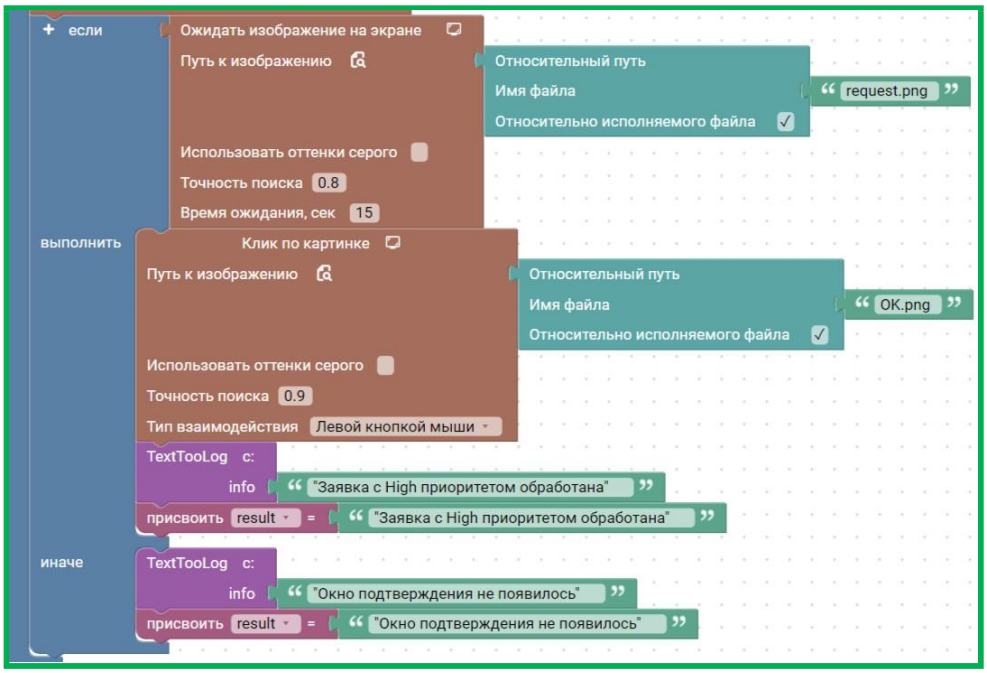
Что возвращает блок: Возвращает True, если окно появилось в течение 15 секунд. Возвращает False, если окно не появилось.

Шаг 9. Подтверждение удаления (ветка если)

Какой блок используется: Клик по картинке

Что делаем: Кликаем по кнопке confirm_button.png внутри окна подтверждения. Это подтверждает удаление заявки.

Что происходит на экране: Заявка удаляется из таблицы.



Шаг 10. Ожидание информационного окна (внутреннее условие)

Какой блок используется: Ожидать изображение на экране

Что делаем: После удаления может появиться информационное окно с изображением request.png. Ожидаем его появление с точностью 0.8 и таймаутом 15 секунд.

Что это за окно: В некоторых приложениях после успешного удаления появляется всплывающее уведомление, которое нужно закрыть.

Шаг 11. Закрытие информационного окна

Какой блок используется: Клик по картинке

Что делаем: Кликаем по кнопке OK.png, чтобы закрыть информационное окно.

Шаг 12. Лог успеха

Какой блок используется: TextTooLog

Что делаем: Записываем в лог сообщение "Заявка с High приоритетом обработана".

Шаг 13. Установка результата (успех)

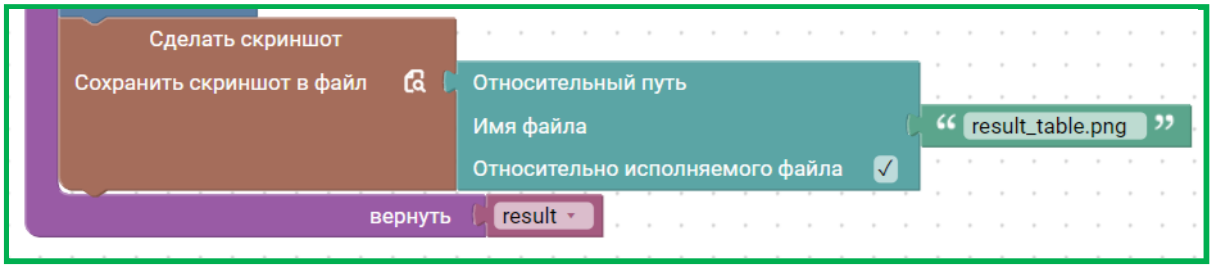
Что делаем: Присваиваем переменной result значение "Заявка с High приоритетом обработана".

Шаг 14. Обработка случая, когда информационное окно не появилось

Если окно с изображением request.png не появилось, то:

• Записываем в лог сообщение "Окно подтверждения не появилось"

• Присваиваем переменной result значение "Окно подтверждения не появилось"



Шаг 15. Сохранение скриншота

Какой блок используется: Сделать скриншот

Что делаем: Сохраняем скриншот всего экрана или окна приложения в файл result_table.png. Это позволяет зафиксировать итоговое состояние таблицы после обработки.

Шаг 16. Возврат результата

Что делаем: Возвращаем переменную result в основную программу. Теперь вызывающий код знает, успешно ли прошла обработка

Важные замечания:

Первое. Изображение high_priority.png должно быть подготовлено заранее. Это скриншот ячейки со значением High в таблице заявок. Важно, чтобы изображение было обрезано точно по границам ячейки, без лишних элементов.

Второе. Точность поиска 0.8 (80 процентов) используется для большинства изображений. Для окна подтверждения и кнопок используется точность 0.9 (90 процентов), так как эти элементы должны распознаваться более строго.

Третье. В функции используется вложенное условие: сначала проверяется наличие окна подтверждения, а затем – наличие информационного окна. Это пример обработки нескольких последовательных модальных окон.

Четвертое. Скриншот сохраняется в любом случае – и при успешном удалении, и при ошибке. Это позволяет проанализировать состояние экрана в момент завершения работы функции.

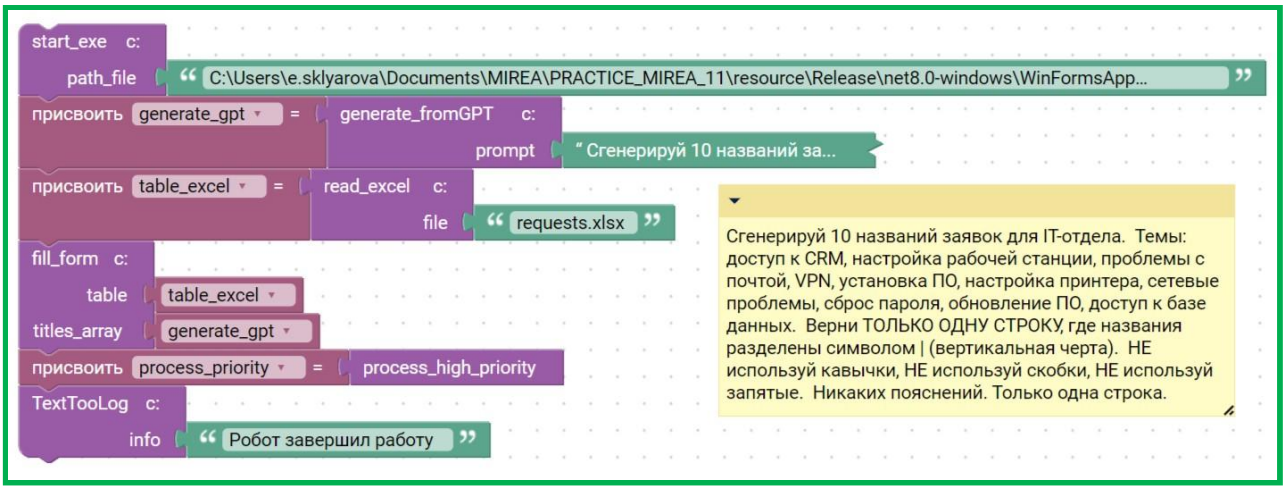
Основной процесс Main

Основной процесс является точкой входа в робота. Он координирует работу всех вспомогательных функций и реализует полный сценарий автоматизации.

Что делает основной процесс:

Первое – запускает десктопное приложение через функцию start_exe. Второе – генерирует 10 названий заявок с помощью искусственного интеллекта через функцию generate_fromGPT. Третье – читает данные из Excel-файла через функцию read_excel. Четвёртое – заполняет 10 заявок в приложении через функцию fill_form. Пятое – обрабатывает заявку с приоритетом High (меняет статус, удаляет) через функцию process_high_priority. Шестое – записывает в лог сообщение о завершении работы робота.

Это демонстрация комплексной автоматизации, где объединены все изученные технологии: Machine Vision, работа с Excel, интеграция с искусственным интеллектом (GPT), работа с модальными окнами и логирование.



Шаг 1. Запуск приложения

Какой блок используется: Вызов функции start_exe

Что делаем: Вызываем функцию start_exe и передаём ей полный путь к исполняемому файлу десктопного приложения.

Что происходит внутри функции:

• Функция запускает приложение по указанному пути

• Ожидает появления поля Request Title на экране (таймаут 15 секунд)

• Если поле появилось – записывает в лог «Приложение загружено»

• Если поле не появилось – записывает в лог «Приложение не загружено»

Шаг 2. Генерация названий заявок через GPT

Какой блок используется: Вызов функции generate_fromGPT

Что делаем: Вызываем функцию generate_fromGPT и передаём ей промпт – текст с заданием для искусственного интеллекта. Результат сохраняем в переменную generate_gpt.

Промпт для GPT: «Сгенерируй 10 названий заявок для IT-отдела. Темы: доступ к CRM, настройка рабочей станции, проблемы с почтой, VPN, установка ПО, настройка принтера, сетевые проблемы, сброс пароля, обновление ПО, доступ к базе данных. Верни список в одну строку, где названия разделены запятой и пробелом (", "). Никаких номеров, каждая новая строка – не нужно.»

Что происходит внутри функции:

• Функция отправляет запрос к серверу PuzzleGPT

• Получает строку с названиями, разделёнными вертикальной чертой

• Преобразует строку в массив из 10 элементов

• Показывает уведомление с массивом

• Возвращает массив названий

Что сохраняется в переменную generate_gpt: Массив из 10 строк – названий заявок.

Шаг 3. Чтение данных из Excel

Какой блок используется: Вызов функции read_excel

Что делаем: Вызываем функцию read_excel и передаём ей имя файла 'requests.xlsx'. Результат сохраняем в переменную table_excel.

Что происходит внутри функции:

• Функция получает полный путь к файлу requests.xlsx в папке с роботом

• Читает данные с первого листа Excel-файла

• Возвращает таблицу с двумя колонками: Employee и Priority

Что сохраняется в переменную table_excel: Таблица с данными из Excel. Колонка A содержит имена сотрудников (Ivan Petrov, Anna Smirnova и так далее). Колонка B содержит приоритеты (High, Medium, Low).

Шаг 4. Заполнение заявок

Какой блок используется: Вызов функции fill_form

Что делаем: Вызываем функцию fill_form и передаём ей два параметра: таблицу из Excel (table_excel) и массив названий от GPT (generate_gpt).

Что происходит внутри функции:

• Запускается цикл на 10 итераций

• Все 10 заявок добавляются в таблицу приложения

Шаг 5. Обработка заявки с приоритетом High

Какой блок используется: Вызов функции process_high_priority

Что делаем: Вызываем функцию process_high_priority и сохраняем результат в переменную process_priority.

Что происходит внутри функции:

• Находит в таблице заявку с приоритетом High

• Выделяет строку с этой заявкой

• Меняет статус на Done

• Удаляет заявку

• Подтверждает удаление в модальном окне

• Сохраняет скриншот result_table.png

• Возвращает результат обработки

Шаг 6. Лог завершения работы

Какой блок используется: TextTooLog

Что делаем: Вызываем функцию TextTooLog и передаём ей сообщение 'Робот завершил работу'.

Что происходит: Сообщение записывается в файл log.txt с указанием текущей даты и времени.

В рамках данной практической работы была успешно реализована автоматизация процесса регистрации и обработки сервисных заявок с использованием RPA-робота в среде Puzzle RPA. В ходе выполнения работы были применены передовые технологии машинного зрения (Machine Vision) и искусственного интеллекта (LLM), что позволило создать устойчивого к внешним условиям робота, способного работать в виртуальных средах и через удалённые рабочие столы.

Кейс 2. Автоматизация веб-приложения с использованием Machine Vision в Puzzle

Постановка задачи

Компания EduStreamLab разрабатывает онлайн-образовательные программы и интенсивы для специалистов в сфере IT.

Веб-портал регистрации уже создан и активно используется. В условиях виртуальных рабочих станций, удалённых браузеров и Citrix/RDP-сессий традицион-

ные селекторы недоступны. Поэтому применяется подход Machine Vision – робот находит элементы интерфейса по их изображению.

Необходимо разработать RPA-робота в Puzzle RPA, который автоматически авторизуется в веб-приложении, заполняет 8 тестовых заявок из Excel-файла, отправляет их, анализирует заявки (Online → Approved, Offline → Rejected), удаляет одну запись и формирует итоговый отчёт с помощью GPT.

Для работы робота были подготовлены следующие изображения элементов интерфейса. Поле логина – login_field.png. Поле пароля – password_field.png. Кнопка входа – sign_in_button.png. Поле имени – name.png. Поле почты – mail.png. Поле мероприятия – event.png. Поле формата – format.png. Значение Online – online.png. Значение Offline – offline.png. Кнопка принятия условий – button_yes.png. Кнопка отправки – submit_button.png. Таблица заявок – table_request.png. Кнопка удаления – delete.png. Кнопка подтверждения удаления – delete_button.png. Онлайн заявка в таблице – online_table.png. Офлайн заявка в таблице – offline_format.png. Отметка одобрено – mark_approved.png. Отметка отклонено – mark_rejected.png.

Исходные данные: Excel-файл с именем applications.xlsx содержит 8 тестовых заявок. В файле используются следующие колонки. Колонка A содержит имя участника. Колонка B содержит email. Колонка C содержит название мероприятия. Колонка D содержит формат участия – Online или Offline.

Перед запуском робота необходимо запустить локальный веб-сервер командой python -m http.server 8000. После этого веб-приложение становится доступно по адресу http://localhost:8000/index.html. Важно: сервер должен оставаться запущенным на всё время работы робота.

В рамках практической работы был разработан продвинутый RPA-робот в среде Puzzle RPA.

После выполнения робота все 8 заявок будут добавлены. Заявки с форматом Online получат статус Approved. Заявки с форматом Offline получат статус Rejected. Одна запись будет удалена. Будет сформирован аналитический отчёт GPT о работе процесса.

Функция TextTooLog

Функция TextTooLog предназначена для записи информационных сообщений в текстовый файл лога с указанием точного времени события.

Что делает функция: Первое – получает текущую дату и время. Второе – добавляет к ним переданное в функцию сообщение. Третье – записывает всё в файл с именем log.txt. Каждое сообщение записывается с новой строки.

Функция принимает один параметр – info. Это строковое значение, содержащее сообщение, которое нужно записать в лог.



Шаг 1. Получение текущей даты и времени

Какой блок используется: Получить текущую дату

Что делаем в этом блоке: Указываем формат даты и времени.

Что возвращает блок: Возвращает строку с текущей датой и временем в указанном формате.

Шаг 2. Формирование строки для записи

Какой блок используется: Создать текст из

Что делаем в этом шаге: Нам нужно собрать одну строку из трёх частей. Первая часть – это дата и время, полученные на предыдущем шаге. Вторая часть – это символы двоеточие и пробел, чтобы отделить время от сообщения. Третья часть – это само сообщение, которое пришло в параметре info. В конце добавляем символ перевода строки, чтобы каждое новое сообщение записывалось с новой строки.

Шаг 3. Получение пути к файлу лога

Какой блок используется: Относительный путь

Что делаем в этом блоке: Указываем имя файла, в который будем писать лог. В данном случае это log.txt. Блок автоматически определит полный путь к папке, где находится ваш робот, и добавит к нему имя файла.

Шаг 4. Запись строки в файл

Какой блок используется: Дописать в файл.

Что делаем в этом блоке: Берём сформированную на шаге 2 строку и записываем её в файл, путь к которому мы получили на шаге 3. Важно: мы используем режим дописать, а не перезаписать. Это значит, что новое сообщение добавится в конец файла, а старые сообщения останутся на месте. Указываем кодировку utf-8, чтобы корректно записывались русские буквы.

Функция open_browser

Функция open_browser предназначена для открытия браузера, перехода по указанному URL и проверки загрузки страницы авторизации.

Что делает функция:

Первое – инициализирует сессию браузера Google Chrome в обычном режиме.

Второе – открывает новую вкладку с указанным адресом страницы.

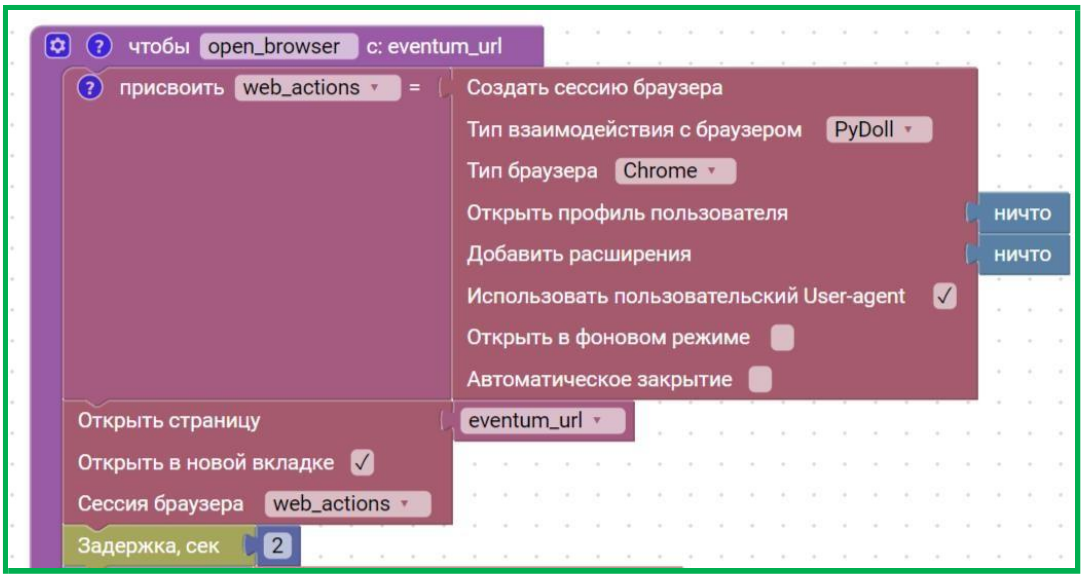
Третье – делает задержку на 2 секунды для ожидания загрузки страницы.

Четвёртое – проверяет, появилось ли на экране изображение поля авторизации.

Пятое – записывает в лог сообщение о том, загрузилось окно для ввода или нет.

Шестое – возвращает объект web_actions для дальнейшего управления браузером.

Функция принимает один параметр – eventum_url. Это строковое значение, содержащее адрес страницы, которую нужно открыть в браузере.



Шаг 1. Создание сессии браузера

Какой блок используется: Создать сессию браузера

Что делаем в этом блоке: Указываем тип браузера google, режим headless="FALSE" (браузер будет виден на экране), параметр close="FALSE" (не закрывать браузер после выполнения), custom_user_agent="TRUE" (использовать стандартный User-Agent).

Что возвращает блок: Возвращает объект web_actions для управления браузером.

Шаг 2. Открытие страницы в браузере

Какой блок используется: Открыть страницу в браузере

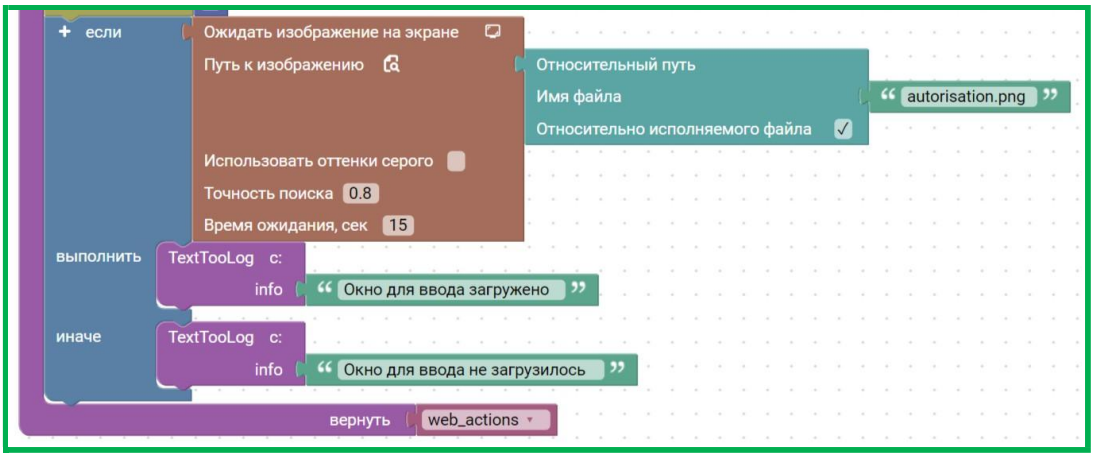
Что делаем в этом блоке: Передаём в блок адрес страницы из параметра eventum_url.

Что возвращает блок: Ничего не возвращает. Открывает указанную страницу в браузере.

Шаг 3. Задержка

Какой блок используется: Задержка

Что делаем в этом блоке: Указываем время задержки 2 секунды. Это нужно, чтобы страница успела полностью загрузиться перед проверкой изображения.



Шаг 4. Ожидание изображения на экране

Какой блок используется: Ожидать изображение на экране

Что делаем в этом блоке: Указываем путь к изображению-эталону autorisation.png. Указываем точность совпадения 0.8 (80 процентов). Указываем таймаут 15 секунд – это максимальное время ожидания. Если элемент появится раньше, блок завершится досрочно.

Что возвращает блок: Возвращает значение True, если изображение найдено в течение таймаута. Возвращает False, если изображение не появилось за 15 се- кунд.

Шаг 5. Проверка результата ожидания (условие)

Какой блок используется: Конструкция Если – выполнить-иначе

Что делаем в этом блоке: Проверяем, что вернул блок ожидания. Если вернул True, значит окно авторизации загрузилось. Если вернул False, значит окно не загрузилось.

Шаг 6. Действие при успешной загрузке (ветка Если)

Какой блок используется: TextTooLog (наша собственная функция)

Что делаем в этом блоке: Вызываем функцию TextTooLog и передаём ей сообщение Окно для ввода загружено. Это сообщение запишется в файл лога с указанием текущего времени.

Шаг 7. Действие при ошибке загрузки (ветка Иначе)

Какой блок используется: TextTooLog (наша собственная функция)

Что делаем в этом блоке: Вызываем функцию TextTooLog и передаём ей сообщение Окно для ввода не загрузилось. Это сообщение запишется в файл лога с указанием текущего времени.

Шаг 8. Возврат объекта браузера

Какой блок используется: Возврат значения

Что делаем в этом блоке: Возвращаем объект web_actions, который был создан на шаге 1.

Что возвращает блок: Возвращает объект web_actions для дальнейшего управления браузером (заполнение полей, нажатие кнопок и т.д.).

Важные замечания:

Первое. Изображение autorisation.png должно быть подготовлено заранее. Нужно открыть страницу вручную, сделать скриншот поля авторизации и сохранить его по указанному пути.

Второе. Точность 0.8 означает 80-процентное совпадение. Это значение подходит для большинства случаев. Если робот не находит изображение, точность можно уменьшить. Если находит не то, что нужно – увеличить.

Третье. Таймаут 15 секунд выбран с учётом того, что страница может загружаться медленно. Если страница загружается дольше, таймаут можно увеличить.

Функция autorisation

Функция autorisation предназначена для авторизации в веб-приложении: ввода логина, пароля и нажатия кнопки входа.

Что делает функция:

Первое – кликает по полю ввода логина и вводит переданный логин.

Второе – кликает по полю ввода пароля и вводит переданный пароль.

Третье – ожидает появления кнопки входа на экране.

Четвёртое – если кнопка входа появилась, нажимает на неё и записывает в лог сообщение об успешном входе.

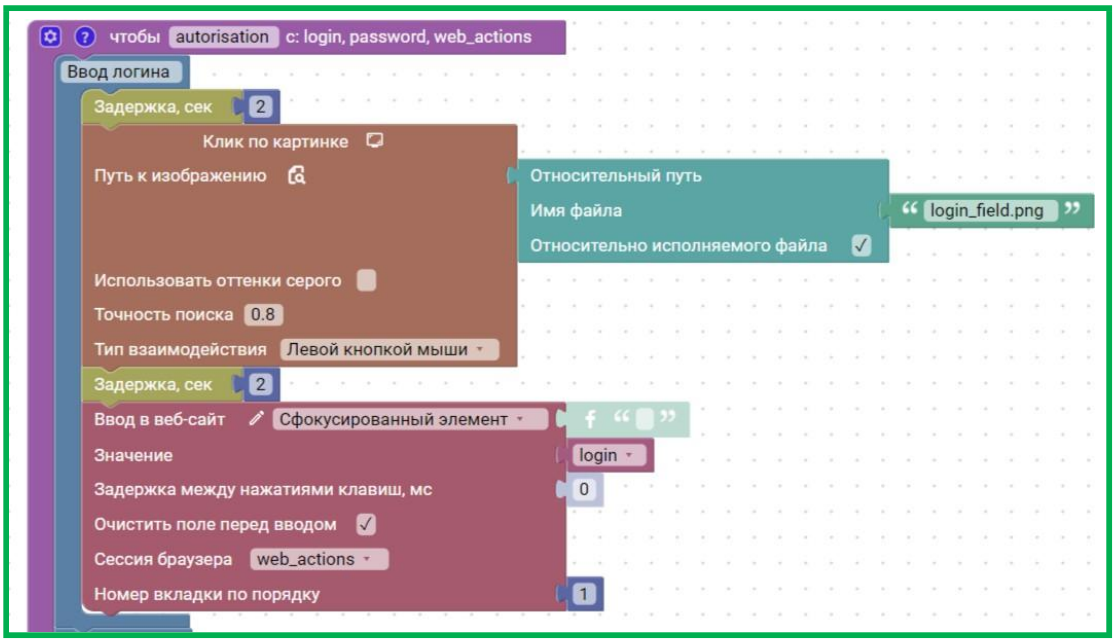
Пятое – если кнопка входа не появилась, записывает в лог сообщение о неуспешной авторизации.

Функция принимает три параметра:

login – строковое значение, содержащее логин пользователя.

password – строковое значение, содержащее пароль пользователя.

web_actions – объект для управления браузером, полученный из функции open_browser.



Шаг 1. Задержка перед вводом логина

Какой блок используется: Задержка

Что делаем в этом блоке: Указываем время задержки 2 секунды. Это нужно, чтобы страница полностью загрузилась перед тем, как робот начнёт вводить данные.

Шаг 2. Клик по полю ввода логина

Какой блок используется: Клик по картинке

Что делаем в этом блоке: Указываем путь к изображению-эталону поля ввода логина login_field.png. Указываем точность совпадения 0.8 (80 процентов). Указываем задержку 0 секунд.

Что возвращает блок: Ничего не возвращает. Кликает мышью по найденному изображению.

Шаг 3. Задержка после клика

Какой блок используется: Задержка

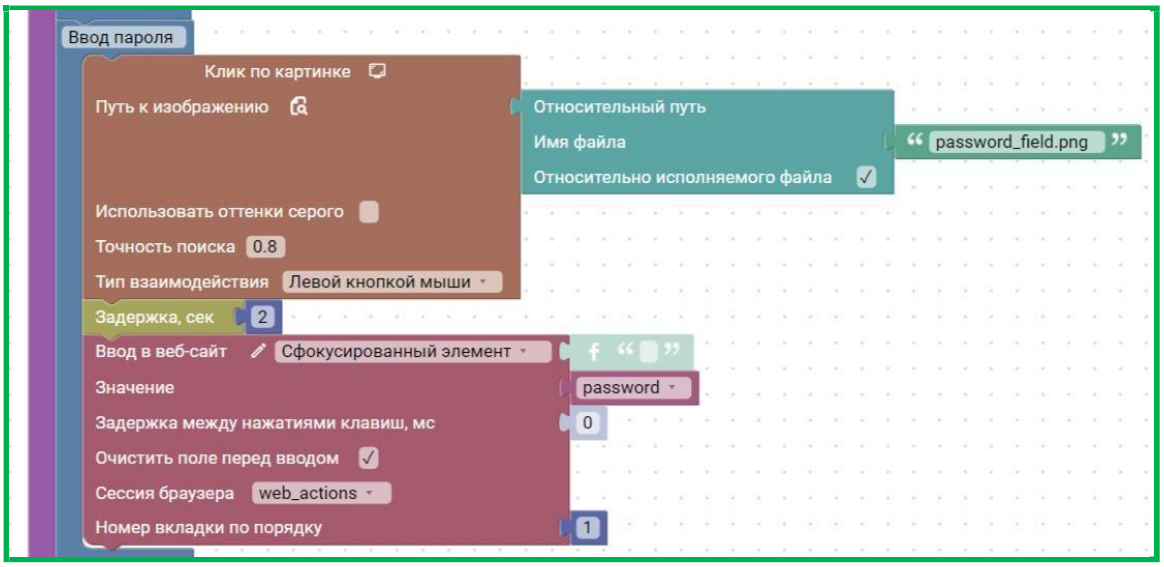
Что делаем в этом блоке: Указываем время задержки 2 секунды. Это нужно, чтобы поле ввода стало активным.

Шаг 4. Ввод логина в браузере

Какой блок используется: Ввод в веб-сайт

Что делаем в этом блоке: Указываем тип элемента (активный элемент на странице). Передаём текст для ввода из параметра login.

Что возвращает блок: Ничего не возвращает. Вводит указанный текст в активное поле ввода.



Шаг 5. Клик по полю ввода пароля

Какой блок используется: Клик по картинке

Что делаем в этом блоке: Указываем путь к изображению-эталону поля ввода пароля password_field.png. Указываем точность совпадения 0.8 (80 процентов). Указываем задержку 0 секунд.

Что возвращает блок: Ничего не возвращает. Кликает мышью по найденному изображению.

Шаг 6. Задержка после клика

Какой блок используется: Задержка

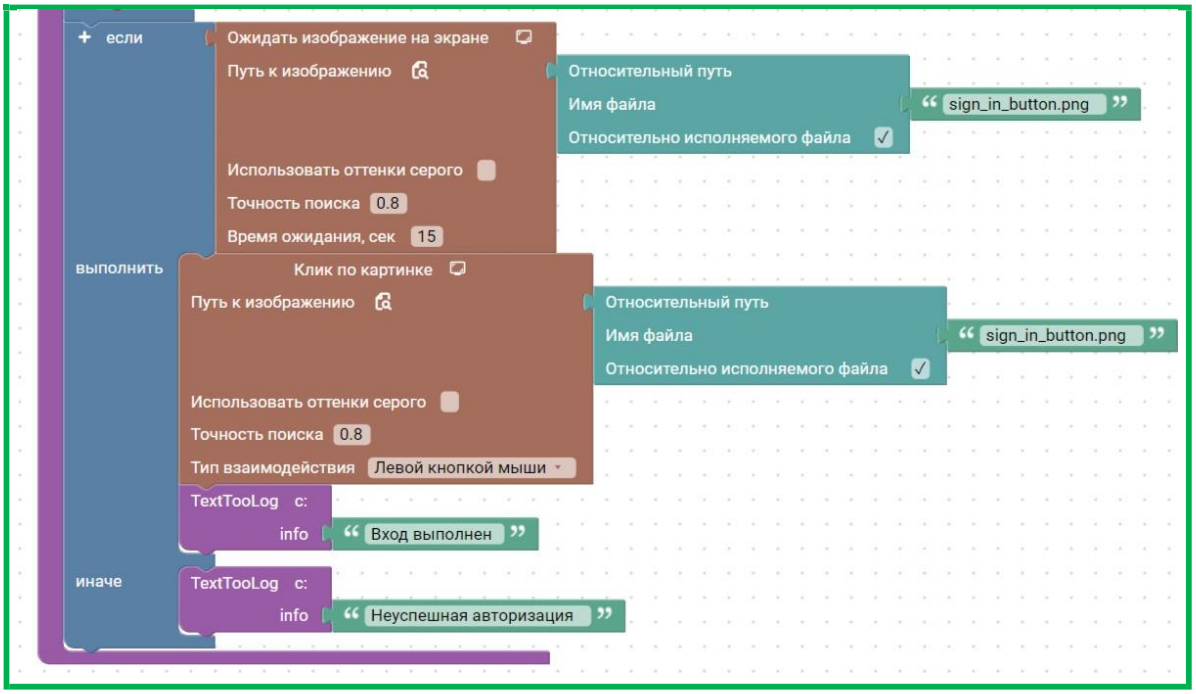
Что делаем в этом блоке: Указываем время задержки 2 секунды. Это нужно, чтобы поле ввода пароля стало активным.

Шаг 7. Ввод пароля в браузере

Какой блок используется: Ввод в веб-сайт

Что делаем в этом блоке: Указываем тип элемента (активный элемент на странице). Передаём текст для ввода из параметра password.

Что возвращает блок: Ничего не возвращает. Вводит указанный текст в активное поле ввода.



Шаг 8. Ожидание появления кнопки входа

Какой блок используется: Ожидать изображение на экране

Что делаем в этом блоке: Указываем путь к изображению-эталону кнопки входа sign_in_button.png. Указываем точность совпадения 0.8 (80 процентов). Указываем таймаут 15 секунд – это максимальное время ожидания.

Что возвращает блок: Возвращает значение True, если изображение найдено в течение таймаута. Возвращает False, если изображение не появилось за 15 се- кунд.

Шаг 9. Проверка результата ожидания (условие)

Какой блок используется: Конструкция Если – выполнить-иначе

Что делаем в этом блоке: Проверяем, что вернул блок ожидания. Если вернул True, значит кнопка входа появилась и можно нажимать. Если вернул False, значит кнопка не появилась.

Шаг 10. Действие при успешном появлении кнопки (ветка Если)

Какой блок используется: Клик по картинке

Что делаем в этом блоке: Указываем путь к изображению-эталону кнопки входа sign_in_button.png. Указываем точность совпадения 0.8. Указываем задержку 0 секунд.

Что возвращает блок: Ничего не возвращает. Нажимает на кнопку входа.

Шаг 11. Запись об успешном входе (ветка Если) Какой блок используется: TextTooLog (наша собственная функция)

Что делаем в этом блоке: Вызываем функцию TextTooLog и передаём ей сообщение Вход выполнен. Это сообщение запишется в файл лога с указанием текущего времени.

Шаг 12. Действие при ошибке (ветка Иначе)

Какой блок используется: TextTooLog (наша собственная функция)

Что делаем в этом блоке: Вызываем функцию TextTooLog и передаём ей сообщение Неуспешная авторизация. Это сообщение запишется в файл лога с указанием текущего времени.

Важные замечания:

Первое. Изображения login_field.png, password_field.png и sign_in_button.png должны быть подготовлены заранее. Нужно открыть страницу вручную, сделать скриншоты этих элементов и сохранить их в папку resource.

Второе. Точность 0.8 означает 80-процентное совпадение. Если робот не находит изображение, точность можно уменьшить. Если находит не то, что нужно – увеличить.

Третье. Таймаут 15 секунд выбран с учётом того, что кнопка может появляться не мгновенно после ввода данных. Если страница обрабатывает ввод дольше, таймаут можно увеличить.

Четвёртое. Задержки по 2 секунды между действиями нужны для стабильной работы. Если сайт работает быстро, задержки можно уменьшить. Если медленно – увеличить.

Пятое. Параметр «активный элемент» означает, что текст будет введён в элемент, который в данный момент активен на странице (находится в фокусе).

Функция read_excel

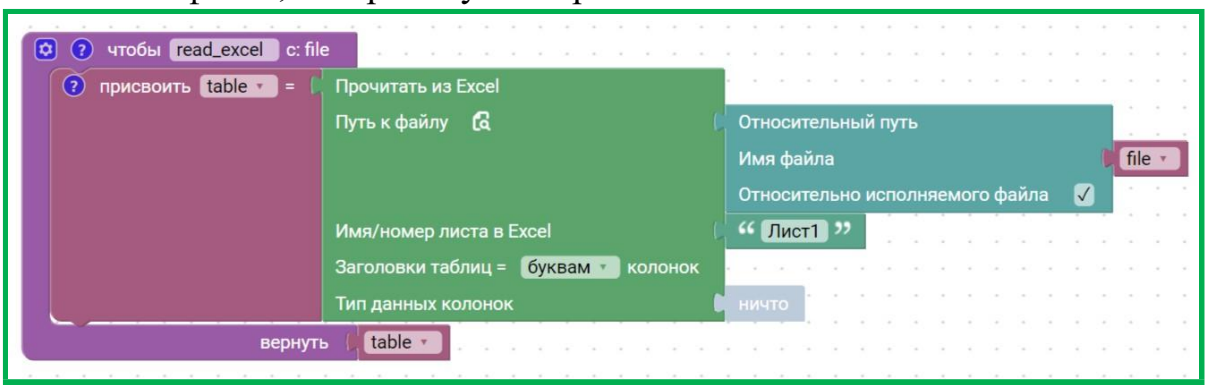
Функция read_excel предназначена для чтения данных из Excel-файла, который содержит информацию о сотрудниках и приоритетах заявок.

Что делает функция:

Первое – получает полный путь к Excel-файлу по его имени. Второе – читает данные с первого листа файла. Третье – возвращает таблицу с данными в основную программу для дальнейшей обработки.

Почему это важно: Входные данные для робота хранятся в Excel-файле. Функция читает этот файл и преобразует его в формат, с которым удобно рабо- тать внутри Puzzle RPA. Робот получает список сотрудников и соответствующих им приоритетов, чтобы затем заполнять заявки.

Функция принимает один параметр – file. Это строковое значение, содержащее имя Excel-файла, который нужно прочитать.



Шаг 1. Получение полного пути к файлу

Какой блок используется: Относительный путь

Что делаем в этом блоке: Передаём в блок имя файла, которое пришло в параметре file. Блок автоматически определяет папку, в которой находится исполняемый файл робота, и добавляет к ней имя файла.

Что возвращает блок: Возвращает полный путь к файлу. Почему это важно: Робот не знает, где лежит файл, если ему просто передать имя. Блок Относительный путь решает эту проблему – он всегда находит файл в папке с роботом, даже если робота перенести на другой компьютер.

Шаг 2. Чтение данных из Excel

Какой блок используется: Прочитать из Excel

Что делаем в этом блоке: Передаём в блок полный путь к файлу, полученный на предыдущем шаге. Указываем имя листа, с которого нужно читать данные – 'Лист1'. Указываем параметр True, который означает, что первая строка файла содержит заголовки столбцов.

Что возвращает блок: Возвращает таблицу данных. К этой таблице можно обращаться по номерам строк и столбцов или по именам колонок.

Шаг 3. Возврат результата

Что делаем: Возвращаем таблицу table в основную программу. Теперь вызывающий код может использовать эту таблицу для получения данных о сотрудниках и приоритетах.

Функция new_request

Функция new_request предназначена для массового заполнения заявок в веб-приложении на основе данных из Excel-таблицы.

Что делает функция:

Первое – перебирает все строки таблицы с данными, начиная со второй строки (первая строка содержит заголовки).

Второе – извлекает из каждой строки значения: имя участника, email, название мероприятия и формат участия.

Третье – проверяет, что окно для заполнения заявки доступно.

Четвёртое – заполняет поля: имя, почта, мероприятие.

Пятое – выбирает формат участия (Online или Offline) в зависимости от данных в таблице.

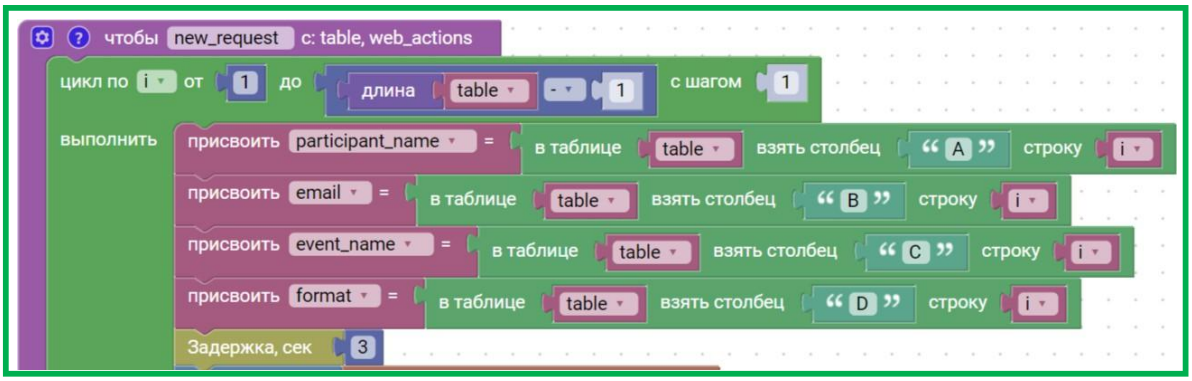
Шестое – принимает условия и нажимает кнопку отправки заявки.

Седьмое – проверяет, успешно ли отправилась заявка, и записывает результат в лог.

Функция принимает два параметра:

table – таблица с данными из Excel, полученная из функции read_excel.

web_actions – объект для управления браузером, полученный из функции open_browser.



Шаг 1. Определение количества строк для перебора

Какой блок используется: Присвоить значение переменной

Что делаем в этом блоке: Вычисляем количество строк в таблице с помощью len(table) и вычитаем 1, чтобы исключить строку с заголовками.

Что возвращает блок: Возвращает число – количество строк данных в таблице.

Шаг 2. Цикл перебора строк

Какой блок используется: Цикл по (от до с шагом)

Что делаем в этом блоке: Запускаем цикл от 1 до i_end с шагом 1. Переменная i принимает значение текущего номера строки.

Шаг 3. Получение имени участника

Какой блок используется: Получить значение ячейки табличных данных

Что делаем в этом блоке: Передаём таблицу table, указываем колонку 'A' и индекс строки i. Сохраняем результат в переменную participant_name.

Что возвращает блок: Возвращает значение из ячейки таблицы.

Шаг 4. Получение email

Какой блок используется: Получить значение ячейки табличных данных

Что делаем в этом блоке: Передаём таблицу table, указываем колонку 'B' и индекс строки i. Сохраняем результат в переменную email.

Что возвращает блок: Возвращает значение из ячейки таблицы.

Шаг 5. Получение названия мероприятия

Какой блок используется: Получить значение ячейки табличных данных

Что делаем в этом блоке: Передаём таблицу table, указываем колонку 'C' и индекс строки i. Сохраняем результат в переменную event_name.

Что возвращает блок: Возвращает значение из ячейки таблицы.

Шаг 6. Получение формата участия

Какой блок используется: Получить значение ячейки табличных данных

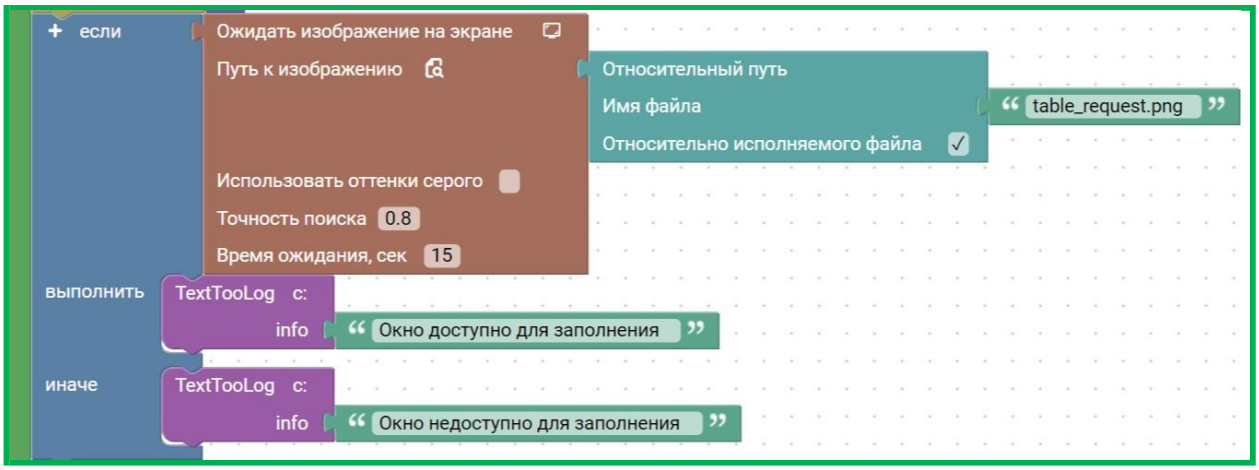
Что делаем в этом блоке: Передаём таблицу table, указываем колонку 'D' и индекс строки i. Сохраняем результат в переменную format.

Что возвращает блок: Возвращает значение из ячейки таблицы.

Шаг 7. Задержка перед проверкой окна

Какой блок используется: Задержка

Что делаем в этом блоке: Указываем время задержки 3 секунды. Это нужно, чтобы страница подготовилась к заполнению.



Шаг 8. Проверка доступности окна для заполнения

Какой блок используется: Ожидать изображение на экране

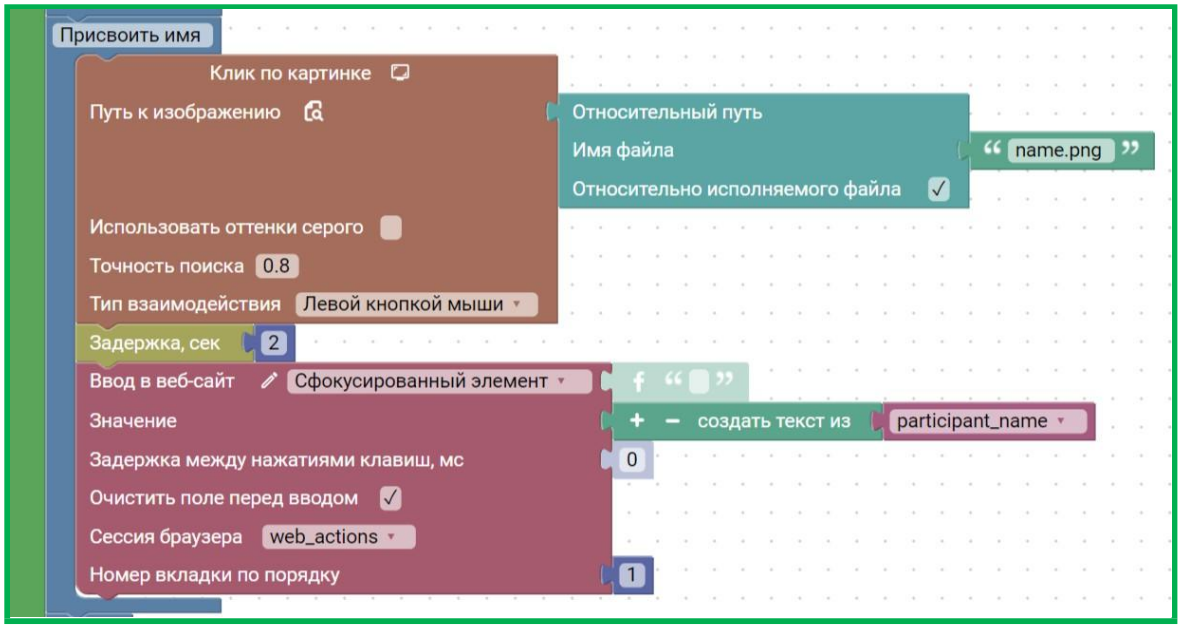
Что делаем в этом блоке: Указываем путь к изображению-эталону таблицы заявок table_request.png. Указываем точность 0.8 и таймаут 15 секунд.

Что возвращает блок: Возвращает True, если окно доступно. Возвращает False, если окно недоступно.

Шаг 9. Запись результата проверки окна

Какой блок используется: TextTooLog

Что делаем в этом блоке: Если окно доступно – записываем сообщение Окно доступно для заполнения. Если недоступно – записываем Окно недоступно для заполнения.



Шаг 10. Клик по полю имени

Какой блок используется: Клик по картинке

Что делаем в этом блоке: Указываем путь к изображению поля для ввода имени name.png. Точность 0.8, задержка 0.

Шаг 11. Задержка после клика

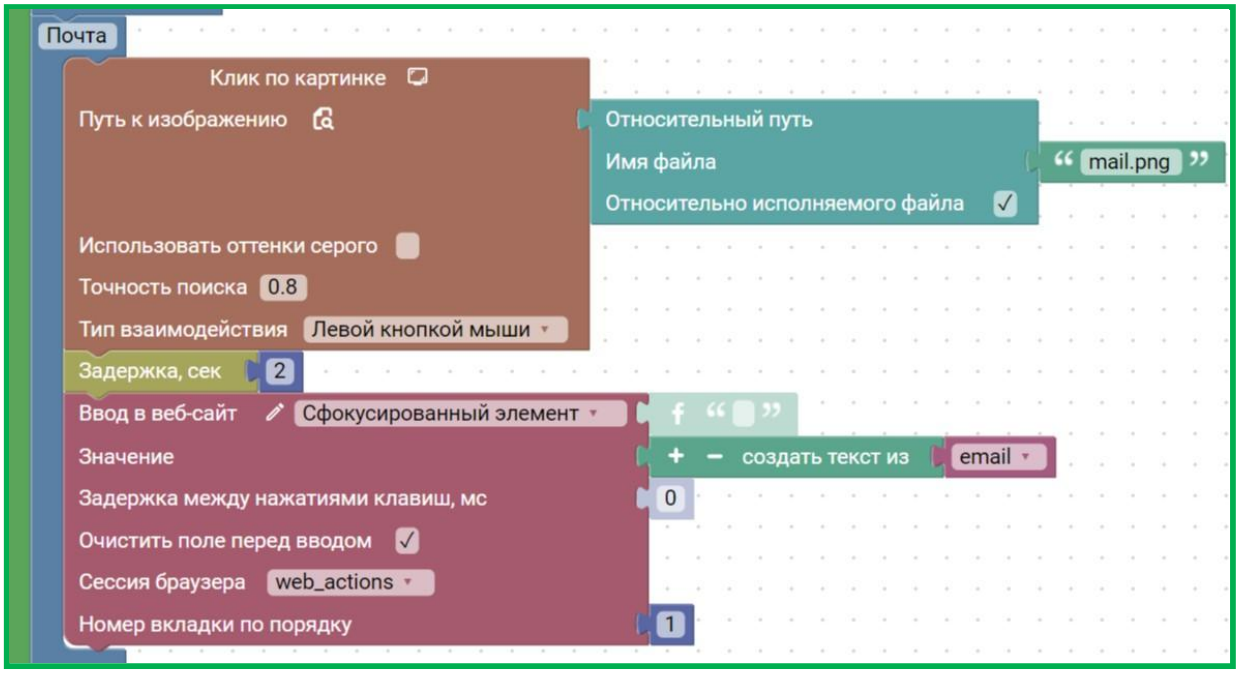
Какой блок используется: Задержка

Что делаем в этом блоке: Задержка 2 секунды для активации поля.

Шаг 12. Ввод имени

Какой блок используется: Ввод в веб-сайт

Что делаем в этом блоке: Указываем тип active_element. Передаём текст из переменной participant_name, преобразованный в строку.



Шаг 13. Клик по полю почты

Какой блок используется: Клик по картинке

Что делаем в этом блоке: Указываем путь к изображению поля для ввода почты mail.png. Точность 0.8.

Шаг 14. Задержка после клика

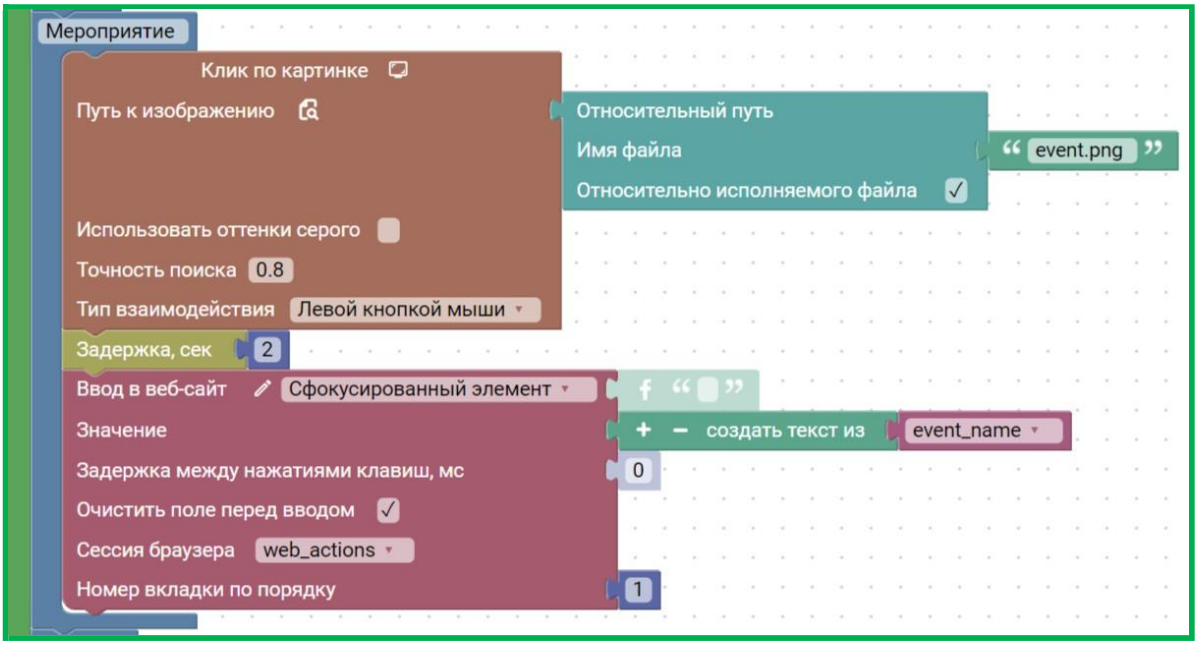
Какой блок используется: Задержка

Что делаем в этом блоке: Задержка 2 секунды.

Шаг 15. Ввод почты

Какой блок используется: Ввод в веб-сайт

Что делаем в этом блоке: Передаём текст из переменной email, преобразованный в строку.



Шаг 16. Клик по полю мероприятия

Какой блок используется: Клик по картинке

Что делаем в этом блоке: Указываем путь к изображению поля для ввода мероприятия event.png.

Шаг 17. Задержка после клика

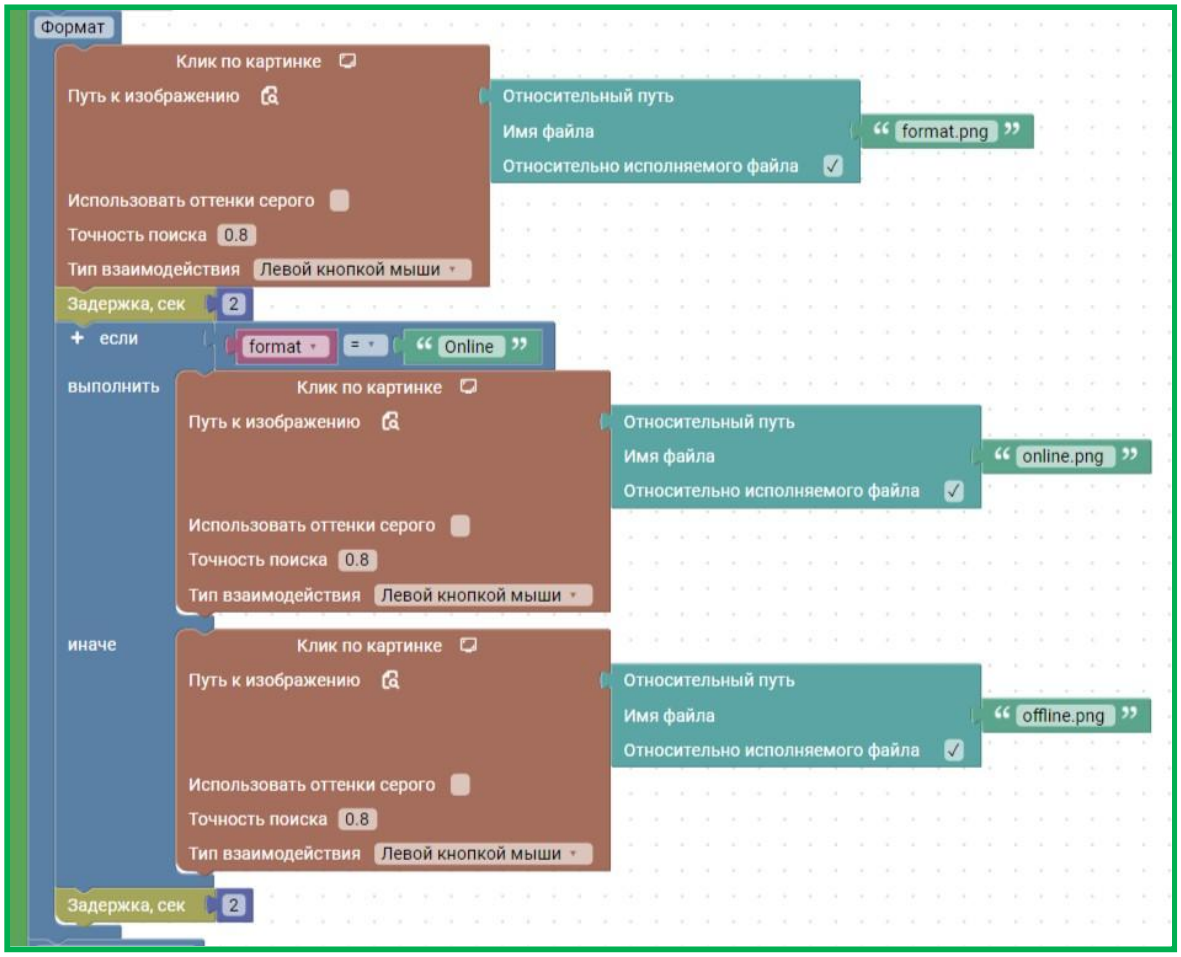
Какой блок используется: Задержка

Что делаем в этом блоке: Задержка 2 секунды.

Шаг 18. Ввод названия мероприятия

Какой блок используется: Ввод в веб-сайт

Что делаем в этом блоке: Передаём текст из переменной event_name, преобразованный в строку.



Шаг 19. Клик по полю формат

Какой блок используется: Клик по картинке

Что делаем в этом блоке: Указываем путь к изображению поля для выбора формата format.png.

Шаг 20. Задержка после клика

Какой блок используется: Задержка

Что делаем в этом блоке: Задержка 2 секунды.

Шаг 21. Проверка формата участия

Какой блок используется: Конструкция Если – выполнить-иначе

Что делаем в этом блоке: Проверяем значение переменной format. Если значение равно 'Online', выбираем онлайн-формат. Если нет – выбираем офлайн-формат.

Шаг 22. Выбор формата Online

Какой блок используется: Клик по картинке

Что делаем в этом блоке: Указываем путь к изображению online.png.

Шаг 23. Выбор формата Offline

Какой блок используется: Клик по картинке

Что делаем в этом блоке: Указываем путь к изображению offline.png.

Шаг 24. Задержка после выбора формата

Какой блок используется: Задержка

Что делаем в этом блоке: Задержка 2 секунды.



Шаг 25. Задержка перед принятием условий

Что делаем в этом блоке: Задержка 2 секунды.

Шаг 26. Клик по кнопке принятия условий

Какой блок используется: Клик по картинке

Что делаем в этом блоке: Указываем путь к изображению кнопки button_yes.png.



Шаг 27. Проверка активности кнопки отправки

Какой блок используется: Ожидать изображение на экране

Что делаем в этом блоке: Указываем путь к изображению кнопки отправки submit_button.png. Таймаут 15 секунд.

Шаг 28. Нажатие кнопки отправки

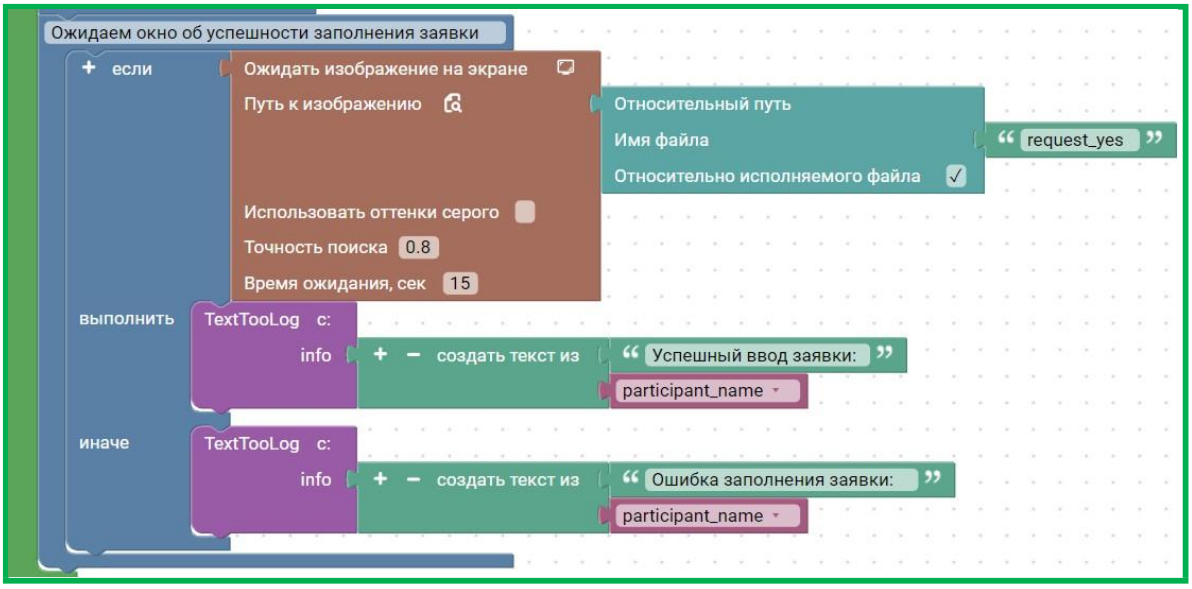
Какой блок используется: Клик по картинке

Что делаем в этом блоке: Если кнопка активна, нажимаем на неё.

Шаг 29. Запись об ошибке при неактивной кнопке

Какой блок используется: TextTooLog

Что делаем в этом блоке: Если кнопка не активна, записываем сообщение Кнопка не активна. Заполнены не все поля.



Шаг 30. Ожидание результата отправки заявки

Какой блок используется: Ожидать изображение на экране

Что делаем в этом блоке: Снова проверяем появление кнопки отправки. Если кнопка исчезла и появилась снова – значит заявка обработана.

Шаг 31. Запись об успешной отправке

Какой блок используется: TextTooLog

Что делаем в этом блоке: Записываем сообщение Успешный ввод заявки: с указанием имени участника.

Шаг 32. Запись об ошибке отправки

Какой блок используется: TextTooLog

Что делаем в этом блоке: Записываем сообщение Ошибка заполнения заявки: с указанием имени участника.

Важные замечания:

Первое. Таблица должна иметь колонки: A – имя участника, B – email, C – название мероприятия, D – формат участия (Online/Offline).

Второе. Первая строка таблицы считается заголовком и не обрабатывается.

Третье. Изображения name.png, mail.png, event.png, format.png, online.png, offline.png, button_yes.png, submit_button.png должны быть подготовлены заранее.

Четвёртое. Задержки между действиями нужны для стабильной работы веб-приложения.

Функция process_applications

Функция process_applications предназначена для автоматической обработки заявок в десктопном приложении: прокрутки списка, выделения заявок онлайн и офлайн формата, простановки отметок "одобрено" или "отклонено".

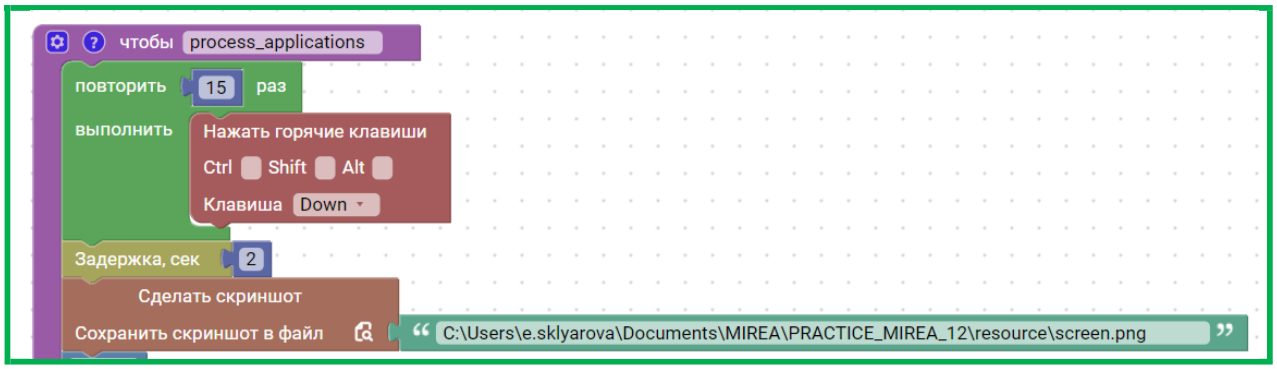
Что делает функция:

Первое – делает скриншот экрана для фиксации начального состояния. Второе – находит все заявки с онлайн-форматом на экране, перебирает их, выделяет каждую и ставит отметку "одобрено".

Третье – находит все заявки с офлайн-форматом на экране, перебирает их, выделяет каждую и ставит отметку "отклонено".

Четвёртое – возвращает количество обработанных онлайн и офлайн заявок.

Функция не принимает параметров. Возвращает словарь с двумя ключами: online – количество обработанных онлайн заявок, offline – количество обработанных офлайн заявок.



Шаг 1. Прокрутка страницы вверх

Какой блок используется: Цикл повторить n-раз

Что делаем в этом блоке: Запускаем цикл на 15 повторений. Внутри цикла нажимаем горячую клавишу (клавиша "Вниз").

Шаг 2. Задержка после прокрутки

Какой блок используется: Задержка

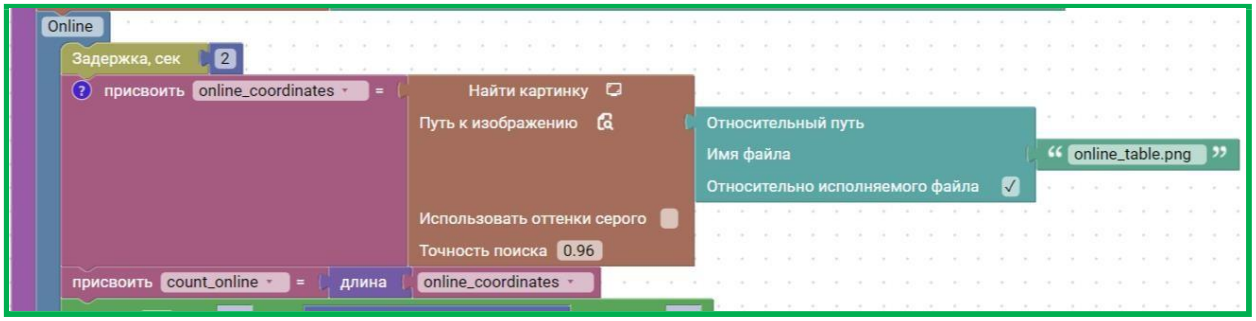
Что делаем в этом блоке: Задержка 2 секунды для стабилизации отображения.

Шаг 3. Создание скриншота

Какой блок используется: Сделать скриншот

Что делаем в этом блоке: Указываем путь для сохранения скриншота C:\Users\e.sklyarova\Documents\MIREA\PRACTICE_MIREA_12\resource\screen.png

Что возвращает блок: Ничего не возвращает. Сохраняет скриншот экрана по указанному пути.



Шаг 4. Задержка перед обработкой онлайн

Какой блок используется: Задержка

Что делаем в этом блоке: Задержка 2 секунды.

Шаг 5. Поиск онлайн заявок

Какой блок используется: Найти картинку

Что делаем в этом блоке: Указываем путь к изображению-эталону online_table.png через блок "Относительный путь". Указываем точность 0.96 (96 процентов).

Что возвращает блок: Возвращает массив координат всех найденных изображений.

Шаг 6. Подсчёт количества онлайн заявок

Какой блок используется: Присвоить значение переменной

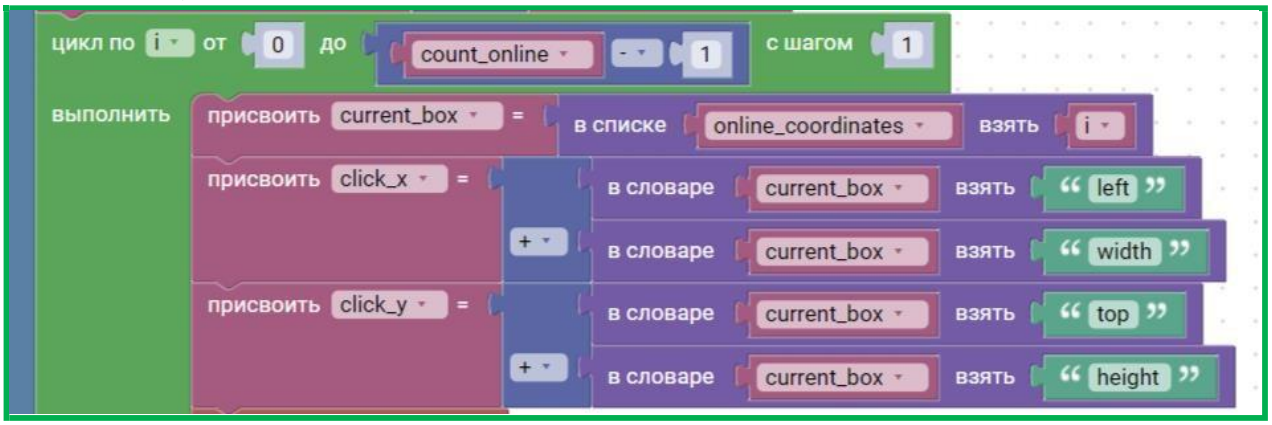
Что делаем в этом блоке: Вычисляем длину массива online_coordinates с помощью len() и сохраняем в переменную count_online.

Что возвращает блок: Возвращает количество найденных онлайн заявок.

Шаг 7. Определение количества итераций для цикла

Какой блок используется: Присвоить значение переменной

Что делаем в этом блоке: Вычисляем i_end = count_online – 1 (последний индекс массива).



Шаг 8. Цикл перебора онлайн заявок

Какой блок используется: Цикл по (от до с шагом)

Что делаем в этом блоке: Запускаем цикл от 0 до i_end с шагом 1. Переменная i принимает значение текущего индекса.

Шаг 9. Получение текущей заявки

Какой блок используется: Присвоить значение переменной

Что делаем в этом блоке: Извлекаем из массива online_coordinates элемент с индексом i и сохраняем в переменную current_box.

Шаг 10. Вычисление координаты X для клика

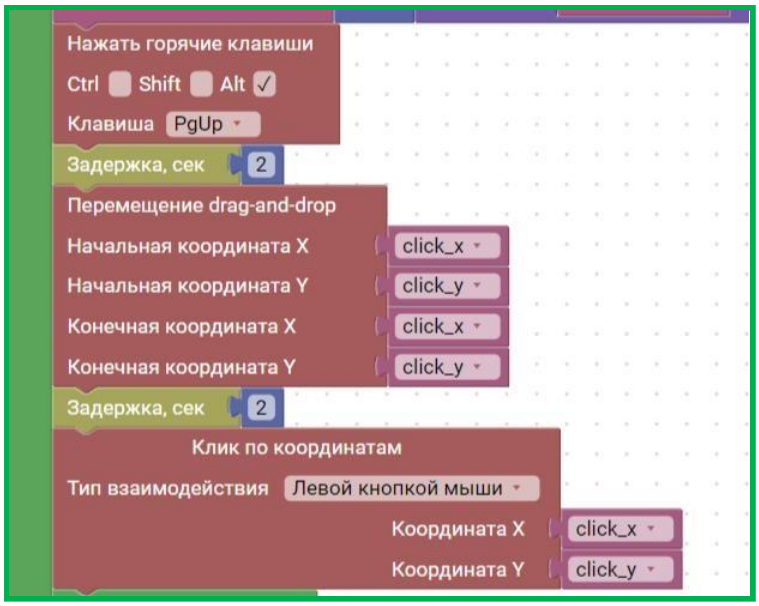
Какой блок используется: Присвоить значение переменной

Что делаем в этом блоке: Вычисляем click_x = (current_box['left']) + (current_box['width']). Координата X = левая граница + ширина изображения (правый край).

Шаг 11. Вычисление координаты Y для клика

Какой блок используется: Присвоить значение переменной

Что делаем в этом блоке: Вычисляем click_y = (current_box['top']) + (current_box['height']). Координата Y = верхняя граница + высота изображения (нижний край).



Шаг 12. Нажатие клавиши для выделения

Какой блок используется: Нажать горячие клавиши

Что делаем в этом блоке: Нажимаем горячую клавишу (клавиша "Page Down").

Шаг 13. Задержка после нажатия

Какой блок используется: Задержка

Что делаем в этом блоке: Задержка 2 секунды.

Шаг 14. Перемещение курсора (drag-and-drop)

Какой блок используется: Перемещение drag-and-drop

Что делаем в этом блоке: Перемещаем курсор из точки (click_x, click_y) в ту же самую точку. Фактически это просто установка курсора в нужное место.

Шаг 15. Задержка после перемещения

Какой блок используется: Задержка

Что делаем в этом блоке: Задержка 2 секунды.

Шаг 16. Клик по координатам

Какой блок используется: Клик по координатам

Что делаем в этом блоке: Выполняем клик левой кнопкой мыши (первый параметр 0) по координатам (click_x, click_y).



Шаг 17. Прокрутка вниз после выделения

Какой блок используется: Цикл повторить n-раз

Что делаем в этом блоке: Запускаем цикл на 15 повторений. Внутри цикла нажимаем горячую клавишу (Page Down).

Шаг 18. Задержка после прокрутки

Какой блок используется: Задержка

Что делаем в этом блоке: Задержка 2 секунды.



Шаг 19. Клик по кнопке "одобрено"

Какой блок используется: Клик по картинке

Что делаем в этом блоке: Находим изображение mark_approved.png через относительный путь и кликаем по нему. Точность 0.8.

Шаг 20. Задержка после клика

Какой блок используется: Задержка

Что делаем в этом блоке: Задержка 2 секунды.



Шаг 21. Прокрутка вверх для следующей заявки

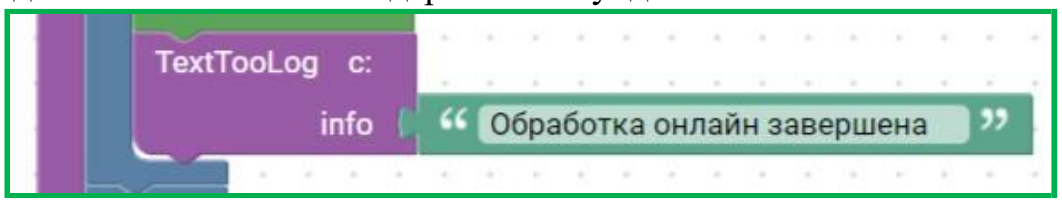
Какой блок используется: Цикл повторить n-раз

Что делаем в этом блоке: Запускаем цикл на 15 повторений. Внутри цикла нажимаем горячую клавишу (клавиша "Вверх").

Шаг 22. Задержка после прокрутки

Какой блок используется: Задержка

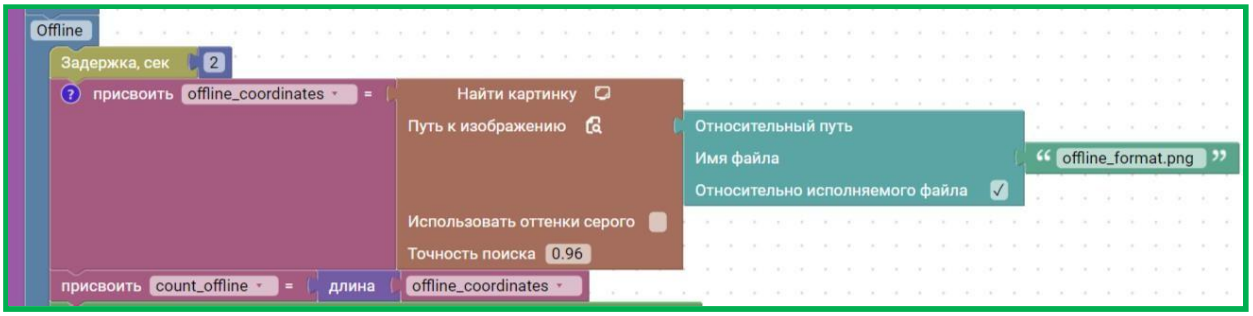
Что делаем в этом блоке: Задержка 2 секунды.



Шаг 23. Запись в лог о завершении обработки онлайн

Какой блок используется: TextTooLog

Что делаем в этом блоке: Записываем сообщение Обработка онлайн завершена.


Offline
Задержка, сек
2
присвоить
offline_coordinates
=
Найти картинку
Путь к изображению
Относительный путь
Имя файла
offline_format.png
Относительно исполняемого файла
Использовать оттенки серого
Точность поиска
0.96
присвоить
count_offline
=
длина
offline_coordinates

Шаг 24. Задержка перед обработкой офлайн

Какой блок используется: Задержка

Что делаем в этом блоке: Задержка 2 секунды.

Шаг 25. Поиск офлайн заявок

Какой блок используется: Найти картинку

Что делаем в этом блоке: Указываем путь к изображению-эталону offline_format.png через относительный путь. Точность 0.96.

Что возвращает блок: Возвращает массив координат всех найденных изображений.

Шаг 26. Подсчёт количества офлайн заявок

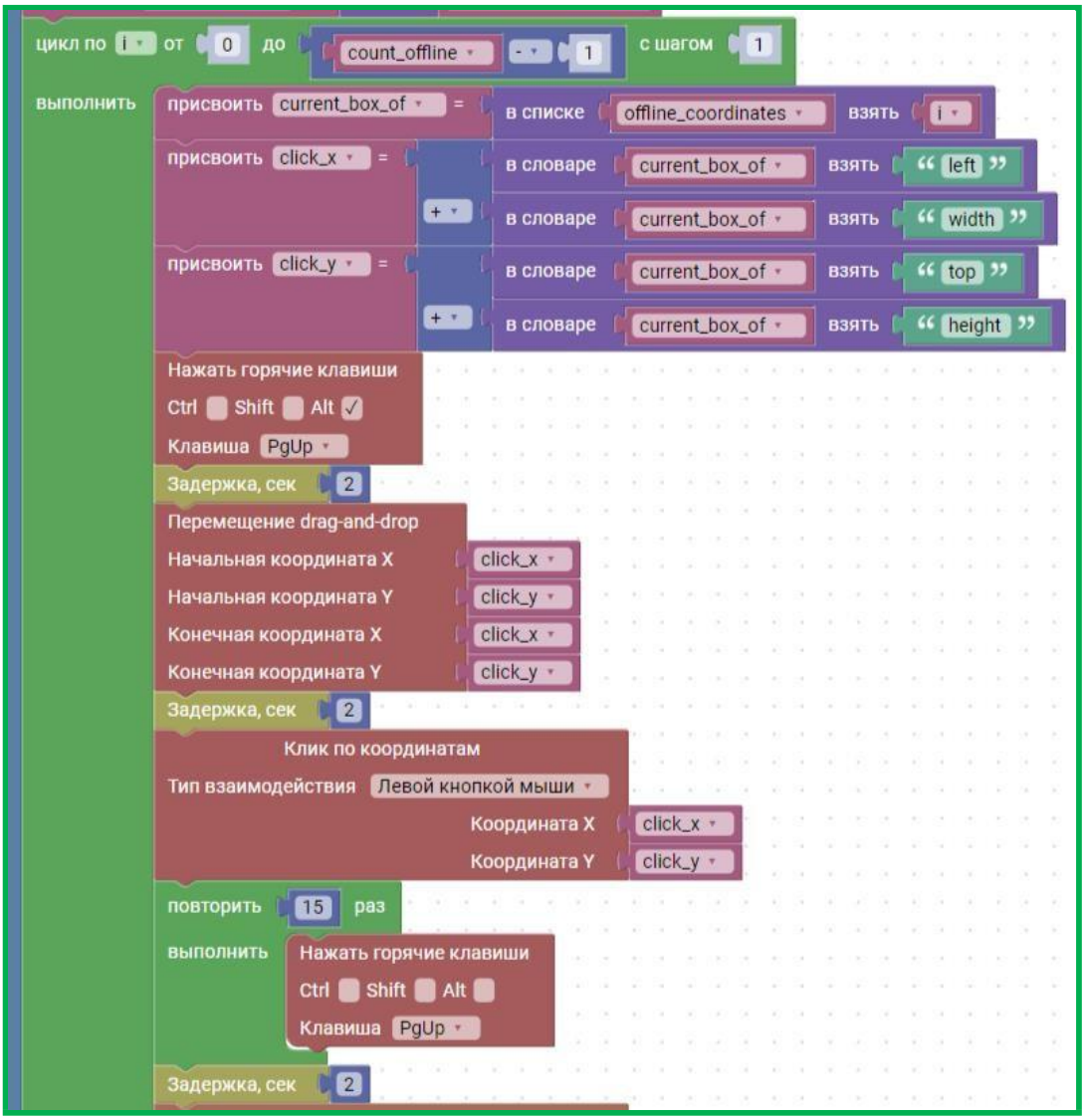
Какой блок используется: Присвоить значение переменной

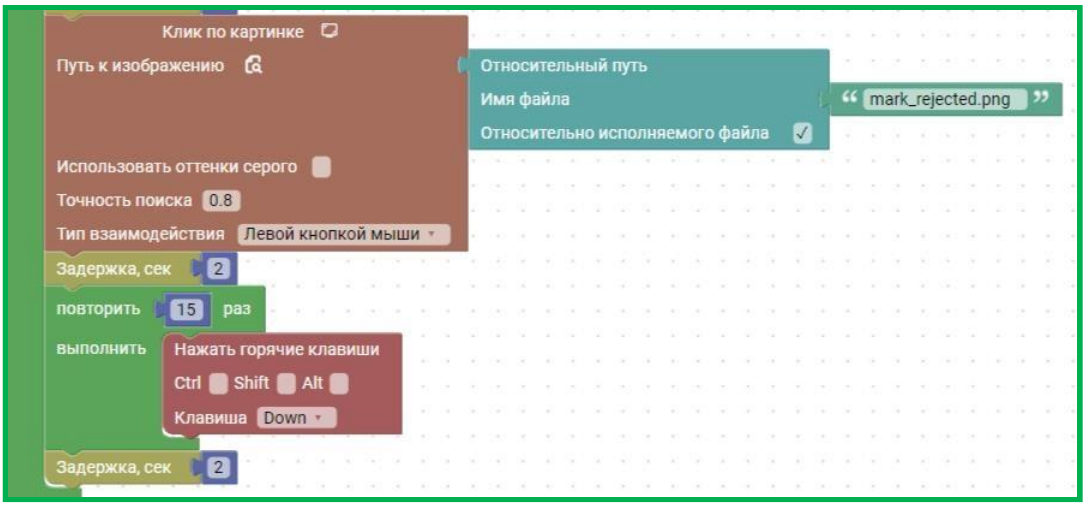
Что делаем в этом блоке: Вычисляем длину массива offline_coordinates и сохраняем в переменную count_offline.

Шаг 27. Определение количества итераций для цикла

Какой блок используется: Присвоить значение переменной

Что делаем в этом блоке: Вычисляем i_end = count_offline – 1.





Шаг 28. Цикл перебора офлайн заявок

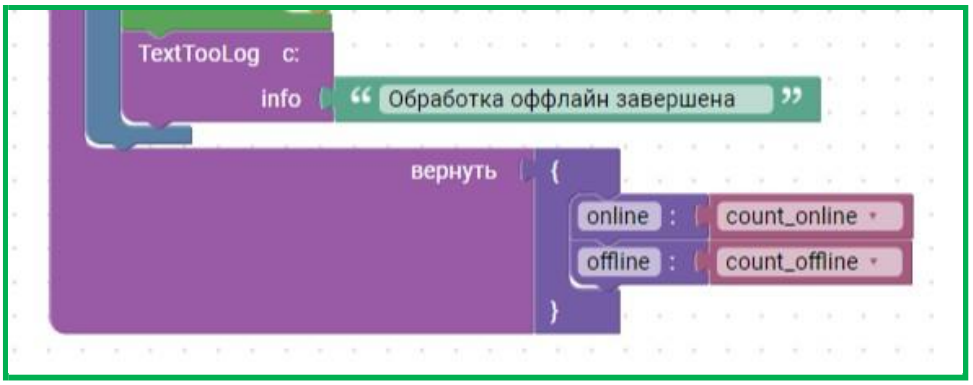
Какой блок используется: Цикл по (от до с шагом)

Что делаем в этом блоке: Запускаем цикл от 0 до i_end с шагом 1.

Шаг 29-40. Обработка офлайн заявок

Что делаем в этом блоке: Повторяем те же действия, что и для онлайн заявок, но с двумя отличиями:

Вместо mark_approved.png используем mark_rejected.png (отклонено)



Шаг 41. Запись в лог о завершении обработки офлайн

Какой блок используется: TextTooLog

Что делаем в этом блоке: Записываем сообщение Обработка оффлайн завершена.

Шаг 42. Возврат результата

Какой блок используется: Возврат значения

Что делаем в этом блоке: Возвращаем словарь с ключами online и offline, содержащими количество обработанных заявок.

Что возвращает блок: Возвращает словарь: {'online': count_online, 'offline': count_offline}.

Важные замечания:

Первое. Изображения online_table.png, offline_format.png, mark_approved.png, mark_rejected.png должны быть подготовлены заранее.

Второе. Точность 0.96 для поиска изображений используется для более точного совпадения.

Третье. Функция обрабатывает только те заявки, которые видны на экране. Для обработки всех заявок может потребоваться дополнительная прокрутка.

Функция delete_button

Функция delete_button предназначена для удаления записи в десктопном приложении: нажатия кнопки удаления, подверждения удаления и проверки результата.

Что делает функция:

Первое – инициализирует счётчик удалённых записей значением 0.

Второе – нажимает на кнопку удаления.

Третье – прокручивает страницу вниз для появления кнопки подтверждения.

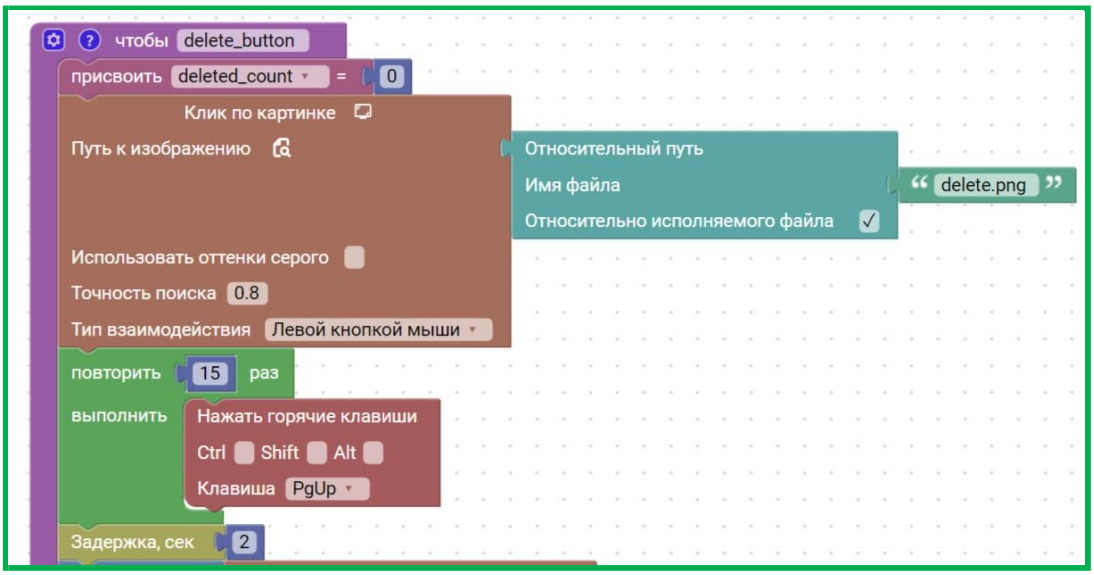
Четвёртое – ожидает появления кнопки подтверждения удаления.

Пятое – если кнопка подтверждения появилась, нажимает на неё, увеличивает счётчик на 1 и записывает в лог сообщение об успехе.

Шестое – если кнопка подтверждения не появилась, записывает в лог сообщение об ошибке.

Седьмое – возвращает количество удалённых записей.

Функция не принимает параметров. Возвращает целое число – количество удалённых записей (0 или 1).



Шаг 1. Инициализация счётчика удалений

Какой блок используется: Присвоить значение переменной

Что делаем в этом блоке: Создаём переменную deleted_count и присваиваем ей значение 0. Эта переменная будет хранить количество успешно удалённых записей.

Что возвращает блок: Ничего не возвращает. Создаёт переменную с начальным значением 0.

Шаг 2. Нажатие кнопки удаления

Какой блок используется: Клик по картинке

Что делаем в этом блоке: Указываем путь к изображению-эталону кнопки удаления delete.png через блок "Относительный путь". Указываем точность совпадения 0.8 (80 процентов). Задержка 0 секунд.

Что возвращает блок: Ничего не возвращает. Кликает по кнопке удаления.

Шаг 3. Прокрутка страницы вниз

Какой блок используется: Цикл повторить n-раз

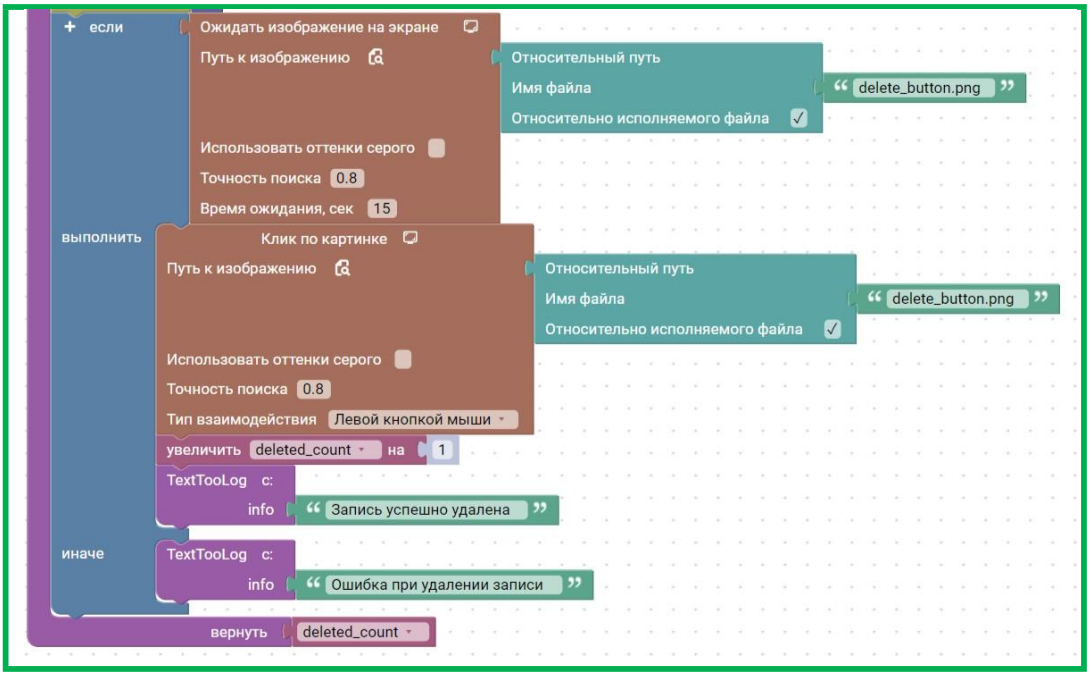
Что делаем в этом блоке: Запускаем цикл на 15 повторений. Внутри цикла нажимаем горячую клавишу (клавиша "Page Down").

Что возвращает блок: Ничего не возвращает. Прокручивает страницу вниз 15 раз, чтобы появилась кнопка подтверждения удаления.

Шаг 4. Задержка перед поиском кнопки

Какой блок используется: Задержка

Что делаем в этом блоке: Задержка 2 секунды. Это нужно, чтобы страница завершила прокрутку и кнопка подтверждения стала видимой.



Шаг 5. Ожидание появления кнопки подтверждения

Какой блок используется: Ожидать изображение на экране

Что делаем в этом блоке: Указываем путь к изображению-эталону кнопки подтверждения delete_button.png через блок "Относительный путь". Указываем точность совпадения 0.8 (80 процентов). Указываем таймаут 15 секунд – это максимальное время ожидания.

Что возвращает блок: Возвращает значение True, если изображение найдено в течение таймаута. Возвращает False, если изображение не появилось за 15 се- кунд.

Шаг 6. Проверка результата ожидания (условие)

Какой блок используется: Конструкция Если – выполнить-иначе

Что делаем в этом блоке: Проверяем, что вернул блок ожидания. Если вернул True, значит кнопка подтверждения появилась. Если вернул False, значит кнопка не появилась.

Шаг 7. Нажатие кнопки подтверждения (ветка Если)

Какой блок используется: Клик по картинке

Что делаем в этом блоке: Указываем путь к изображению-эталону кнопки подтверждения delete_button.png через блок "Относительный путь". Точность 0.8, задержка 0.

Что возвращает блок: Ничего не возвращает. Нажимает на кнопку подтверждения удаления.

Шаг 8. Увеличение счётчика удалений (ветка Если)

Какой блок используется: Увеличить значение переменной

Что делаем в этом блоке: Увеличиваем значение переменной deleted_count на 1.

Что возвращает блок: Возвращает новое значение переменной deleted_count.

Шаг 9. Запись об успешном удалении (ветка Если)

Какой блок используется: TextTooLog (наша собственная функция)

Что делаем в этом блоке: Вызываем функцию TextTooLog и передаём ей сообщение Запись успешно удалена. Это сообщение запишется в файл лога с указанием текущего времени.

Шаг 10. Запись об ошибке удаления (ветка Иначе) Какой блок используется: TextTooLog (наша собственная функция)

Что делаем в этом блоке: Вызываем функцию TextTooLog и передаём ей сообщение Ошибка при удалении записи. Это сообщение запишется в файл лога с указанием текущего времени.

Шаг 11. Возврат количества удалённых записей

Какой блок используется: Возврат значения

Что делаем в этом блоке: Возвращаем значение переменной deleted_count. Что возвращает блок: Возвращает 1, если удаление прошло успешно. Возвращает 0, если удаление не удалось.

Важные замечания:

Первое. Изображения delete.png (кнопка удаления) и delete_button.png (кнопка подтверждения) должны быть подготовлены заранее.

Второе. Точность 0.8 означает 80-процентное совпадение. Если робот не находит изображение, точность можно уменьшить.

Третье. Таймаут 15 секунд выбран с учётом того, что кнопка подтверждения может появляться не мгновенно.

Четвёртое. Прокрутка на 15 нажатий Page Down нужна, если кнопка подтверждения находится ниже по странице.

Пятое. Функция возвращает 1 при успешном удалении и 0 при ошибке. Это можно использовать для подсчёта общего количества удалённых записей.

Функция generate_fromGPT

Функция generate_fromGPT предназначена для формирования отчёта о работе портала с помощью искусственного интеллекта (GPT) и отображения его пользователю.

Что делает функция:

Первое – вычисляет общее количество заявок (сумма онлайн и офлайн заявок).

Второе – присваивает переменным значения: количество одобренных (онлайн), количество отклонённых (офлайн) и количество удалённых записей.

Третье – формирует текстовый запрос к GPT с данными статистики.

Четвёртое – отправляет запрос к GPT-серверу и получает сгенерированный отчёт.

Пятое – показывает полученный отчёт пользователю в виде уведомления.

Шестое – возвращает сгенерированный текст отчёта.

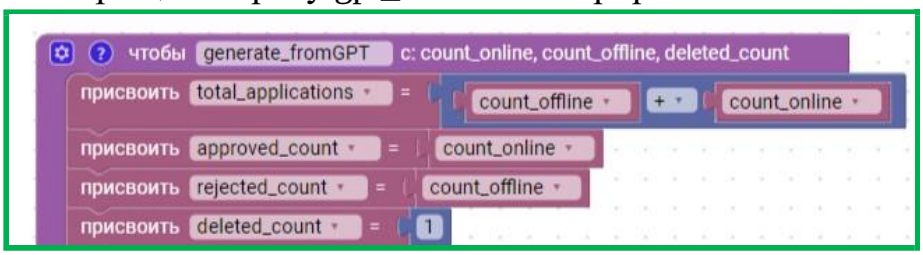
Функция принимает три параметра:

count_online – количество одобренных (онлайн) заявок.

count_offline – количество отклонённых (офлайн) заявок.

deleted_count – количество удалённых записей.

Функция возвращает строку gpt_titles – сгенерированный GPT текст отчёта.



Шаг 1. Вычисление общего количества заявок

Какой блок используется: Присвоить значение переменной

Что делаем в этом блоке: Складываем значения count_offline и count_online и сохраняем результат в переменную total_applications.

Что возвращает блок: Возвращает сумму двух чисел.

Шаг 2. Присвоение значения одобренным заявкам

Какой блок используется: Присвоить значение переменной

Что делаем в этом блоке: Присваиваем переменной approved_count значение переменной count_online.

Что возвращает блок: Возвращает количество одобренных заявок.

Шаг 3. Присвоение значения отклонённым заявкам

Какой блок используется: Присвоить значение переменной

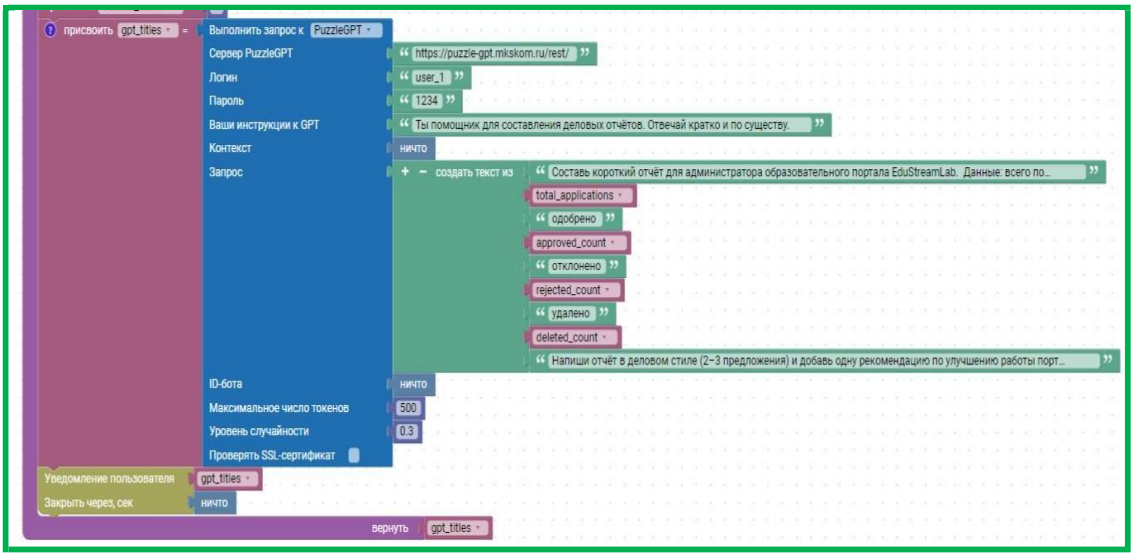
Что делаем в этом блоке: Присваиваем переменной rejected_count значение переменной count_offline.

Что возвращает блок: Возвращает количество отклонённых заявок.

Шаг 4. Присвоение значения удалённым записям

Какой блок используется: Присвоить значение переменной

Что делаем в этом блоке: Присваиваем переменной deleted_count значение 1. (Примечание: здесь происходит перезапись входящего параметра).



Шаг 5. Формирование запроса к GPT

Какой блок используется: Собрать строку

Что делаем в этом блоке: Собираем текст запроса из следующих частей:

Составь короткий отчёт для администратора образовательного портала EduStreamLab. Данные: всего подано заявок –

total_applications (общее количество заявок)

одобрено

approved_count (количество одобренных)

отклонено

rejected_count (количество отклонённых)

удалено

deleted_count (количество удалённых)

Напиши отчёт в деловом стиле (2–3 предложения) и добавь одну рекомендацию по улучшению работы портала.

Что возвращает блок: Возвращает готовую строку запроса к GPT.

Шаг 6. Выполнение запроса к GPT

Какой блок используется: Выполнить запрос к GPT

Что делаем в этом блоке: Указываем параметры подключения к GPT-серверу:

base_url – адрес сервера GPT: https://puzzle-gpt.mkskom.ru/rest/

username – имя пользователя: user_1

password – пароль: 1234

gtp_instructions – инструкция для GPT: Ты помощник для составления деловых отчётов. Отвечай кратко и по существу.

prompt_text – сформированный на шаге 5 текст запроса

max_tokens – максимальная длина ответа: 500

temperature – креативность ответа: 0.3 (низкая, ответ будет более точным и предсказуемым)

verify_ssl – проверка сертификата: "FALSE"

Что возвращает блок: Возвращает строку с текстом отчёта, сгенерированную GPT.

Шаг 7. Отображение уведомления пользователю

Какой блок используется: Уведомление пользователя

Что делаем в этом блоке: Передаём в блок сгенерированный GPT текст gpt_titles.

Что возвращает блок: Ничего не возвращает. Показывает всплывающее окно с текстом отчёта.

Шаг 8. Возврат текста отчёта

Какой блок используется: Возврат значения

Что делаем в этом блоке: Возвращаем переменную gpt_titles, содержащую сгенерированный GPT текст отчёта.

Важные замечания:

Первое. Параметр deleted_count внутри функции перезаписывается значением 1. Если нужно использовать переданное значение, эту строку следует удалить.

Второе. Температура 0.3 даёт более предсказуемые и точные ответы. Для более креативных ответов температуру можно повысить до 0.7-1.0.

Третье. Максимальная длина ответа 500 токенов – это примерно 350-400 слов, достаточно для короткого отчёта.

Четвёртое. Для работы функции требуется доступ к GPT-серверу по указанному адресу.

Пятое. Уведомление пользователя покажет всплывающее окно с текстом отчёта прямо на экране.

Главный процесс Main

Главный процесс предназначен для выполнения полного сценария робота: от открытия браузера до формирования отчёта с помощью GPT.

Что делает главный процесс:

Первое – открывает браузер и переходит на страницу авторизации.

Второе – выполняет авторизацию с указанными логином и паролем.

Третье – читает данные из Excel-файла с заявками.

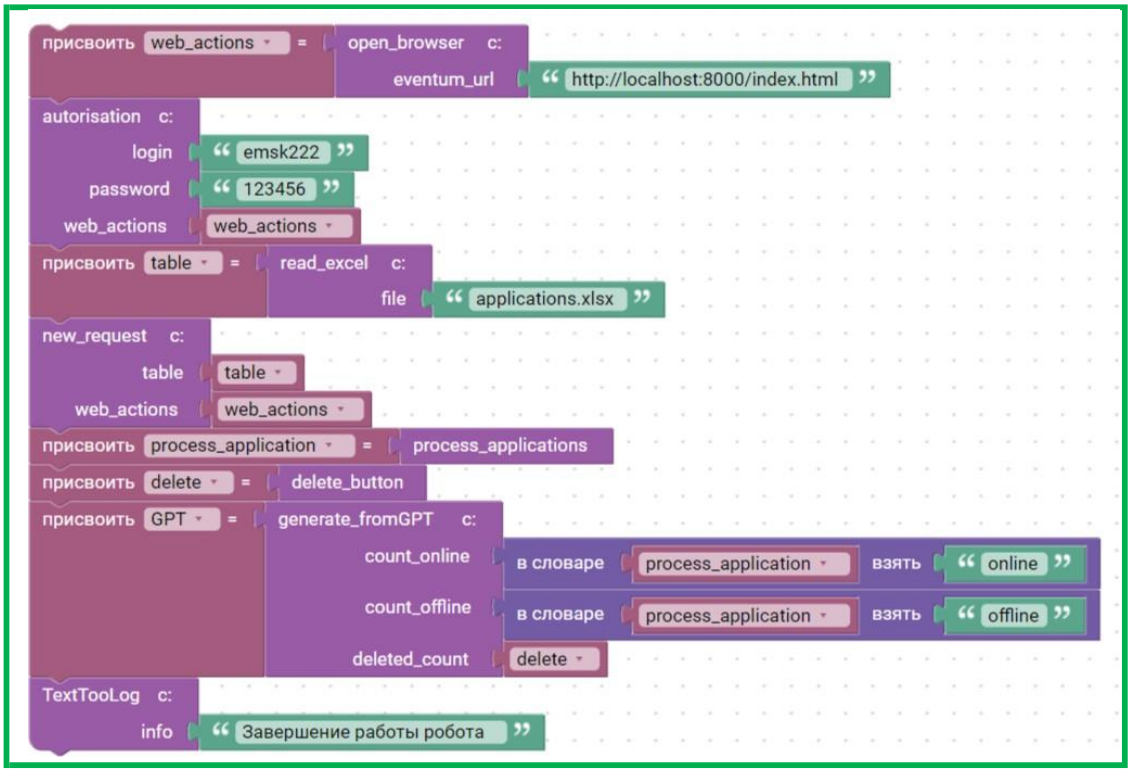
Четвёртое – заполняет новые заявки на основе данных из Excel.

Пятое – обрабатывает заявки (одобряет онлайн, отклоняет офлайн).

Шестое – удаляет запись (имитация удаления).

Седьмое – формирует отчёт с помощью GPT на основе полученной статистики.

Восьмое – записывает в лог сообщение о завершении работы робота.



Шаг 1. Открытие браузера

Какой блок используется: open_browser (наша собственная функция)

Что делаем в этом блоке: Вызываем функцию open_browser и передаём ей адрес страницы http://localhost:8000/index.html.

Что возвращает блок: Возвращает объект web_actions для управления браузером.

Шаг 2. Авторизация

Какой блок используется: autorisation (наша собственная функция)

Что делаем в этом блоке: Вызываем функцию autorisation и передаём ей:

Логин: 'emsk222'

Пароль: '123456'

Объект web_actions из шага 1

Что возвращает блок: Ничего не возвращает. Выполняет вход в систему.

Шаг 3. Чтение Excel-файла

Какой блок используется: read_excel (наша собственная функция)

Что делаем в этом блоке: Вызываем функцию read_excel и передаём ей имя файла 'applications.xlsx'.

Что возвращает блок: Возвращает таблицу с данными из Excel-файла.

Шаг 4. Заполнение новых заявок

Какой блок используется: new_request (наша собственная функция)

Что делаем в этом блоке: Вызываем функцию new_request и передаём ей:

Таблицу table из шага 3

Объект web_actions из шага 1

Что возвращает блок: Ничего не возвращает. Заполняет все заявки из Excel.

Шаг 5. Обработка заявок

Какой блок используется: process_applications (наша собственная функция)

Что делаем в этом блоке: Вызываем функцию process_applications без параметров.

Что возвращает блок: Возвращает словарь с ключами online и offline, содержащими количество обработанных заявок.

Шаг 6. Удаление записи

Какой блок используется: delete_button (наша собственная функция)

Что делаем в этом блоке: Вызываем функцию delete_button без параметров.

Что возвращает блок: Возвращает 1 при успешном удалении, 0 при ошибке.

Шаг 7. Формирование отчёта через GPT

Какой блок используется: generate_fromGPT (наша собственная функция)

Что делаем в этом блоке: Вызываем функцию generate_fromGPT и передаём ей:

Количество онлайн заявок: process_application['online']

Количество офлайн заявок: process_application['offline']

Результат удаления: delete

Что возвращает блок: Возвращает текст отчёта, сгенерированный GPT.

Шаг 8. Запись в лог о завершении

Какой блок используется: TextTooLog (наша собственная функция)

Что делаем в этом блоке: Вызываем функцию TextTooLog и передаём ей сообщение 'Завершение работы робота'.

В рамках данной практической работы была успешно реализована автоматизация процесса регистрации и обработки заявок на участие в образовательных программах с использованием RPA-робота в среде Puzzle RPA.

В ходе выполнения работы были применены передовые технологии машинного зрения (Machine Vision) и искусственного интеллекта (LLM), что позволило создать устойчивого к внешним условиям робота, способного работать в виртуальных средах и через удалённые рабочие столы.

Разработанный RPA-робот выполняет полный цикл обработки заявок: от авторизации в веб-приложении до формирования аналитического отчёта. Робот автоматически читает данные из Excel-файла, заполняет и отправляет 8 тестовых заявок, анализирует формат участия (Online или Offline) и проставляет соответствующие статусы (Approved или Rejected), удаляет одну запись и генерирует итоговый отчёт с помощью GPT.

Все этапы работы робота сопровождаются подробным логированием с помощью собственной функции TextTooLog, что позволяет отслеживать ход выполнения и при необходимости выявлять ошибки.

В результате работы робота все заявки с форматом Online получили статус Approved, все заявки с форматом Offline получили статус Rejected, одна запись была удалена, а администратор портала получил аналитический отчёт с рекомендацией по улучшению работы портала.

Таким образом, цели практической работы достигнуты, а разработанное решение может быть рекомендовано для использования в реальных бизнес-процессах компании EduStreamLab.

Практическая часть

Задание

Необходимо разработать процесс автоматизации в **Puzzle RPA**, который:

- использует **GPT для генерации наименований посылок**;
- взаимодействует с интерфейсом **только через Machine Vision**.

Допускается использование блоков Machine Vision:

- Клик по картинке
- Ожидать изображение на экране
- Найти картинку
- Сделать скриншот

УРОВЕНЬ 1 — Базовая автоматизация интерфейса

Шаг 1. Запуск приложения

Зачем выполняется шаг

Перед началом автоматизации необходимо убедиться, что интерфейс приложения загружен.

Что необходимо сделать

1. Запустить приложение.
2. Использовать блок:

Ожидать изображение на экране

Изображение:

package_field.png

Шаг 2. Генерация наименований посылок через GPT

Зачем выполняется шаг

Робот должен **самостоятельно придумать описания посылок** для разных типов доставки, используя искусственный интеллект.

Это демонстрирует интеграцию **LLM в RPA-процесс**.

Что необходимо сделать

Использовать блок:

AI → Выполнить запрос в GPT

Параметры блока

Параметр	Значение
Выполнить запрос к	PuzzleGPT
Запрос	Сгенерируй 10 наименований посылок для логистической службы. Распредели по типам: 4 Standard (документы, офисные принадлежности, одежда), 3 Express (электроника, медицинские препараты), 3 Same-day (продукты, срочные контракты). Верни только список названий без номеров, каждое название с новой строки.
Максимальное число токенов	500
Уровень случайности	0.5

Пример ответа GPT

Пакет бухгалтерской отчётности
Набор офисной канцелярии
Зимняя куртка (размер M)
Серверный блок питания
Аптечка первой помощи
Срочный договор поставки
Коробка с образцами тканей
Ноутбук в защитном кейсе
Фермерские продукты (охлаждённые)
Архивные документы за 2025 год

Шаг 3. Преобразование ответа GPT в массив

Зачем выполняется шаг

Ответ GPT возвращается как **одна строка текста**.

Чтобы использовать каждое наименование посылки отдельно, необходимо **разделить текст на массив строк**.

Что необходимо сделать

Использовать операцию:

Разделение строки по символу перевода строки

Результат:

массив из **10 наименований посылок**.

Шаг 4. Чтение данных из Excel

Использовать блок:

Excel → Read Range

Источник:

deliveries.xlsx

Шаг 5. Заполнение поля Package

1. Найти изображение:
   package_field.png
2. Выполнить:
   Клик по картинке
3. Ввести наименование посылки из **массива GPT**.

Шаг 6. Заполнение поля Driver

1. Найти изображение:
   driver_field.png
2. Выполнить:
   Клик по картинке
3. Ввести значение из Excel.

Шаг 7. Выбор Delivery Type

1. Найти:
   delivery_type_dropdown.png
2. Кликнуть по списку.
3. Найти изображение:
   type_standard.png
   type_express.png
   type_sameday.png
4. Кликнуть по соответствующему значению.

Шаг 8. Выбор Priority

1. Найти:
   priority_dropdown.png
2. Кликнуть по списку.
3. Найти изображение:
   priority_low.png
   priority_medium.png
   priority_high.png
4. Кликнуть по соответствующему значению.

Шаг 9. Добавление доставки

Найти изображение:
add_button.png
Выполнить:
Клик по картинке
Шаги **5–9** выполняются **в цикле 10 раз**.

УРОВЕНЬ 2 — Обработка доставок

Шаг 10. Назначение водителя для каждой доставки

1. Последовательно выбрать каждую строку в таблице.
2. Найти:
   assign_driver_button.png
3. Выполнить:
   Клик по картинке
   (Данный шаг выполняется в цикле для всех 10 записей)

Шаг 11. Найти доставку с приоритетом High

Использовать блок:
Найти картинку
Изображение:
priority_high.png

Шаг 12. Выбор строки таблицы

После обнаружения изображения выполнить:
Клик по картинке

Шаг 13. Отправка доставки (Dispatch)

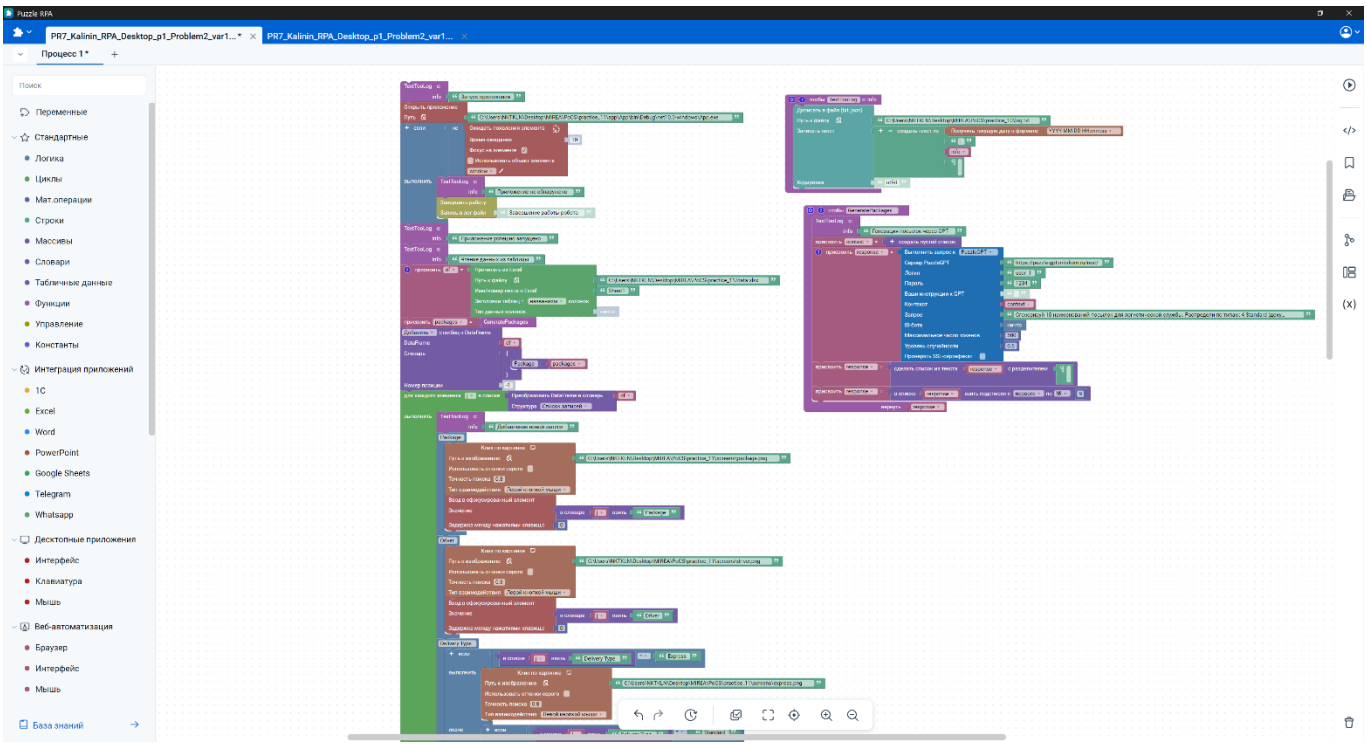
Найти:
dispatch_delivery_button.png
Выполнить:
Клик по картинке
(Статус меняется на Dispatched)

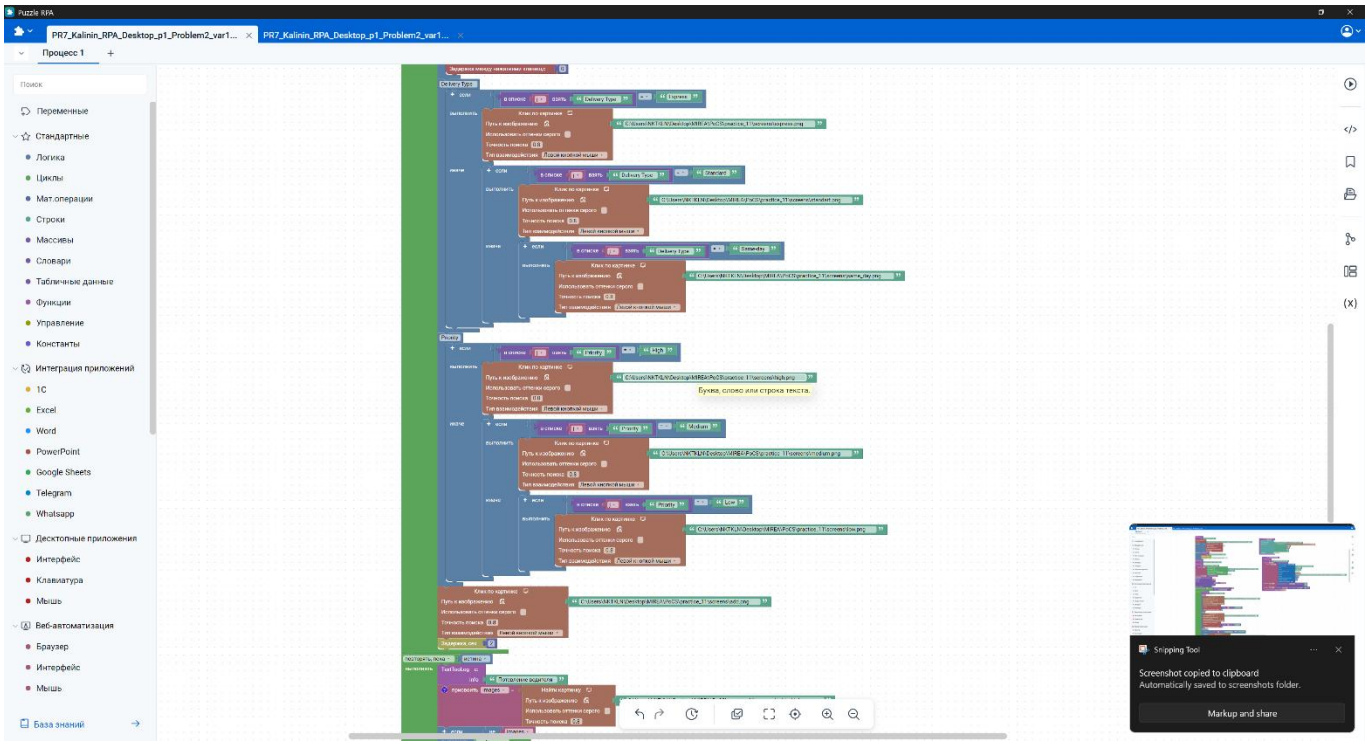
Шаг 14. Подтверждение действия (при необходимости)
Использовать:
Ожидать изображение на экране
confirm_dialog.png
Найти:
confirm_button.png
Выполнить:
Клик по картинке

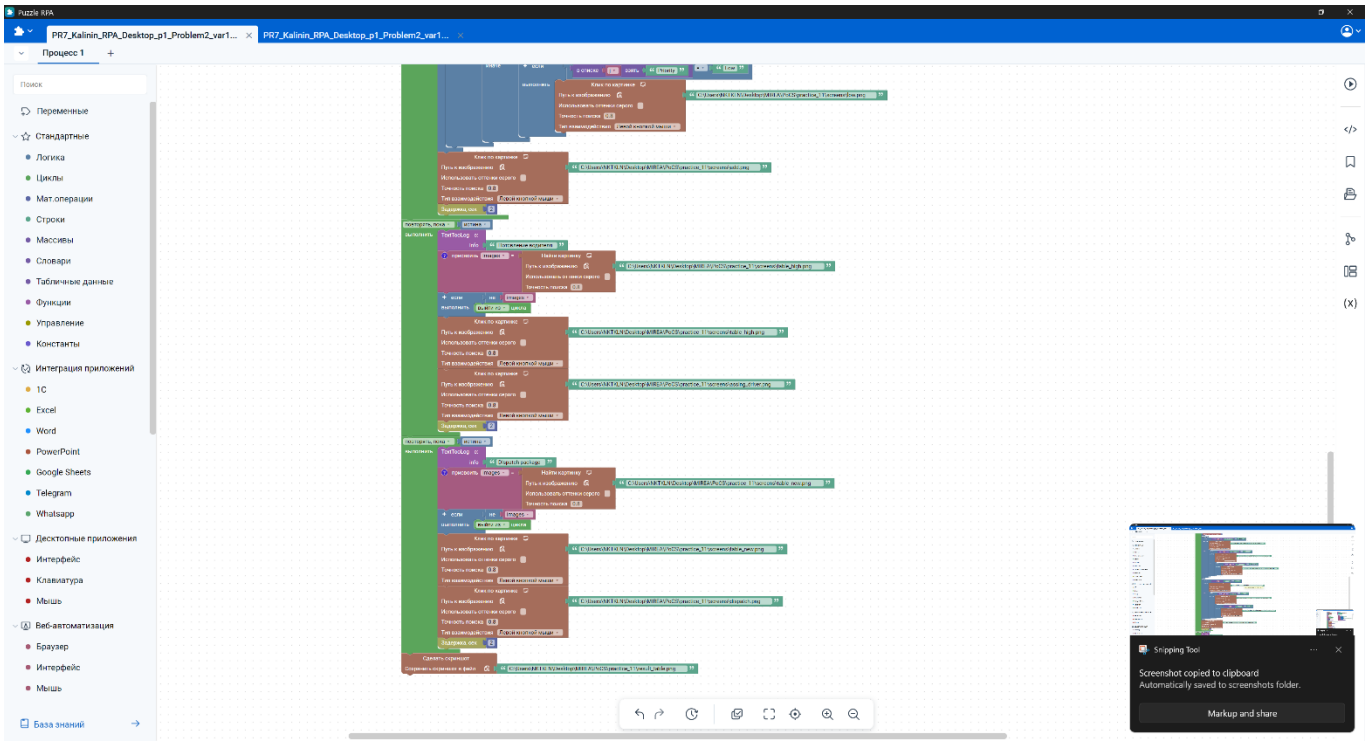
Шаг 15. Сохранение результата
Использовать блок:
Сделать скриншот
Файл:
result_table.png

📊 **Ожидаемый результат**
После выполнения процесса:

- **10 наименований посылок сгенерированы GPT**;
- создано **10 доставок**;
- для всех доставок **выполнено действие Assign Driver**;
- доставки с приоритетом **High → Dispatched**;
- сохранён **скриншот таблицы**.




Puzzle RPA
PR7_Kalinin_RPA_Desktop_p1_Problem2_var1...
Процесс 1
Переменные
Стандартные
Логика
Циклы
Мат.операции
Строки
Массивы
Словари
Табличные данные
Функции
Управление
Константы
Интеграция приложений
1С
Excel
Word
PowerPoint
Google Sheets
Telegram
Whatsapp
Десктопные приложения
Интерфейс
Клавиатура
Мышь
Веб-автоматизация
Браузер
Интерфейс
Мышь
База знаний
Буква, слово или строка текста.
Snipping Tool
Screenshot copied to clipboard
Automatically saved to screenshots folder.
Markup and share


Puzzle RPA
PR7_Kalinin_RPA_Desktop_p1_Problem2_var1...
Процесс 1
Переменные
Стандартные
Логика
Циклы
Мат.операции
Строки
Массивы
Словари
Табличные данные
Функции
Управление
Константы
Интеграция приложений
1С
Excel
Word
PowerPoint
Google Sheets
Telegram
Whatsapp
Десктопные приложения
Интерфейс
Клавиатура
Мышь
Веб-автоматизация
Браузер
Интерфейс
Мышь
База знаний
Snipping Tool
Screenshot copied to clipboard
Automatically saved to screenshots folder.
Markup and share

Система регистрации доставок

Информация о доставке

Package:

Driver:

Delivery Type: Standard Express Same-day

Priority: Low Medium High

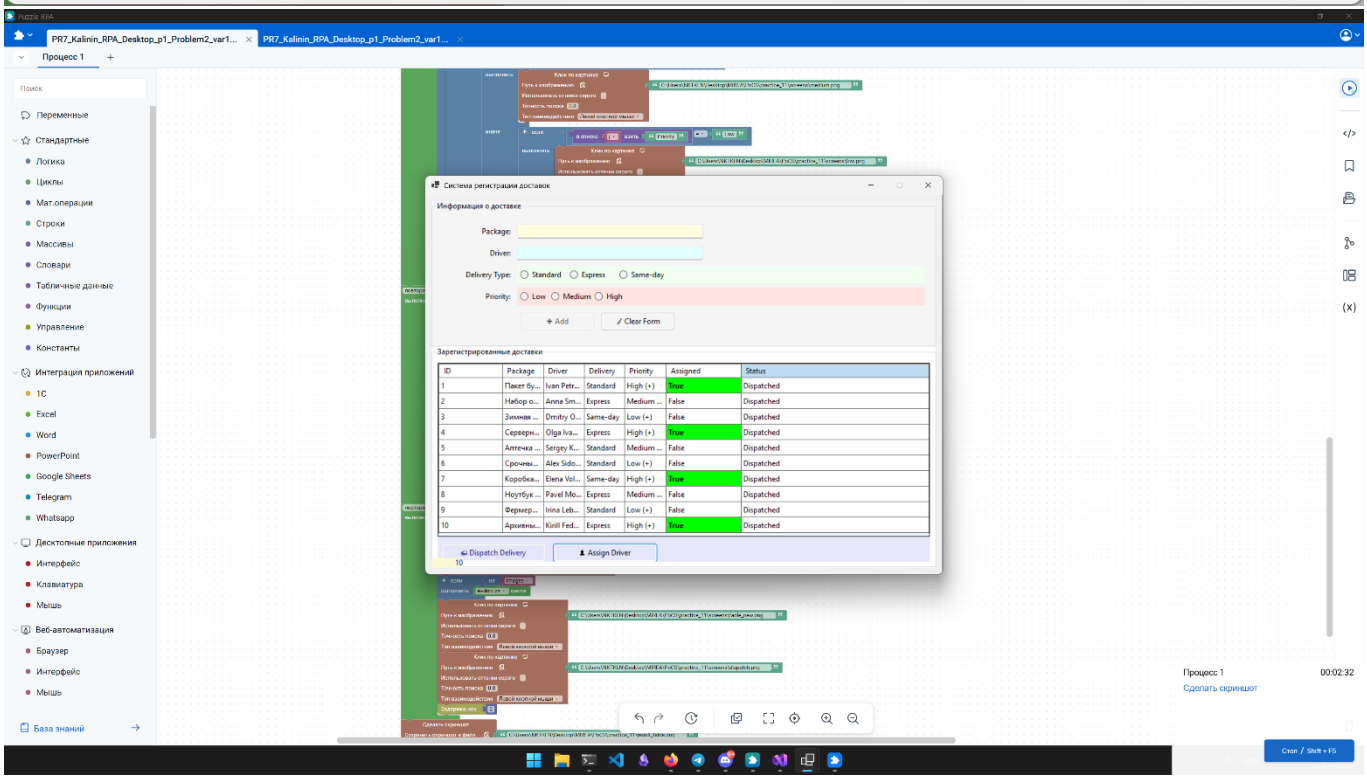
+ Add Clear Form

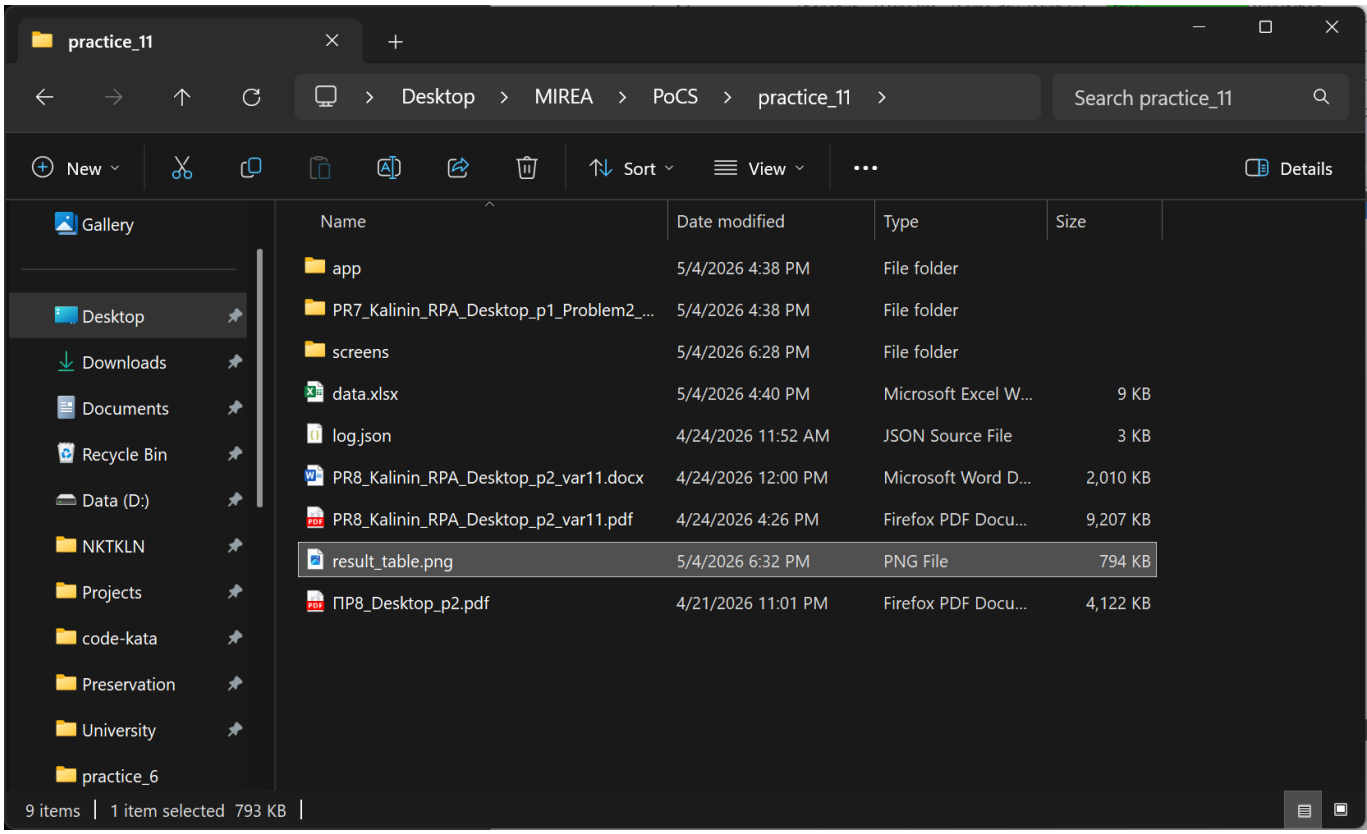
Зарегистрированные доставки

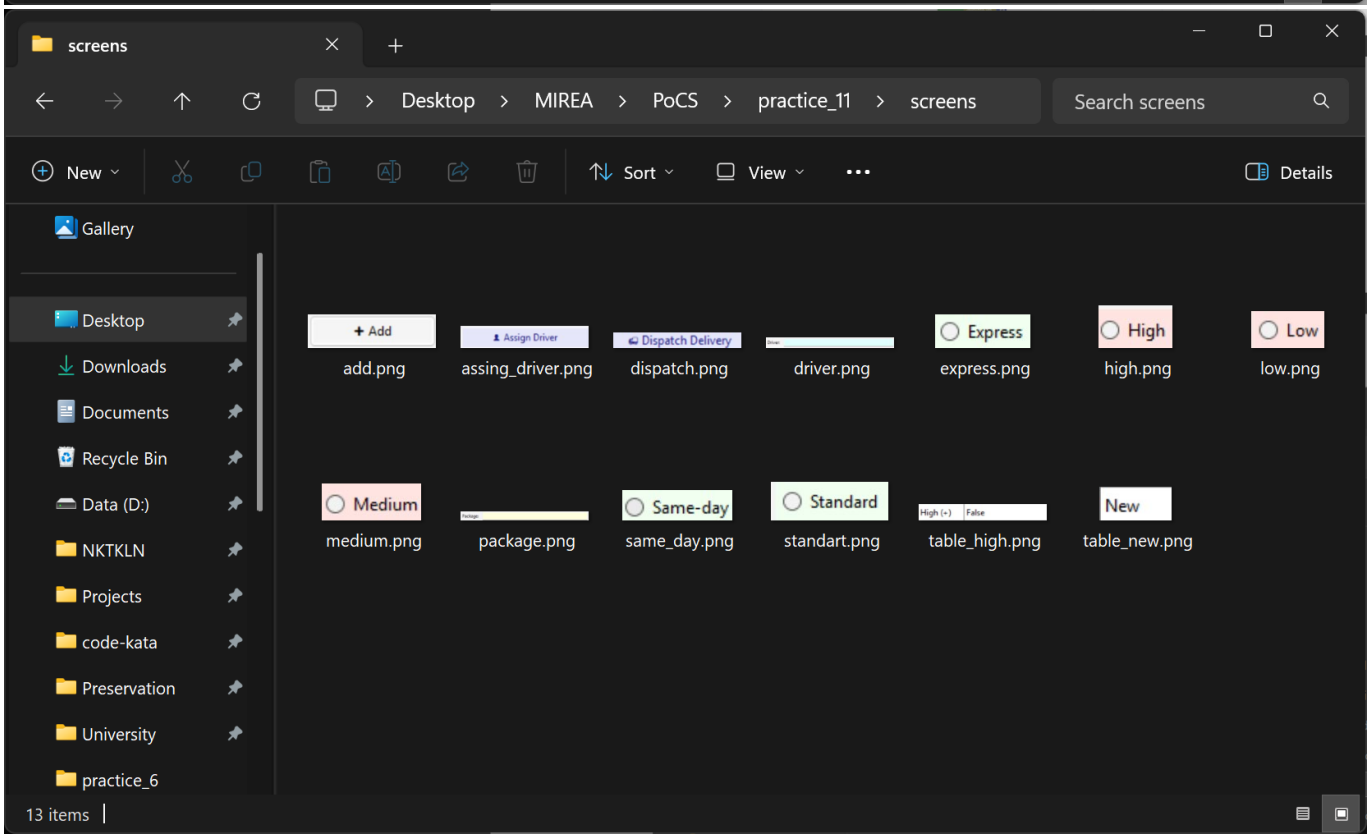
ID	Package	Driver	Delivery	Priority	Assigned	Status
1	Пакет бу...	Ivan Petr...	Standard	High (+)	True	Dispatched
2	Набор о...	Anna Sm...	Express	Medium ...	False	Dispatched
3	Зимняя ...	Dmitry O...	Same-day	Low (+)	False	Dispatched
4	Серверн...	Olga Iva...	Express	High (+)	True	Dispatched
5	Аптечка ...	Sergey K...	Standard	Medium ...	False	Dispatched
6	Срочны...	Alex Sido...	Standard	Low (+)	False	Dispatched
7	Коробка...	Elena Vol...	Same-day	High (+)	True	Dispatched
8	Ноутбук ...	Pavel Mo...	Express	Medium ...	False	Dispatched
9	Фермер...	Irina Leb...	Standard	Low (+)	False	Dispatched
10	Архивны...	Kirill Fed...	Express	High (+)	True	Dispatched

Dispatch Delivery Assign Driver

10







DeepSeek

https://chat.deepseek.com/share/k3i06bew7w8agexahm

GitHub https://github.com/NKTKLN/mirea-puzzle-rpa/tree/main/practice_11